Principles of Programming Languages
Course Notes

Elaine Li

March 3, 2026

Preface

This document contains the lecture notes for the NYU undergraduate course CSCI-UA.0480-055, "Principles of Programming Languages", in fall 2023. The document will be extended throughout the semester. So please stay tuned!

Course Summary

Computing professionals have to learn new programming languages all the time. This course teaches the fundamental principles of programming languages that enable you to learn new languages quickly and help you decide which one is best suited for a given task.

We will explore new ways of viewing computation and programs, and new ways of approaching algorithmic problems, making you better programmers overall. The topics covered in this course include:

- recursion and induction
- algebraic data types and pattern matching
- higher-order functions
- continuations and tail recursion
- programming language syntax and semantics
- type systems
- monads
- objects and classes

We will explore this material by building interpreters for programming languages of increasing complexity. The course will thus be accompanied by extensive programming assignments. We will use the programming language Scala for these assignments, which you will also learn in this course.

Contents

1 Scala Basics **11**
- 1.1 Getting Started . . . 11
  - 1.1.1 Compiling and Running Scala Applications . . . 11
  - 1.1.2 The Scala REPL and Worksheets in the IDE . . . 12
- 1.2 Scala Crash Course . . . 13
  - 1.2.1 Expressions, Values, and Types . . . 13
  - 1.2.2 Names . . . 14
  - 1.2.3 Functions . . . 15
  - 1.2.4 Scopes . . . 17
  - 1.2.5 Tuples . . . 17
- 1.3 Recursion . . . 18
  - 1.3.1 Evaluating Recursive Functions . . . 19
  - 1.3.2 Tail Recursion . . . 20
- 1.4 Classes and Objects . . . 23
  - 1.4.1 Classes, Fields, and Methods . . . 23
  - 1.4.2 Overriding Methods . . . 25
  - 1.4.3 Singleton and Companion Objects . . . 26
  - 1.4.4 The `apply` Method . . . 27
- 1.5 Algebraic Data Types . . . 28
  - 1.5.1 Enumerations . . . 28
  - 1.5.2 Pattern Matching . . . 30
  - 1.5.3 Binding Names in Patterns . . . 32
  - 1.5.4 Pattern Guards . . . 32
  - 1.5.5 Exhaustiveness Checks . . . 33
  - 1.5.6 Option Types . . . 35

2 Foundations **37**
- 2.1 Notation . . . 37
- 2.2 Structural Recursion and Induction . . . 40
  - 2.2.1 Structurally Recursive Definitions . . . 40
  - 2.2.2 Recursive Definitions of Functions . . . 42
  - 2.2.3 Structural Induction . . . 43
  - 2.2.4 Well-founded Induction (optional) . . . 45
- 2.3 Lists . . . 48

2.3.1 Defining Lists using Structural Recursion 48
2.3.2 Functions on Lists 49
2.3.3 Proving Properties of Functions on Lists 50

3 Syntax **53**
3.1 Concrete Syntax (optional) 53
3.1.1 Formal Languages 54
3.1.2 Context-Free Languages and Grammars 55
3.1.3 Backus-Naur-Form 56
3.1.4 Eliminating Ambiguity 56
3.1.5 Regular Languages 59
3.2 Abstract Syntax 60
3.2.1 Abstract Syntax Trees 60
3.2.2 Environments and Expression Evaluation 63
3.2.3 Substitutions 64
3.3 Binding and Scoping 65
3.3.1 Expressions with Constant Declarations 65
3.3.2 Evaluation with Static Binding 69
3.3.3 Substitutions and Bindings 70

4 Semantics **75**
4.1 Big-Step Structural Operational Semantics 75
4.1.1 Defining the Big-Step SOS 76
4.1.2 Short-Circuit Evaluation 79
4.1.3 Interpreting Big-Step Derivations 81
4.2 Small-Step Operational Semantics 83
4.2.1 Step-Wise Expression Evaluation 83

5 Procedural Abstraction **89**
5.1 Functions and Function Calls 89
5.2 Currying and Partial Function Application 92
5.3 Operational Semantics of Function Calls 93
5.3.1 Environment-based Semantics with Dynamic Binding . . 93
5.3.2 Dynamic Type Errors 94
5.3.3 Dynamic vs. Static Binding 95
5.3.4 Substitution-based Semantics with Static Binding 98
5.4 Higher-Order Functions 100
5.4.1 Abstracting from Computations 100
5.4.2 Realizing `for` Loops with Higher-Order Functions 103
5.5 Higher-Order Functions and Collections in Scala 104
5.5.1 Higher-Order Functions in Scala 104
5.5.2 Curried Functions in Scala 105
5.5.3 Higher-Order Functions on Lists 106
5.5.4 Folding Lists 109
5.5.5 Scala's Collection Classes 111
5.6 Computability (optional) 112

5.6.1 Church Encodings 113
5.6.2 Expressing Recursion 116

List of Figures

4.1 Inference rules that define the big-step SOS of our expression language . . . 80
4.2 Do rules for the small-step SOS of our expression language . . . 85
4.3 Search rules for the small-step SOS of our expression language . 85

5.1 Inference rules that define the big-step SOS of our language with functions. The modifications and additions to the rules from Figure 4.1 in Section 4.1.1 are highlighted. . . . 96
5.2 Big-step operational semantics: dynamic type error rules 97
5.3 Inference rules that define the substitution-based big-step SOS for static binding semantics of our language with functions 101
5.4 Substitution-based big-step operational semantics: dynamic type error rules . . . 102

Chapter 1

Scala Basics

1.1 Getting Started

In the following, we assume that you have installed sbt and Intellij Idea with the Scala plugin. If you have not yet done so, please do it now. You can find a link to the installation instructions on the course web site on Brightscape.

1.1.1 Compiling and Running Scala Applications

Compiling and running Scala applications works similar to Java. For example, you can open a text editor and type in the following Scala code:

```
package greeter

object Hello extends App:
  def main(args: Array[String])
    println("Hello␣World!")
```

This code creates an object `Hello` in the package `greeter`. The object `Hello` contains a method called `main`, which means that `Hello` can serve as the entry point of a Scala application that calls `main` upon start, providing the command line arguments via the parameter `args`. When this application is started, the object `main` prints the message `"Hello␣World!"` on standard output.

The above code is roughly equivalent to the following Java code:

```
package greeter;

public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello␣World!");
  }
}
```

You can safe the Scala code, say, in a file called `Hello.scala`, and then compile it with the Scala compiler. To do this, open a command prompt, go to the directory where you saved the file, and type `scalac Hello.scala`. This will create a file `Hello.class`, which contains the compiled byte code of the object `Hello`.

To execute the program, type `scala greeter.Hello` in your terminal. This will start the Scala runtime environment, which will execute the byte code in `Hello.class` using the Java virtual machine. You should see the message `"Hello␣World!"` printed in your terminal.

If you are using the Intelij Idea IDE, you can import the `in-class-code` project following the instructions provided in the `README.md` file of the repository. The repository contains a file `ScalaGreeter.scala`, which you can find in the package `popl.class03`. To compile and run the application, right-click on the file and select "Run 'ScalaGreeter' ". This should print `"Hello Scala"` in the output view at the bottom of the window.

1.1.2 The Scala REPL and Worksheets in the IDE

If you want to experiment with the Scala language, it is quite cumbersome to write an extra application for every small code snippet that you would like to execute. To make life easier, Scala provides a useful tool called a read-eval-print loop, or *REPL* for short. The Scala REPL is essentially a command line calculator on steroids. It allows you to type arbitrary Scala code in a terminal. The code is then evaluated and the result of the evaluation is printed in the terminal.

To start the Scala REPL, open a terminal and execute `scala`. This will start the REPL and a message similar to the following should appear:

```
Welcome to Scala 3.3.0 (18.0.2-ea, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala>
```

Now, you can type a Scala expression. For example typing 3 + 4 yields

```
scala> 3 + 4
val res0: Int = 7
```

You can exit the REPL by typing `:quit` or by pressing `Ctrl-d` (respectively, `Cmd-d` on OS X).

If you only installed sbt and Intelij Idea, you can start the REPL by typing `sbt console` in a terminal. Intelij Idea provides a feature similar to the REPL, called *Worksheets*. You can use this feature as follows:

1. Start Intelij Idea. (In the following, I assume you have previously imported the `in-class-code` project with the `popl` package.)
2. Right-click the `popl` package in the package explorer and choose New/Scala Worksheet. Name the worksheet "MyWorksheet" and press Finish.

3. An empty Scala source file called `MyWorksheet.sc` will open in the editor window.

4. You can now type Scala expressions in the editor window. Each time you click the "Evaluate Worksheet" button at the top of the editor view (the button with the green arrow icon), the expressions in the file are evaluated and the result of the evaluation appear in a separate view next to the editor.

5. If you click the 'Show worksheet settings' button (the button with the wrench icon), you can set the worksheet to Interactive Mode, which will cause all expressions to be evaluated automatically as you type.

1.2 Scala Crash Course

In the following, we assume that you have started the Scala REPL. Though, (almost) all of these steps can also be done in a worksheet.

1.2.1 Expressions, Values, and Types

After you type an expression in the REPL, such as `3 + 4`, and hit enter:

```
scala> 3 + 4
```

The interpreter will print:

```
val res0: Int = 7
```

This line includes:

- the keyword **`val`**, indicating that you have defined a new value resulting from evaluating the expression.
- an automatically generated name `res0` that is *bound* to that new value,
- a colon `:`, followed by the type `Int` of the expression,
- an equals sign `=`,
- the value `7` resulting from evaluating the expression.

The type `Int` names the class `Int` in the package `scala`. Packages in Scala partition the global name space and provide mechanisms for information hiding, similar to Java packages. Values of class `Int` correspond to values of Java's primitive type `int` (Scala makes no difference between primitive and object types). More generally, all of Java's primitive types have corresponding classes in the `scala` package.

We can reuse the automatically generated name `res0` to refer to the computed value in subsequent expressions (this only works in the REPL but not in a worksheet):

```
scala> res0 * res0
val res1: Int = 9
```

Java's ternary conditional operator `? :` has an equivalent in Scala, which looks as follows:

```
scala> if res1 > 10 then res0 - 5 else res0 + 5
val res2: Int = 2
```

In addition to the `? :` operator, Java also has if-then-else statements. Scala, on the other hand, is a functional language and makes no difference between expressions and statements: every programming construct is an expression that evaluates to some value. In particular, we can use if-then-else expressions where we would normally use if-then-else statements in Java.

```
scala> if res1 > 2 then println("Large!")
       else println("Not␣so␣large!")
Large!
```

Note that the result value is not automatically bound to a name in this case. The if-then-else expression still evaluates to the value `()`, which is of type `Unit`. This type indicates that the sole purpose of evaluating the expression is the side-effect of the evaluation (here, printing a message). In other words, in Scala, statements are expressions of type `Unit`. Thus, the type `Unit` is similar to the type void in Java (which however, has no values). The value `()` is the only value of type `Unit`.

1.2.2 Names

We can use the **`val`** keyword to give a user-defined name to a value, so that we can subsequently refer to it in other expressions:

```
scala> val x = 3
val x: Int = 3
scala> x * x
val res0: Int = 9
```

Note that Scala automatically infers that `x` has type `Int`. Sometimes, automated type inference fails, in which case you have to provide the type yourself. This can be done by annotating the declared name with its type:

```
scala> val x: Int = 3
val x: Int = 3
```

A **`val`** is similar to a `final` variable in Java. That is, you cannot reassign it another value:

```
-- [E052] Type Error: -------------------------------------------------------
1 |x = 5
  |^^^^^
  |Reassignment to val x
```

Scala also has an equivalent to standard Java variables, which can be reassigned. These are declared with the **var** keyword

```
scala> var y = 5
var y: Int = 5
scala> y = 3
y: Int = 5
```

The type of a variable is the type inferred from its initialization expression. It is fixed throughout the lifetime of the variable. Attempting to reassign a value of incompatible type results in a type error:

```
-- [E007] Type Mismatch Error: ------------------------------------------------
1 |y = "Hello"
  |    ^^^^^^^
  |    Found:    ("Hello" : String)
  |    Required: Int
```

For the time being, we will pretend that variables do not exist. Repeat after me: **val**s are gooood! **var**s are baaaad!

1.2.3 Functions

Here is how you write functions in Scala:

```
scala> def max(x: Int, y: Int): Int =
         if x > y then x else y
def max(x: Int, y: Int): Int
```

Function definitions start with **def**, followed by the function's name, in this case `max`. After the name comes a comma separated list of parameters enclosed by parenthesis, here `x` and `y`. Note that the types of parameters must be provided explicitly since the Scala compiler does not infer parameter types. The type annotation after the parameter list gives the result type of the function. The result type is followed by the equality symbol, indicating that the function returns a value, and the body of the function which computes that value. The expression in the body that defines the result value is enclosed in curly braces.

If the defined function is not recursive, as is the case for `max`, the result type can be omitted because it is automatically inferred by the compiler. However, it is often helpful to provide the result type anyway to document the signature of the function.

Once you have defined a function, you can call it using its name:

```
scala> max(6, 3)
val res3: Int = 3
```

Naturally, you can use values and functions that are defined outside of a function's body in the function's body:

```
scala> val pi = 3.14159
val pi: Double = 3.14159

scala> def circ(r: Double) = 2 * pi * r
def circ(x: Double): Double
```

You can also nest value and function definitions:

```
scala> def area(r: Double) =
         val pi = 3.14159
         def square(x: Double) = x * x
         pi * square(r)

def area(Double): Double
```

Note that the scope of the body of a function definition is determined automatically by the indentation level. For instance, the following code does not compile because the last line is no longer interpreted to be part of the body of the function `area`:

```
scala> def area(r: Double) =
         val pi = 3.14159
         def square(x: Double) = x * x
       pi * square(r)
-- [E006] Not Found Error: ------------------------------------------------
4 |pi * square(r)
  |^^
  |Not found: pi
```

If you have a longer function definition, it can be helpful to mark the end of the function explicitly using an **end** marker, followed by the name of the function:

```
def area(r: Double) =
  val pi = 3.14159
  def square(x: Double) = x * x
  pi * square(r)
end area
```

Recursive functions can be written as expected. For example, the following function `fac` computes the factorial numbers:

```
scala> def fac(n: Int): Int = if n <= 0 then 1 else n*fac(n-1)
def fac(n: Int): Int

scala> fac(5)
val res4: Int = 120
```

1.2.4 Scopes

You can use curly braces `{ ... }` to create block scopes. Scala's scoping rules are almost identical to Java's:

```
val a = 5
// only a in scope
{
  val b = 4
  // b and a in scope

  def f(x: Int) =
    // f, x, b, and a in scope
    a * x + b

  // f, b, and a in scope
}
// only a in scope
```

There are two difference to Java, though. First, the scope of a name extends the entire block in which it is defined. Using a name before its definition leads to an error:

```
val a = 3
{
  val b = a // Refers to 'a' defined on the next line.
  val a = 4 // Does not compile.
}
```

However, unlike Java, Scala allows you to redefine names in nested scopes, thereby shadowing definitions in outer scopes.

```
val a = 3
{
  val a = 4 // Shadows outer definition of a.
  a + a     // Yields 8
}
```

As in Java, you cannot redefine a name in the same scope:

```
val a = 3
val a = 4 // Does not compile.
```

1.2.5 Tuples

Scala provides ways to create new compound data types without requiring you to define simplistic data-heavy classes. One of the most useful of these constructs are *tuples*. A tuple combines a fixed number of items together so that they can

be passed around as a whole. The individual items can have different types. For example, here is a tuple holding an `Int` and a `String`:

```
scala> val p = (1, "banana")
val p: (Int, String) = (1, "banana")
```

and here is a tuple holding three items: two `String`s and a `Double` value:

```
scala> val q = ("apple", "pear", 1.0)
val q: (String, String, Double) = (apple, pear, 1.0)
```

To access the items of a tuple, you can use method `_1` to access the first item, method `_2` to access the second, and so on:

```
scala> p._1
val res5: Int = 1

scala> p._2
val res6: String = banana
```

Additionally, you can assign each element of the tuple to its own **`val`**:

```
scala> val (fst, snd) = p
val fst: Int = 1
val snd: String = banana
```

Be aware that tuples are not automatically decomposed when you pass them to functions:

```
def f(x: Int, s: String) = x

f(p._1, p._2) // Works.
f(p) // Does not compile.

def g(p: (Int, String)) = p._1

g(p) // Works.
g((1, "banana")) // Works.
g(1, "banana") // Works.
```

1.3 Recursion

Recursion will be our main device for expressing unbounded computations. In the following, we study how recursive functions are evaluated. We will further see that there is a close connection between certain recursive functions and loops in imperative programs.

1.3.1 Evaluating Recursive Functions

Consider the following function which computes the sum of the integer values in the interval given by the parameters `a` and `b`.

```
def sum(a: Int, b: Int): Int =
  if a < b then a + sum(a + 1, b) else 0
```

How are calls to such functions evaluated? Conceptually, we can think of the evaluation of a Scala expression as a process that rewrites expressions into simpler expressions. This rewriting process terminates when we obtain an expression that cannot be further simplified, e.g., an integer number. Expressions that cannot be simplified further are called *values*. Concretely, if we have a function call such as `sum(1 + 1, 0 + 2)`, we proceed as follows to compute a value using rewriting:

- First, we rewrite the call expression by rewriting the arguments of the call until they are reduced to values. In our example, this step yields the simplified call expression `sum(2, 2)`.
- Next, we replace the entire call expression by the body of the function. At the same time, we replace the formal parameters occurring in the function body (i.e., the occurrences of `a` and `b` in the example) by the actual arguments of the call. In our example, this step yields the expression

  ```
  if 2 < 2 then 2 + sum(2 + 1, 2) else 0
  ```

- Finally, we continue rewriting the function body recursively in the same manner until we obtain a value that cannot be simplified further. In our example, this process eventually terminates, producing the result value `0`.

Here is how we compute the value of `sum(1, 4)` using rewriting:

```
sum(1, 4)
-> if 1 < 4 then 1 + sum(1 + 1, 4) else 0
-> if true then 1 + sum(1 + 1, 4) else 0
-> 1 + sum(1 + 1, 4)
-> 1 + sum(2, 4)
-> 1 + (if 2 < 4 then 2 + sum(2 + 1, 4) else 0)
-> 1 + (if true then 2 + sum(2 + 1, 4) else 0)
-> 1 + (2 + sum(2 + 1, 4))
-> 1 + (2 + sum(3, 4))
-> 1 + (2 + (if 3 < 4 then 3 + sum(3 + 1, 4) else 0))
-> 1 + (2 + (if true then 3 + sum(3 + 1, 4) else 0))
-> 1 + (2 + (3 + sum(3 + 1, 4)))
-> 1 + (2 + (3 + sum(4, 4)))
-> 1 + (2 + (3 + (if 4 < 4 then 4 + sum(4 + 1, 4) else 0)))
-> 1 + (2 + (3 + (if false then 4 + sum(4 + 1, 4) else 0)))
-> 1 + (2 + (3 + 0))
```

```
-> 1 + (2 + 3)
-> 1 + 5
-> 6
```

We refer to this sequence of rewriting steps as an *execution trace*.

Termination. Does the rewriting process always terminate and produce a finite execution trace? Consider the following recursive function:

```
def loop(x: Int): Int = loop(x)
```

If we evaluate, e.g., the call `loop(0)`, we obtain an infinite rewriting sequence:

```
loop(0) -> loop(0) -> loop(0) -> ...
```

In order to guarantee termination of a recursive function, we have to make sure that each recursive call makes progress according to some progress measure. For example, in the recursive call to the function `sum` in our example above, the difference `b - a` between the arguments decreases with every recursive call. This means that `b - a` will eventually reach 0 or become negative. At this point, we take the `else` branch in the body of `sum` and the recursion stops. For our non-terminating function `loop`, it is impossible to find such a progress measure.

1.3.2 Tail Recursion

If we apply the function `sum` to larger intervals we observe the following:

```
scala> sum(1, 1000000)
java.lang.StackOverflowError
...
```

The problem is that a call to a function requires the Scala runtime environment to allocate stack space that stores the arguments of the call and any intermediate results obtained during the evaluation of the function body in memory. For the function `sum`, the intermediate results of the evaluation must be kept on the stack until the final recursive call returns. We can see this nicely in the execution trace for the call `sum(1, 4)`. The length of the expression that we still need to simplify grows with each recursive call:

```
sum(1, 4)
-> ...
-> 1 + sum(2, 4)
-> ...
-> 1 + (2 + sum(2 + 1, 4))
-> ...
-> 1 + (2 + (3 + sum(2 + 1, 4)))
-> ...
-> 6
```

Only when the final call to `sum` has returned, can we simplify the entire expression to a value.

During execution of a Scala expression, the arguments of functions that have been called, but have not yet returned, are maintained on the *call stack*. The stack space that is needed for evaluating a call `sum(a, b)` grows linearly with the recursion depth, which is given by the size of the interval `b - a`. Since the Scala runtime environment only reserves a relatively small amount of memory for the call stack, a call to `sum` for large interval sizes runs out of stack space. This is signaled by a `StackOverflowError` exception.

Can we implement the function `sum` so that it only requires constant space? To this end, consider the following *imperative* implementation of `sum`, which uses a **`while`** loop and mutable variables to perform the summation:

```
def sumImp(a: Int, b: Int): Int =
  var acc = 0
  var i = a
  while i < b do
    acc = i + acc
    i = i + 1
  acc
```

This implementation requires only constant space, since it involves only a single function call. Moreover, the execution of a single loop iteration for the summation does not allocate memory that persists across iterations. The intermediate results are stored in the variables `i` and `acc`, which are reused in each iteration. Unfortunately, this implementation uses mutable variables. Mutable variables make it more difficult to reason about the correctness of the code. However, we can turn the imperative **`while`** loop into a recursive function by hoisting the loop counter `i` and accumulator `acc` to function parameters:

```
def loop(acc: Int, i: Int, b: Int): Int =
  if i < b then loop(i + acc, i + 1, b) else acc

def sumTail(a: Int, b: Int): Int =
  loop(0, a, b)
```

Note how the function `loop` closely mimics the **`while`** loop in the imperative implementation without relying on mutable variables. We simply pass the new values that we obtain for the loop counter `i` and the accumulator `acc` to the recursive call of `loop`.

The function `loop` has an important property: the recursive call to `loop` in the *then* branch of the conditional expression is the final computation that is performed before the function returns. That is, in the recursive case, the function directly returns the result of the recursive call. We refer to functions in which all recursive calls are of this form as *tail-recursive* functions. Contrast the new implementation of `sum` with our original implementation, which added `a` to the result of the recursive call and was therefore not tail-recursive. The tail recursive implementation has an interesting effect on the execution trace:

```
sumTail(1, 4)
   -> loop(0, 1, 4)
   -> if 1 < 4 then loop(1 + 0, 1 + 1, 4) else 0
   -> if true then loop(1 + 0, 1 + 1, 4) else 0
   -> loop(1, 2, 4)
   -> if 2 < 4 then loop(2 + 1, 2 + 1, 4) else 1
   -> if true then loop(2 + 1, 2 + 1, 4) else 1
   -> loop(3, 3, 4)
   -> if 3 < 4 then loop(3 + 3, 3 + 1, 4) else 3
   -> if true then loop(3 + 3, 3 + 1, 4) else 3
   -> loop(6, 3, 4)
   -> if 4 < 4 then loop(4 + 6, 4 + 1, 4) else 6
   -> if false then loop(4 + 6, 4 + 1, 4) else 6
   -> 6
```

Observe that the size of the expressions that we obtain throughout the trace does not grow with the recursion depth. This is because the tail-recursive call to `loop` is the final computation that is performed in the body of `loop`, before the function returns.

To simplify our implementation, we can move the declaration of the function `loop` inside the body of the function `sumTail`:

```
def sumTail(a: Int, b: Int): Int =
  def loop(acc: Int, i: Int): Int =
    if i < b then loop(i + acc, i + 1) else acc
  loop(0, a)
```

Note that in this version, we have dropped the third parameter `b` of the first version of the function `loop` since it is just passed to the recursive call without change. The occurrence of `b` in the body of the new nested version of `loop` now always refers to the parameter `b` of the outer function `sumTail`.

For tail-recursive functions, the stack space that is allocated for the current call can be reused by the recursive call. In particular, the memory that is needed to store the arguments of the current call can be reused to store the arguments of the recursive call. By reusing the current stack space, we effectively turn the recursive function back into an imperative loop. This optimization is referred to as *tail call elimination*. Many modern compilers, including the Scala compiler, automatically eliminate tail calls. Thus, tail-recursive functions are guaranteed to execute in constant stack space. We can test this feature by rerunning the tail-recursive version of `sum` for large interval sizes:

```
scala> sumTail(1, 1000000)
val res0: Int = 704982704
```

This time the function terminates normally without throwing an exception.

With tail call elimination we get the best of both worlds: we obtain the efficiency of an imperative implementation and the simplicity of a functional implementation. If you are unsure about how to write a tail-recursive function, it

is often helpful to first write the function using a **while** loop and then transform the loop into a tail-recursive function, as we have done above. Once you get more used to functional programming, you will find writing tail-recursive functions as natural as writing loops.

If you are in doubt whether a recursive function that you wrote is tail-recursive, you can add the `@tailrec` annotation to the declaration of the function:

```
import scala.annotation.tailrec
...
def sumTail(a: Int, b: Int): Int =
  @tailrec def loop(acc: Int, i: Int): Int =
    if i < b then loop(i + acc, i + 1) else acc
  loop(0, a)
```

If the compiler fails to apply tail call elimination to a `@tailrec` annotated function, then it will issue a warning:

```
-- Error: ----------------------------------------------------
2 |  if a < b then a + sum(a + 1, b) else 0
  |                     ^^^^^^^^^^^^^
  |   Cannot rewrite recursive call: it is not in tail position
```

You may wonder whether non-tail-recursive functions should be avoided at all costs. This depends on the function. Often, tail-recursive functions are harder to understand than a recursive function that performs the same computation, but that is not tail-recursive. [1]. If you know that the recursion depth of the calls to your function will be small in practice, you may want to write the function without tail-recursion. In general, if you are in doubt, you should always value the clarity of your code higher than its efficiency. When you observe that your code is inefficient, you can still optimize it later.

1.4 Classes and Objects

In the previous sections, we have learned about the basic language features of Scala. In this section, we will learn how Scala programs are organized. Scala is an object-oriented language, so Scala programs are organized using *classes* and *objects*.

1.4.1 Classes, Fields, and Methods

Similar to Java, Scala allows you to define classes with *fields* and *methods*, which you can extend using inheritance, override, etc. Fortunately, Scala's syntax for classes is much more lightweight than Java's. For example, consider the following Java class which we can use to wrap pairs of integer values in a single

[1] Similarly, recursive functions are easier to understand than computations that use imperative loops.

object:

```
public class Pair {
  private int first;
  private int second;

  public Pair(int fst, int snd) {
    first = fst;
    second = snd;
  }

  public int getFirst() {
    return first;
  }

  public int getSecond() {
    return second;
  }
}
```

The class consists of:

- two fields called `first` and `second` of type `int` to store the two values;
- a constructor, which takes values to initialize the two fields;
- two "getter" methods to retrieve the two values (we follow good practice and declare all non-final fields as private so that their values cannot be modified without explicit method calls.).

Let us ignore for the moment that we can represent pairs directly in Scala using a tuple type. Here is how we can define the corresponding class in Scala:

```
class Pair(val first: Int, val second: Int)
```

There are some important differences between the Java and Scala version of the class `Pair`:

- In Scala, the class name is followed by a list of class parameters. These parameters serve two purposes:
  1. Parameters that are prefixed by a **`val`** or **`var`** keyword automatically create a field with the given name and type.
  2. The parameter list implicitly defines a constructor with a corresponding list of arguments. The values that are provided for arguments prefixed with **`val`** or **`var`** will be used to initialize the associated fields.
- The default visibility of classes, fields, and methods in Scala is `public`. Hence, we can access the values of the fields `first` and `second` directly and we do not need to define extra getter methods. Note that this makes

sense because Scala discourages mutable state. In particular, we defined the two fields as **val**s, so their values cannot be changed, once an instance of class `Pair` has been created. In Scala, you can leave out the braces around an empty class body, so **class** `C` is the same as **class** `C {}`.

We can create instances of class `Pair` and access their fields as usual:

```
scala> val p = new Pair(1,2)
val p: Pair = Pair@1458e1cc
scala> p.first
val res0: Int = 1
```

What if we do want to modify the values stored in a `Pair` object? In Java, we would do this by adding appropriate "setter" methods to the class:

```
public class Pair {
  private int first;
  private int second;

  ...

  public void setFirst(int fst) {
    first = fst;
  }
  public void setSecond(int snd) {
    second = snd;
  }
}
```

In Scala, we could follow the same route: change all **val**s into **var**s, make them private, and add getter and setter methods. However, we want to avoid using **var** declarations as much as possible. The idiomatic solution in Scala is to make a copy of the entire object and change the appropriate value:

```
class Pair(val first: Int, val second: Int):
  def setFirst(fst: Int): Pair = new Pair(fst, second)
  def setSecond(snd: Int): Pair = new Pair(first, snd)
```

1.4.2 Overriding Methods

Java allows us to override methods that are declared in super classes. Since method calls are dynamically dispatched at run-time, this feature allows us to modify the behavior of an object of the subclass when it is used in a context where an object of the super class is expected.

All Java classes extend the class `Object`. The class `Object` provides, among others, a method `toString`, which computes a textual representation of the object. In particular, the `toString` method can be used to pretty-print objects. By default, the textual representation of objects consists of the name of the

object's class, followed by a unique object ID. We can modify the way objects of a specific class are printed, by overriding the `toString` method. In Java, this can be done as follows:

```
public class Pair {
  private int first;
  private int second;

  ...

  public String toString() {
    return "Pair(" + first + ", " + second + ")";
  }
}
```

In Scala, all classes extend the class `scala.Any` which also provides a method called `toString`. Scala's class hierarchy is further subdivided into the classes `scala.AnyVal` and `scala.AnyRef`, which are directly derived from `scala.Any`. All instances of `scala.AnyVal` are immutable, whereas instances of `scala.AnyRef` may have mutable state. That is, `scala.AnyRef` corresponds to Java's `Object` class.

If we want to override a method in a Scala class, we have to explicitly say so by using the **`override`** qualifier:

```
class Pair(val first: Int, val second: Int):
  ...
  override def toString = "Pair(" + first + ", " + second + ")"
```

The pretty printer in the REPL will now use the new `toString` method to print `Pair` objects:

```
scala> val p = new Pair(1,2)
val p: Pair = Pair(1, 2)
```

1.4.3 Singleton and Companion Objects

If the construction of an object involves complex initialization code, it is often useful to declare dedicate *factory* methods that perform this initialization. In Java, we would declare such methods as *static* members of the corresponding class:

```
public class Pair {
  ...
  public static Pair make(int fst, int, snd) {
    return new Pair(fst, snd);
  }
}
```

We can now call `Pair.make` to create new `Pair` instances.

Scala does not support static methods as they violate the philosophy of object-oriented programming that "everything is an object". Instead of static methods, it provides *singleton objects*. Singleton objects are declared just like classes, but using the keyword **object** instead of **class**. There exists exactly one instance of each **object**, which is automatically created from the **object** declaration when the program is started. Since no further instances of the object can be created, object declarations do not have parameter lists.

For every class `C` in a Scala program, one can declare a singleton object that is also called `C`. This object is referred to as the *companion object* of `C`. Companion objects have access to all private members of instances of `C`. Consequently, a method or field that is defined in the companion object is equivalent to a static method/field of `C` in Java:

```
class Pair(val first: Int, val second: Int):
  ...
object Pair:
  def make(fst: Int, snd: Int) = new Pair(fst, snd)
```

We can access members of companion objects just like static class members in Java:

```
scala> def p = Pair.make(3,4)
val p: Pair = Pair(3, 4)
```

1.4.4 The `apply` Method

Methods with the name `apply` are treated specially by the Scala compiler. For example, if we rename the factory method `make` in our companion object for the `Pair` class to `apply`

```
object Pair:
  def apply(fst: Int, snd: Int) = new Pair(fst, snd)
```

then we can call this method simply by referring to the `Pair` companion object, followed by the argument list of the call:

```
scala> def p = Pair(3,4)
p: Pair = Pair(3, 4)
```

This is equivalent to the following explicit call to the `apply` method:

```
scala> def p = Pair.apply(3,4)
p: Pair = Pair(3, 4)
```

The compiler automatically expands `Pair(3,4)` to `Pair.apply(3, 4)`. That is, objects with an `apply` method can be used as if they were functions[2]. This feature is particularly useful to enable concise calls to factory methods. In fact, factory methods for the data structures in the Scala standard library are typically implemented using `apply` methods provided by companion objects.

[2] If you are familiar with C++, then you will notice that this feature is similar to overloading the function call operator `()` in C++.

1.5 Algebraic Data Types

Algebraic data types (ADTs) and pattern matching are constructs that are commonly found in functional programming languages. They allow you to implement regular, non-encapsulated data structures (such as lists and trees) in a convenient fashion. We will make heavy use of this feature throughout this course.

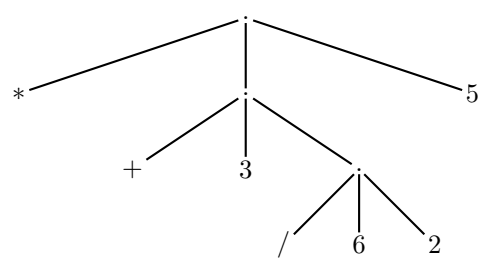
1.5.1 Enumerations

Suppose we want to implement a simple calculator program that takes arithmetic expressions such as

$$(3 + 6/2) * 5$$

as input and evaluates these expressions. This problem is quite similar to writing an interpreter for a programming language, except that the language that we are interpreting here (i.e., arithmetic expressions) is much simpler than a full-blown programming language.

One of the first questions that we have to answer is: how do we represent expressions in our program? Our representation should allow us to easily implement common tasks such as pretty printing, evaluation, and simplification of expressions. In particular, the representation should make the precedence of operators in expressions explicit. E.g., consider the expression $3 + 6/2$, then when we evaluate the expression, our representation should immediately tell us that we first have to divide 6 by 2 before we add 3. To achieve this, expressions are represented as *abstract syntax trees*, or ASTs for short. For example, the abstract syntax tree of the expression $(3 + 6/2) * 5$ can be visualized as follows:



For the AST node representing an expression $e_1 \, op \, e_2$, we follow the convention of representing the operator *op* as the left-most child of the node, rather than putting it between the children representing the operands e_1 and e_2.

Note that the AST tells us exactly how to evaluate the expression. We start at the root. At each node that we visit, we first recurse into the second subtree to evaluate the left operand of the operation. Then we do the same for the third subtree, which represents the right operand of the operation. Finally, we combine the results obtained from the two operands according to the operator labeling the first child. We will learn more about ASTs later. For now it suffices if you have an intuitive understanding what ASTs are.

Algebraic data types allow us to represent tree-like data structures such as ASTs. In Scala, algebraic data types are constructed using *enumerations* or "enums" for short. The following enum defines the ASTs of our arithmetic expressions:

```
enum Expr:
  /* Numbers such as 1, 2, etc. */
  case Num(num: Int)
  /* Expressions composed using binary operators */
  case BinOp(op: Bop, left: Expr, right: Expr)

/* Binary operators */
enum Bop:
  case Add /* + */
  case Sub /* - */
  case Mul /* * */
  case Div /* / */
```

This code declares an enum `Expr` whose instances represent the ASTs of our expression language. The code distinguishes two types of `Expr` objects based on the root node of the represented AST. A `Num(num)` object represents an AST consisting of a single node storing the integer constant `num`. Similarly, an object `BinOp(op, left, right)` represents an AST whose root node combines two subexpressions represented by ASTs `left` and `right` using the *binary operator* `op`. Binary operators are represented by the enum `Bop` which consists of four cases representing the different kinds of arithmetic operations: `Add`, `Sub`, `Mul`, and `Div`.

We refer to the cases of an algebraic data type as its *variants*. For example, `Expr` has variants `Num` and `BinOp`.

The Scala compiler adds some convenient functionality to enums. First, it automatically generates companion objects with appropriate factory methods. These methods are particularly useful when you nest them to construct complex expressions:

```
scala> import Expr._, Bop._
scala> val e = BinOp(Add, Num(3), BinOp(Mul, Num(4), Num(5)))
val e: Expr = BinOp(Add, Num(3), BinOp(Mul, Num(4), Num(5)))
```

Note that the enums `Expr` and `Bop` itself have companion objects. In turn, these companion objects have nested companion objects for the variants of the defined ADT within them. For example, the companion object for `Expr` contains companion objects for `Num` and `BinOp`. The nested companion objects `Num` and `BinOp` then have `apply` methods for constructing objects of the corresponding variant. The **`import`** instruction in the above code snippet makes the nested companion objects directly accessible in the code. For instance, in the code `Num(4)`, `Num` refers to the companion object of variant `Num` in `Expr`. Thus, this code expands to `Expr.Num.apply(4)`.

Second, the compiler adds natural implementations of the methods `toString`, `hashCode`, and `equals` for each variant. These will print, hash, and compare a whole tree consisting of the top-level enum instance and (recursively) all its

arguments. In Scala, an expression of the form `x == y` always translates into a call of the form `x.equals(y)` (just like in Java). The overriden `equals` method therefore ensures that enum instances are always compared structurally. For example, we have:

```
scala> val e1: BinOp = BinOp(Add, Num(3), Num(4))
val e1: Expr.BinOp = BinOp(Add, Num(3), Num(4))
scala> val e2 = BinOp(Add, Num(3), Num(4))
val e2: Expr.BinOp = BinOp(Add, Num(3), Num(4))
scala> e1 == e2
val res2: Boolean = true
```

In the example above, `e1` and `e2` point to two different objects in memory. However, the two ASTs represented by these objects have exactly the same structure. Hence, `e1 == e2` evaluates to **`true`**.

Next, the compiler implicitly adds a **`val`** prefix to all arguments in the parameter list of an enum object, so that they are maintained as fields:

```
scala> val n = e1.left
val n: Expr = Num(3)
```

Note that the above code only works because we explicitly declared `e1` to be of type `BinOp` earlier. Without this type annotation, the inferred type of `e1` would be `Expr`. However, `Expr` objects are not guaranteed to have a field called `left` since `Expr` also includes `Num` objects.

Finally, the compiler adds a `copy` method to your enum cases for making modified copies. This method is useful if you need to create a copy of an existing enum object `o` that is identical to `o` except for some of `o`'s attributes:

```
scala> e1.copy(op = Sub)
val res4: Expr = BinOp(Sub, Num(3), Num(4))
```

1.5.2 Pattern Matching

Suppose we want to implement an algorithm that simplifies expressions by recursively applying the following simplifications rules:

- $e + 0 \Rightarrow e$
- $e * 1 \Rightarrow e$
- $e * 0 \Rightarrow 0$

To identify whether a given expression matches one of the left-hand sides of the rules, we have to look at some of its subexpressions. E.g., to check whether an expression of the form $e_1 + e_2$ matches the left-hand side of the first rule, we have to look at the left subexpression e_1 to check whether $e_1 = 0$. Implementing this kind of pattern matching is quite tedious in many languages (including Java). Fortunately, the Scala language has inbuilt support for pattern matching that works hand-in-hand with enumerations.

Let us first reformulate the three simplification rules in terms of our enum representation of expressions:

```
BinOp(Add, e1, Num(0)) => e1
BinOp(Mul, e1, Num(1)) => e1
BinOp(Mul, e1, Num(0)) => Num(0)
```

Using pattern matching, these rules almost directly give us the implementation of the following function `simplifyTop`, which applies the rules at the top-level of the given expression `e`:

```
def simplifyTop(e: Expr) =
  e match
    case BinOp(Add, e1, Num(0)) => e1
    case BinOp(Mul, e1, Num(1)) => e1
    case BinOp(Mul, _, Num(0)) => Num(0)
    case _ => e
```

The body of `simplifyTop` is a *match expression*. A match expression consists of a *selector*, in this case `e`, followed by the keyword **`match`**, followed by a sequence of match alternatives.

Each match alternative starts with the keyword **`case`**, followed by a pattern, followed by an expression that is evaluated if the pattern matches the selector. The pattern and expression are separated by an arrow symbol **`=>`**.

A match expression is evaluated by checking whether the selector matches one of the patterns in the alternatives. The patterns are tried in the order in which they appear in the program. The first pattern that matches is selected and the expression following the arrow is evaluated. The result of the entire match expression is the result of the expression in the selected alternative.

Here is an example of a recursive function that uses pattern matching to pretty print arithmetic expressions:

```
def pretty(e: Expr): String =
  e match
    case BinOp(bop, e1, e2) =>
      val bop_str = bop match
        case Add => "␣+␣"
        case Sub => "␣-␣"
        case Mul => "␣*␣"
        case Div => "␣/␣"
      "(" + pretty(e1) + bop_str + pretty(e2) + ")"
    case Num(n) => n.toString()
```

```
scala> val e = BinOp(Add, BinOp(Mult, Num(3), Num(4)), Num(1))
val e: BinOp = BinOp(Add, BinOp(Mult, Num(3), Num(4)), Num(1))
scala> pretty(e)
val res0: String = ((3 * 4) + 1)
```

There are different types of patterns. The most important types are:

- *Constant patterns*: A constant pattern such as `0` matches values that are equal to the constant (with respect to `==`).
- *Variable patterns*: A variable pattern such as `e1` matches every value. Here, `e1` is a variable that is bound in the pattern. The variable refers to the matched value in the right-hand side of the match alternative.
- *Wildcard patterns*: A wildcard pattern `_` also matches every value, but it does not introduce a variable that refers to the matched value.
- *Constructor patterns*: A constructor pattern such as `BinOp(Add, e, Num(0))` matches all values of type `BinOp` whose first argument matches `Add`, whose second argument matches `e`, and whose third argument matches `Num(0)`. Note that the arguments to the constructor `BinOp` are themselves patterns. This allows you to write deep patterns that match complex enum values using a concise notation.

1.5.3 Binding Names in Patterns

Sometimes we want to match a subexpression against a specific pattern and also bind the matched expression to a name. This is useful when we want to reuse a matched subexpression in the right-hand side of the match alternative. For example, in the third simplification rule of `simplifyTop` we are returning `Num(0)`, which is also the second subexpression of the matched expression `e`. Instead of creating a new expression, `Num(0)` on the right-hand side of the rule, we can also directly return the second subexpression of `e`. We can do this by binding a name to that subexpression in the pattern using the operator `@` as follows:

```
def simplifyTop(e: Expr) =
  e match
    case BinOp(Add, e1, Num(0)) => e1
    case BinOp(Mul, e1, Num(1)) => e1
    case BinOp(Mul, _, e2 @ Num(0)) => e2
    case _ => e
```

Note that the pattern in the third match alternative now binds the name `e2` to the value matched by the pattern `Num(0)`. This value is then returned on the righ-hand side of the rule by referring to it using the name `e2`.

1.5.4 Pattern Guards

Suppose we want to extend our expression simplifier so that it additionally implements the following simplification rule: $e + e \Rightarrow 2 \times e$

If we directly translate the rule to a corresponding match alternative, we obtain the following implementation of `simplilfyTop`:

```
def simplifyTop(e: Expr) =
  e match
```

```
    ...
    case BinOp(Add, e1, e1) => BinOp(Mul, Num(2), e1)
    case _ => e
```

Unfortunately, the compiler will reject this function because we use the name `e1` twice within the same pattern. In general, a variable name such as `e1` may only be used once in a pattern. We can solve this problem by using a different variable name for the second subexpression, say `e2`, and then use a *pattern guard* to enforce that the subexpressions matched by `e1` and `e2` are equal:

```
def simplifyTop(e: Expr) =
  e match
    ...
    case BinOp(Add, e1, e2) if e1 == e2 =>
      BinOp(Mul, Num(2), e2)
    case _ => e
```

In general, a pattern guard can be an arbitrary Boolean expression over the names that are in the scope of the match alternative. The pattern guard is appended to the pattern of a match alternative using the keyword **`if`**.

1.5.5 Exhaustiveness Checks

Consider the following function `simplifyAll` that applies our simplification rules recursively to the given expression:

```
def simplifyAll(e: Expr): Expr =
  e match
    case BinOp(Add, e1, Num(0)) => simplifyAll(e1)
    case BinOp(Mul, e1, Num(1)) => simplifyAll(e1)
    case BinOp(Mul, _, e2 @ Num(0)) => e2
    case BinOp(Add, e1, e2) if e1 == e2 =>
      BinOp(Mul, Num(2), simplifyAll(e2))
    case BinOp(bop, e1, e2) =>
      BinOp(bop, simplifyAll(e), simplifyAll(e2))
```

Observe that in this function, the pattern alternatives are no longer exhaustive. That is, there exist values `e` that are not matched by any of the match alternatives, e.g., the value `Num(0)`. If `simplifyAll` is called with `Num(0)`, it will throw a runtime exception.

We can fix this code by adding an explicit match alternative for the `Num` constructor:

```
def simplifyAll(e: Expr): Expr =
  e match
    case BinOp(Add, e1, Num(0)) => simplifyAll(e1)
    case BinOp(Mul, e1, Num(1)) => simplifyAll(e1)
    case BinOp(Mul, _, e2 @ Num(0)) => e2
    case BinOp(Add, e1, e2) if e1 == e2 =>
```

```
      BinOp(Mul, Num(2), simplifyAll(e2))
    case BinOp(bop, e1, e2) =>
      BinOp(bop, simplifyAll(e), simplifyAll(e2))
    case Num(_) => e
```

With complex patterns like this it can be tricky to keep track of all the possible cases. Fortunately, the compiler will automatically check whether the case analysis is exhaustive. For instance, consider the following faulty implementation of `simplifyAll` where we have omitted the last case for `BinOp` from the previous implementation:

```
def simplifyAll(e: Expr): Expr =
  e match
    case BinOp(Add, e1, Num(0)) => simplifyAll(e1)
    case BinOp(Mul, e1, Num(1)) => simplifyAll(e1)
    case BinOp(Mul, _, e2 @ Num(0)) => e2
    case BinOp(Add, e1, e2) if e1 == e2 =>
      BinOp(Mul, Num(2), simplifyAll(e2))
    case Num(_) => e
```

This version does not compile:

```
-- [E029] Pattern Match Exhaustivity Warning: ----------
2 |  e match
  |  ^
  |  match may not be exhaustive.
  |
  |  It would fail on pattern case: Expr.BinOp(_, _, _)
```

Unfortunately, these exhaustiveness checks can sometimes produce spurious warnings. For example, suppose we have a function that is meant to pretty-print number expressions, but not other expressions which have not yet been reduced:

```
def prettyNumber(e: Expr): String =
  e match
    case Num(num) => num.toString()
```

Further suppose that we know that our program ensures that `prettyNumber` is never called on a `BinOp` expression. Yet, the compiler still complains about the non-exhaustive pattern matching. We can suppress this warning by declaring `e` as *unchecked*:

```
def prettyNumber(e: Expr): String =
  (e: @unchecked) match
    case Num(num) => num.toString()
```

While `@unchecked` notations are sometimes necessary to suppress spurious warnings, you should be very careful about introducing them in your code. In most cases, the compiler generated warnings indicate actual problems in your code that need your attention.

1.5.6 Option Types

Suppose we want to write a function that evaluates arithmetic expressions to `Int` values. One question is: How should we deal with undefined operations such as division by zero:

```
BinOp(Div, e, Num(0)) => ???
```

In Java, we would typically go for one of the following two solutions:

- throw an exception such as `ArithmeticException`;
- return **`null`** to indicate that the intended operation does not yield a valid result.

Both approaches have advantages and drawbacks.

Exceptions are a good solution if the undefined operation is indeed exceptional behavior that should, e.g., abort the program. In this case, we ensure that a computation that returns normally always yields a valid result. However, if the undefined operation commonly occurs in computations, we will have to catch the exception and handle it appropriately. This has two disadvantages. First, the exception mechanism is relatively expensive and should only be used in truly exceptional situations. Second, the exception handlers will clutter the code and the non-structured control flow of thrown exceptions makes it more difficult to understand what the program is doing.

If we return **`null`**, we avoid the two disadvantages of exceptions: the computation always returns normally, and there is no computational overhead such as recording the stack-trace to the point where the exception was thrown. However, **`null`** values introduce their own problems. Since **`null`** can have an arbitrary type, the type checker of the compiler will give us much weaker static correctness guarantees for our code. In particular, it will be unable to statically detect unintended accesses to the return value in cases where the return value is invalid (hello `NullPointerException`!).

In languages that support pattern matching, there is a common idiom that avoids the problem of introducing **`null`** values: option types.

The option type is an algebraic data type with two variants: `Some(v)` to indicate that a computation returned a proper result value `v`, and `None` to indicate that the intended operation was undefined and has no proper result.

In Scala, we can define an option type for `Int` values using an enum as follows:

```
enum IntOption:
  case Some(value: Int)
  case None
```

We can now use the option type similarly to null values in Java:

```
def div(x: Int, y: Int): IntOption =
  if y == 0 then None else Some(x / y)
```

Unlike in Java, where the static type checker is unable to distinguish a **null** value from a genuine result of a computation, the Scala type checker will force us to explicitly unwrap the `Int` value embedded in an `IntOption` before we can access it. Using pattern matching, we can do this conveniently. For example, suppose we want to convert the result of `div` to a double precision floating point number. By using pattern matching on the return value of `div`, we can recover from some of the cases where integer division by 0 is undefined:

```
def divToDouble(x: Int, y: Int): Double =
  div(x,y) match
    case Some(x) => x
    case None =>
      if x < 0 then Double.NegativeInfinity
      else if x > 0 then Double.PositiveInfinity
      else Double.NaN
```

Since option types are so useful, Scala already provides a generic option type, called `Option`, in its standard library. Using the predefined type `Option` we can write the function `div` like this:

```
def div(x: Int, y: Int): Option[Int] =
  if y == 0 then None else Some(x / y)
```

Chapter 2

Foundations

Recursion and induction will be our main tools for formalizing programming languages. In this chapter, we study the mathematical foundations of these two closely related concepts. We will then apply these concepts to understand a ubiquitous recursive data structure: lists. We will later see that this formal approach allows us to prove mathematical theorems about individual programs and, more generally, about a programming language as a whole.

2.1 Notation

Throughout the course notes we will assume a basic understanding of mathematical notation and some concepts and notations from set theory. We briefly recap these in this section.

Properties. We use standard logical notation to express *properties*, i.e. formal mathematical statements that are true or false. For instance, $1 < 2$ is a true property and $2 < 1$ is a false property. For given properties P and Q, we introduce

- the *negation* of P, written $\neg P$, which is true iff[1] P is false,
- the *conjunction* of P and Q, written $P \wedge Q$, which is true iff both P and Q are true,
- the *disjunction* of P and Q, written $P \vee Q$, which is true iff P or Q is true,
- the *implication* of P and Q, written $P \Rightarrow Q$, which is true iff P implies Q (i.e., if P is true then Q is true),
- and the *equivalence* of P and Q, written $P \Leftrightarrow Q$, which is true iff P and Q are logically equivalent (i.e. $P \Rightarrow Q$ and $Q \Rightarrow P$).

[1] "iff" abbreviates "if and only if"

Properties may refer to variables (or unknowns). For instance, the property $0 \leq x \wedge x < 10$ is true for all natural numbers x between 0 and 9. We write $P(x_1, \ldots, x_n)$ to indicate that P refers to the variables x_1 to x_n. For a property $P(x)$, *universal quantification* over x yields the property $\forall x : P(x)$, which is true iff $P(x)$ is true for all possible values of x. Similarly, *existential quantification* over x yields the property $\exists x : P(x)$, which is true iff there exists at least one value for x such that $P(x)$ is true.

Sets. A *set* is an unordered collection of objects, which we refer to as the *elements* or *members* of the set. We write $e \in S$ to indicate that e is an element of the set S and we write $e \notin S$ for $\neg(e \in S)$. We write $\forall x \in S : P(x)$ as a short-hand for the property $\forall x : x \in S \Rightarrow P(x)$ and likewise write $\exists x \in S : P(x)$ for $\exists x : x \in S \wedge P(x)$.

We sometimes denote a set by explicitly enumerating its elements using the notation $\{a, b, c, d, e\}$. Here, $\{a, b, c, d, e\}$ is the set consisting of the elements a, b, c, d, and e. We denote the empty set by $\emptyset$ and we write $\mathbb{N}$ for the set of all natural numbers $\mathbb{N} = \{0, 1, 2, \ldots\}$ and $\mathbb{Z}$ for the set of all integers $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$.

At times, we will define a set in terms of a *set comprehension*, i.e., the set of all elements x that satisfy some given property $P(x)$, written $\{\, x \mid P(x) \,\}$. For example, the set comprehension $\{\, x \mid x \in \mathbb{Z} \wedge x < 0 \,\}$ defines the set of all negative integers.[2]

We define the usual operations on sets such as *union* $S \cup T$, *intersection* $S \cap T$, and *set difference* $S \setminus T$:

$$S \cup T \stackrel{\text{def}}{=} \{\, x \mid x \in S \vee x \in T \,\}$$
$$S \cap T \stackrel{\text{def}}{=} \{\, x \mid x \in S \wedge x \in T \,\}$$
$$S \setminus T \stackrel{\text{def}}{=} \{\, x \mid x \in S \wedge x \notin T \,\}$$

A set S is a *subset* of another set T, written $S \subseteq T$, if every element of S is also an element of T. For example, we have $\mathbb{N} \subseteq \mathbb{Z}$ and for all sets S we have $\emptyset \subseteq S$ and $S \subseteq S$. Two sets S and T are equal if they have the same elements, i.e. if $S \subseteq T$ and $T \subseteq S$.

Given a set S, we write 2^S for the set of all subsets of S:

$$2^S \stackrel{\text{def}}{=} \{\, x \mid x \subseteq S \,\}$$

We call 2^S the *powerset* of S. For instance, the powerset of $\{0, 1\}$ is $2^{\{0,1\}} = \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$.[3]

Given two elements a and b, we construct a new element $\langle a, b \rangle$ called the *(ordered) pair* consisting of a and b. Formally, we can define $\langle a, b \rangle$ as the set

[2] Not every property $P(x)$ defines a set. A notorious example is the property $x \notin x$, which is at the heart of Russel's Paradox.

[3] The notation 2^S is reminiscent of the fact that if S is a finite set consisting of n elements, then the powerset of S has 2^n elements.

$\{\{a\},\{a,b\}\}$.[4] We denote by $S \times T$ the *Cartesian product* of the sets S and T, which is the set of all pairs consisting of elements from S and T:

$$S \times T \stackrel{\text{def}}{=} \{\, \langle x, y\rangle \mid x \in S \wedge y \in T \,\}$$

We generalize these definitions to *n-tuples* $\langle a_1, \ldots, a_n\rangle$ consisting of n elements $a_1,\ldots,a_n$ and Cartesian products over n sets $S_1 \times \cdots \times S_n$ for arbitrary $n \in \mathbb{N}$. In particular, for an element a, the 1-tuple $\langle a\rangle$ is equal to the set $\{a\}$ and the *empty tuple* $\langle\rangle$ is equal to the empty set.

Relations and Functions. Given two sets S and T, we call an element $R \in 2^{S\times T}$ a *binary relation* over S and T. That is, we have $R \subseteq S \times T$ by definition. If $\langle x, y\rangle \in R$ for some $x \in S$ and $y \in T$, we say that R *relates* x to y. We denote by $R^{-1} \subseteq T \times S$ the *inverse* of R:

$$R^{-1} \stackrel{\text{def}}{=} \{\, \langle y, x\rangle \mid \langle x, y\rangle \in R \,\}$$

R is called *total* if R relates every element of S to at least one element of T. R is called *functional* if R relates every element of S to at most one element of T. If R is functional, it is also called a *partial function* from S to T. If R is functional and total, it is called a *(total) function* from S to T. We write $R : S \rightharpoonup T$ to indicate that R is a partial function from S to T and $R : S \to T$ to indicate that it is a function from S to T. Every (total) function is also a partial function but the converse does not hold.

Let $f : S \rightharpoonup T$. We say that f is *defined* for $x \in S$ if there exists y such that $\langle x, y\rangle \in f$. In this case, we say that f *maps* x to y. We denote this unique y by $f(x)$. The *domain* of f, written $\mathsf{dom}(f)$, is the set of all $x \in S$ for which f is defined. Note that f is total iff $\mathsf{dom}(f) = S$.

We typically define a (partial) function from S to T using one or more equations that define $f(x)$ for all $x \in \mathsf{dom}(f)$. For example, the following definition defines the *successor* function on natural numbers:

$$\begin{aligned} succ &: \mathbb{N} \to \mathbb{N} \\ succ(x) &= x + 1 \end{aligned}$$

That is, $succ = \{\langle 0, 1\rangle, \langle 1, 2\rangle, \langle 2, 3\rangle, \ldots\}$.

For a partial function $f : S \rightharpoonup T$ and any elements a and b, we denote by $f[a \mapsto b]$ the partial function from $S \cup \{a\}$ to $T \cup \{b\}$ defined as follows:

$$f[a \mapsto b](x) \stackrel{\text{def}}{=} \begin{cases} f(x) & x \in \mathsf{dom}(f) \setminus \{a\} \\ b & x = a \\ \text{undefined} & \text{otherwise} \end{cases}$$

That is, $f[a \mapsto b]$ maps a to b and otherwise behaves like f. Note that we allow $a \in S$ and $b \in T$ but we do not require this.

[4] Convince yourself that with this definition we have $\langle a, b\rangle = \langle a', b'\rangle$ if and only if $a = a'$ and $b = b'$.

For a binary relation $R \subseteq S \times S$, we define its *reflexive closure* as the relation $R \cup \{ \langle x, x \rangle \mid x \in S \}$. We further define its *transitive closure*, denoted R^+, as the smallest relation that satisfies the following two conditions:

1. $R \subseteq R^+$, and
2. if $\langle x, y \rangle \in R^+$ and $\langle y, z \rangle \in R^+$ then $\langle x, z \rangle \in R^+$.

We denote by R^* the reflexive and transitive closure of R.

2.2 Structural Recursion and Induction

Recursion is a constructive technique for describing infinite sets (and infinite functions and relations on these sets). Induction is a technique for proving properties about recursively defined sets, functions and relations. There exist different variants of recursion and induction. We are interested in the simplest form of these concepts which we refer to as *structural recursion* and *structural induction*, respectively.

2.2.1 Structurally Recursive Definitions

We explain all these concept using a very simple example. To this end, we define a set N that behaves just like (or mathematically speaking, is isomorphic to) the natural numbers $\mathbb{N}$. We will represent the natural numbers as certain tuples. Once we have defined N, we will build structurally recursive functions on N that correspond to addition and multiplication, and then prove properties of these functions.

We represent the natural numbers as follows, using only nesting of tuples: we start by 0, which we represent as the empty tuple $\langle\rangle$. Then, given a natural number n, we construct its successor $n + 1$ by wrapping n in another tuple:

$$\begin{array}{ll} 0 & \langle\rangle \\ 1 & \langle\langle\rangle\rangle \\ 2 & \langle\langle\langle\rangle\rangle\rangle \\ & \dots \end{array}$$

That is, the number 1 is represented by the tuple that contains the empty tuple, the number 2 is the tuple that contains the tuple containing the empty tuple, etc. For the set N, we choose exactly those tuples that represent natural numbers following the above convention[5].

We now show how we can describe the set N using structural recursion, without referring to the natural numbers $\mathbb{N}$. We do this by providing recursive construction rules for the elements of N:

[5] This construction of the natural numbers is similar to the standard set-theoretic construction of natural numbers, except that we use a slightly simpler construction rule for successors here.

1. The empty tuple $\langle\rangle$ is an element of N.
2. If x is an element of N, then the tuple $\langle x\rangle$ is an element of N.
3. N only contains elements that can be constructed using rules 1 and 2.

We can present these construction rules more compactly using the following *inference rules*:

$$\textsc{Rule 1}\quad \langle\rangle \in N \qquad\qquad \textsc{Rule 2}\quad \frac{x \in N}{\langle x\rangle \in N}$$

Rule 3 is left implicit.

In general, an inference rule takes the form

$$\frac{P_1 \qquad \ldots \qquad P_n}{C}$$

Such a rule states that if the properties $P_1, \ldots, P_n$ hold, then also C holds, $P_1 \wedge \ldots \wedge P_n \Rightarrow C$. The properties P_i are called the *premises* of the rule, and the property C the *conclusion* of the rule. If a rule has no premise, then the conclusion always holds. Such rules are also called *axioms* and we omit the line separating the premises from the conclusion in this case. For instance, Rule 1 above is an axiom.

Using Rule 1, we can construct the representation of the number 0. Using Rule 2, we can construct the representation of the number $n + 1$, given the representation of the number n. For the representation of the number 3 we need four construction steps:

1.	$\langle\rangle$	with rule 1
2.	$\langle\langle\rangle\rangle$	with rule 2
3.	$\langle\langle\langle\rangle\rangle\rangle$	with rule 2
4.	$\langle\langle\langle\langle\rangle\rangle\rangle\rangle$	with rule 2

The recursive definition of N is to be understood such that N contains exactly those objects that can be constructed by the inference rules in a *finite number* of steps. Whenever we give a recursive definition of a set in terms of inference rules, then this additional requirement (which corresponds to Rule 3) is always implicit.

Alternatively, we can describe the construction rules for the elements of N using a fixpoint equation:

$$N = \{\langle\rangle\} \cup \{\, \langle x\rangle \mid x \in N \,\}$$

That is, N is defined as the smallest set (with respect to subset inclusion) that satisfies this recursive equation. We also say that N is the *least fixpoint* of the equation. The left hand side of the equation captures the two construction

rules for the elements of N. The fact that N is defined as the smallest set that satisfies the equation (i.e., the least fixpoint rather than any other fixpoint) captures Rule 3.

All of the above definitions of the set N are equivalent, i.e., they define the exact same set of mathematical objects. We will mostly work with definitions of recursive sets that are given in the form of inference rules as these are usually more intuitive than those given as solutions of fixpoint equations.

The recursive definition of N has two important properties that make it a *structurally recursive* definition:

1. Every object in N can be constructed only with exactly one rule. For example, $\langle\rangle$ can only be constructed with the first rule and $\langle\langle\rangle\rangle$ only with the second rule.

2. The recursive rule constructs from an object $x \in N$ a larger object $\langle x\rangle \in N$ that contains x as a proper subobject.

In general, we will work with structurally recursive definitions that have more than one base case rule and more than one recursive rule.

Algebraic data type definitions in Scala can be viewed as structurally recursive definitions of sets of tuples. For example the following declarations give a possible Scala encoding of our definition of the set N:

```
enum N:
  case Zero // rule 1
  case Succ(x: N) // rule 2
```

We have one **case** declaration per construction rule of N. The implicit Rule 3 is enforced by Scala: **enum** types cannot be extended using subtyping.

2.2.2 Recursive Definitions of Functions

The real power of structural recursion is that we can also use it to define functions that operate on the elements of a recursive set. For example, suppose we want to define a function $D : N \to \mathbb{N}$ that maps the elements of N to the natural numbers they represent. We can do this as follows:

$$\begin{aligned} D &: N \to \mathbb{N} \\ D(\langle\rangle) &= 0 \\ D(\langle x\rangle) &= 1 + D(x) \end{aligned}$$

This definition has two important properties that make it a *structurally recursive function definition*:

1. For each defining rule of N, there is a corresponding defining rule for D. This implies that for every element of N exactly one rule for D applies, which ensures that D is a partial function.

2. Recursive applications of D only apply to proper subobjects of its argument. (In the second rule for D, the recursive application of D for the argument $\langle x \rangle$ is on the proper subobject x.) This ensures that D is total.

Hence, together, these properties guarantee that the rules for D define a function, i.e., D is well-defined. We can also view the above definition as a blue print for the declaration of a Scala function on our algebraic data type representation of `N`:

```
def D(y: N): Int =
  y match
    case Zero => 0
    case Succ(x) => 1 + D(x)
```

From the fact that the definition of the function D is structurally recursive, it follows that `D` terminates normally for all input values `y`.

Next, we use structural recursion to define a two-valued function $\oplus : N \times N \to N$ that corresponds to addition on natural numbers:

$$\begin{aligned} \oplus &: N \times N \to N \\ \langle\rangle \oplus y &= y \\ \langle x \rangle \oplus y &= \langle x \oplus y \rangle \end{aligned}$$

We use the symbol $\oplus$ for this function instead of $+$ to distinguish it from the actual addition function $+ : \mathbb{N} \times \mathbb{N}$ on natural numbers. We will see below how these two functions are formally related.

Note that the second rule in the definition of D determines how the value of D for larger arguments is constructed from values of D for smaller arguments. In the case of $\oplus$, the recursion only goes over the first argument of the function.

2.2.3 Structural Induction

Often, we want to prove that a particular property holds for all elements of a recursively defined set. For example, we may want to prove that the function $\oplus$ corresponds to the actual addition function $+$ on natural numbers. In other words, we may want to prove that the property

$$D(x \oplus y) = D(x) + D(y)$$

holds for all elements $x, y \in N$.

In general, if we want to prove that all $x \in N$ satisfy a given property A, we can proceed as follows:

1. Prove that $\langle\rangle$ satisfies A.

2. Prove that for all $x \in N$, if x satisfies A, then $\langle x \rangle$ satisfies A.

We call the proof rule that we just formulated the *induction principle* for N. We can write this rule more compactly as an inference rule:

$$\frac{A(\langle\rangle) \qquad \forall x \in N : A(x) \Rightarrow A(\langle x\rangle)}{\forall x \in N : \ A(x)}$$

Notice that the premises of this inference rule

(1) $A(\langle\rangle)$

(2) $\forall x \in N : A(x) \Rightarrow A(\langle x\rangle)$

are derived directly from the defining rules of N. The first premise is called the *base case* of the induction rule and the second premise is called the *induction step*. The left-hand side of the implication in the induction step is called the *induction hypothesis*. When we prove $A(\langle x\rangle)$ in the induction step for a particular A, we can assume that the induction hypothesis $A(x)$ holds.

For every set that is defined by structural recursion there is a corresponding induction principle that is derived from the definition of the set. Every rule that is an axiom yields a base case and every other rule yields an induction step.

In the next section, we will prove the correctness of the induction principle for N by proving the correctness of a more general induction principle. However, you should already try to develop some intuition for why the induction principle for N is correct. Perhaps the following explanation helps you if you have trouble understanding the rule. Let A be some property for which the premises (1) and (2) of the induction principle hold. We validate that from these premises follows the validity of

$$A(\langle\langle\langle\langle\rangle\rangle\rangle\rangle)$$

That is, we want to validate that A holds for our representation of the number 3. First, from premise (1) follows:

$$A(\langle\rangle)$$

From this fact and premise (2) we conclude:

$$A(\langle\langle\rangle\rangle)$$

Applying premise (2) twice more we obtain the property we wanted to show:

$$A(\langle\langle\langle\langle\rangle\rangle\rangle\rangle) \ .$$

The trick lies in premise (2), which states that for any $x \in N$, if $A(x)$ holds, then also $A(\langle x\rangle)$ holds.

Let us use the induction principle to prove the equivalence of the functions $\oplus$ on N and $+$ on $\mathbb{N}$. To this end, we define the property

$$A(x) \stackrel{\text{def}}{=} \forall y \in N : \ D(x \oplus y) = D(x) + D(y)$$

and then prove that for all $x \in N$, $A(x)$ holds. For the proof to succeed, it is important that we let the induction go over x rather than y because we will need to use the recursive definitions of the functions D and $\oplus$ in the proof. Recall that the recursion for $\oplus$ goes over its first argument, which is x in this case.

The induction principle tells us that it is sufficient to prove the following two properties (these are the premises of the induction principle instantiated with the concrete A that we defined above):

(1) $\forall y \in N : D(\langle\rangle \oplus y) = D(\langle\rangle) + D(y)$

(2) $\forall x \in N : (\forall y \in N : D(x \oplus y) = D(x) + D(y))$
$\Rightarrow (\forall y \in N : D(\langle x\rangle + y) = D(\langle x\rangle) + D(y))$

It is not difficult to prove these properties.

To minimize the amount of writing we have to do (and to improve clarity), we follow a specific pattern when we write induction proofs. We demonstrate this pattern in the proof below.

Lemma 2.1. $\forall x \in N : \forall y \in N : D(x \oplus y) = D(x) + D(y)$

Proof. By structural induction over $x \in N$:

Let $x = \langle\rangle$ and $y \in N$. Then

$$\begin{aligned}
D(x \oplus y) &= D(\langle\rangle + y) \\
&= D(y) && \text{Definition of } \oplus \\
&= 0 + D(y) && 0 \text{ neutral element of } + \\
&= D(\langle\rangle) + D(y) && \text{Definition of } D \\
&= D(x) \oplus D(y)
\end{aligned}$$

Let $x = \langle x'\rangle$ and $y \in N$. Then

$$\begin{aligned}
D(x \oplus y) &= D(\langle x'\rangle \oplus y) \\
&= D(\langle x' \oplus y\rangle) && \text{Definition of } \oplus \\
&= 1 + D(x' \oplus y) && \text{Definition of } D \\
&= 1 + (D(x') + D(y)) && \text{Induction hypothesis} \\
&= (1 + (D(x')) + D(y) && \text{Associativity of } + \\
&= D(\langle x'\rangle) + D(y) && \text{Definition of } D \\
&= D(x) + D(y)
\end{aligned}$$

□

2.2.4 Well-founded Induction (optional)

Recursion and induction appear in many variants. The common idea behind all these variants can be formulated using the notion of well-founded relations.

Let X be a set and $\succ \subseteq X \times X$ a binary relation over X. A (possibly infinite) sequence $x_1, x_2, x_3, \ldots$ of elements in X is called a *descending chain* of $\succ$ if $x_{i-1} \succ x_i$ for all members x_i of the sequence, $i > 0$. The relation $\succ$ is called *well-founded* if it has no infinite descending chains. We call a pair $(X, \succ)$ consisting of a set X and a well-founded relation $\succ$ on X a *well-founded set*.

Let $(X, \succ)$ be a well-founded set. With the notation $x \succ y$ we intuitively mean that x is in some sense larger than y. Well-foundedness then means that starting from any element $x_1 \in X$, we cannot find smaller and smaller elements $x_1 \succ x_2 \succ x_3 \succ \cdots$. At some point a long the chain, we must arrive at an element x_n such that no other element in X is smaller than x_n.

As an example, let us reconsider the set N. The relation

$$\begin{aligned} &\succ_N \colon N \times N \\ &x \succ_N y \overset{\text{def}}{\iff} x = \langle y \rangle \end{aligned}$$

is a well-founded relation on N.[6] The reflexive and transitive closure of $\succ_N$ corresponds to the canonical ordering on natural numbers $\geq \subseteq \mathbb{N} \times \mathbb{N}$.

Let $(X, \succ)$ be a well-founded set and $M \subseteq X$. An element $x \in M$ is called *minimal element* of M if there exists no $y \in M$ such that $x \succ y$.

Lemma 2.2. *Let $(X, \succ)$ be a well-founded set. Then every nonempty subset of X has at least one minimal element.*

Proof. By contradiction. Let M be a nonempty subset of X that has no minimal element. Then there exists for every $x \in M$ some $y \in M$ such that $x \succ y$. Since M contains at least one element, we can construct an infinite descending chain of elements in M, and thus in X. It follows that $\succ$ is not well-founded. Contradiction. □

Let $(X, \succ)$ be a well-founded set and $A(x)$ a property for $x \in X$. We call the following inference rule the *well-founded induction principle*[7] for X, $\succ$, and A:

$$\frac{\forall x \in X : (\forall y \in X : x \succ y \Rightarrow A(y)) \Rightarrow A(x)}{\forall x \in X : A(x)}$$

The following theorem states the correctness of this rule.

Theorem 2.3. *Let $(X, \succ)$ be a well-founded set and $A(x)$ a property of $x \in X$. If*

$$\forall x \in X : (\forall y \in X : x \succ y \Rightarrow A(y)) \Rightarrow A(x)$$

then for all $x \in X$, $A(x)$ holds.

[6] This follows from our definition of tuples in terms of sets and the *axiom of foundation* of set theory.

[7] Sometimes, well-founded induction is also called Noetherian induction, named after the mathematician Emmy Noether.

Proof. By contradiction. Let M be the subset of X that contains all elements for which A does not hold. We assume that M is nonempty. Then we can choose a minimal element x_0 in M according to Lemma 2.2. Since M contains all elements of X for which A does not hold, it follows that

$$\forall y \in X : x_0 \succ y \Rightarrow A(y)$$

Then $A(x_0)$ follows from the premise of the induction rule. Contradiction. □

The rule for well-founded induction only has a single premise and does not distinguish between base case and induction step. This is possible due to the more general formulation of the induction hypothesis in the premise of this rule. When we instantiate the rule for a particular well founded set $(X, \succ)$, then the base case and induction step typically emerge from further case analysis. For example, the induction principle for N is obtained from the well-founded induction principle by instantiating the latter with N for X and $\succ_N$ for $\succ$. The premise of the instantiated rule is:

$$\forall x \in N : (\forall y \in N : x \succ_N y \Rightarrow A(y)) \Rightarrow A(x)$$

To see that this premise is equivalent to the two premises of the induction principle for N, let us first replace $\succ_N$ by its definition:

$$\forall x \in N : (\forall y \in N : x = \langle y \rangle \Rightarrow A(y)) \Rightarrow A(x)$$

Now, in order to prove this premise for a particular A, we have to distinguish the two possible cases how each $x \in N$ was constructed, corresponding to the structurally recursive definition of N. This gives us two cases that we need to consider:

1. $(\forall y \in N : \langle\rangle = \langle y \rangle \Rightarrow A(y)) \Rightarrow A(\langle\rangle)$
2. $\forall x' \in N : (\forall y \in N : \langle x' \rangle = \langle y \rangle \Rightarrow A(y)) \Rightarrow A(\langle x' \rangle)$

The first case simplifies to $A(\langle\rangle)$ (i.e., the first premise of the induction principle for N) because $\langle\rangle = \langle y \rangle$ does not hold for any $y \in N$ and hence the left side of the outer implication is trivially true. The second case simplifies to

$$\forall x' \in N : (\forall y \in N : x' = y \Rightarrow A(y)) \Rightarrow A(\langle x' \rangle)$$

which can be further simplified to just

$$\forall x' \in N : A(x') \Rightarrow A(\langle x' \rangle) \ .$$

Renaming x' to x yields the second premise of the induction principle for N.

Note that well-founded relations are also closely related to the notion of termination measures that we discussed in Section 1.3.1.

2.3 Lists

Lists are one of the most important data structures in functional programming languages. We will consider lists that are sequences of data values of some common element type, e.g., a sequence of integer numbers $3, 6, 1, 2$. Unlike linked lists, which you have studied in your Data Structures course, lists in functional programming languages are *immutable*. That is, once a list has been created, it cannot be changed, e.g. by removing or adding an element in the middle of the list. Such data structures are also called *persistent*.

Persistent data structures have the advantage that their representation in memory can be shared across different instances of the data structure. For example, the two lists $1, 4, 3$ and $5, 2, 4, 3$ have the common sublist $4, 3$. If the two lists are stored in memory at the same time, the shared sublist $4, 3$ only needs to be represented once. If used properly, this feature yields space-efficient, high-level implementations of algorithms over persistent lists. In this section, we will define persistent lists using structural recursion and see that this definition corresponds to the `List` data type defined in Scala's standard library.

2.3.1 Defining Lists using Structural Recursion

Mathematically, we can represent lists of integers as nested tuples. For example, the empty list is represented by the empty tuple $\langle\rangle$, and the list containing the sequence of numbers 5, 2, and 3 is represented by the tuple $\langle 5, \langle 2, \langle 3, \langle\rangle\rangle\rangle\rangle$. The following structurally recursive definition formalizes this idea:

$$\langle\rangle \in \mathit{List} \qquad\qquad \frac{hd \in \mathbb{Z} \qquad tl \in \mathit{List}}{\langle hd, tl\rangle \in \mathit{List}}$$

For a non-empty list ℓ of the form $\langle hd, tl\rangle$, we refer to the integer number hd as the *head* of ℓ, and we call the remaining list tl the *tail* of ℓ. For example, the head of the list $\langle 4, \langle 2, \langle\rangle\rangle\rangle$ is 4 and its tail is $\langle 2, \langle\rangle\rangle$. We also refer to a non-empty list as a *cons cell*. To improve readability, we denote the empty list $\langle\rangle$ by *nil* and write $hd :: tl$ for a cons cell $\langle hd, tl\rangle$. We treat $::$ as a right-associative constructor for lists. That is, $x :: y :: tl$ is the list $\langle x, \langle y, tl\rangle\rangle$.

In Scala, we can define lists of integers using an algebraic data type[8]:

```
enum List:
  case Nil
  case Cons(hd: Int, tl: List)
```

Here, `Nil` represents the empty list and a cons cell $hd :: tl$ is represented by `Cons(hd, tl)`. Note again the close resemblance between the mathematical definition and the Scala definition of lists.

As an example, let us construct a Scala list containing the values $1, 4, 2$:

[8] The Scala standard library actually provides a generic `List` type that is parametric in its element type. The definition of this type is similar to the one that we give here. We will study Scala's `List` type more closely later.

```
scala> val l = Cons(1, Cons(4, Cons(2, Nil)))
val l: List = Cons(1, Cons(4, Cons(2, Nil)))
```

We can also use pattern matching to deconstruct lists into their components:

```
scala> val Cons(h, t) = l
val h: Int = 1
val t: List = Cons(4, Cons(2, Nil))

scala> l match
         case Nil => println("l␣is␣empty")
         case Cons(h, t) => println(s"l's␣head␣is␣$h.")
l's␣head␣is␣1.
```

2.3.2 Functions on Lists

Using structural recursion we can now define simple functions on lists. For example, the following function computes the length of a given list:

$$\begin{aligned} \mathit{length} &: \mathit{List} \to \mathbb{N} \\ \mathit{length}(\mathit{nil}) &= 0 \\ \mathit{length}(\mathit{hd} :: \mathit{tl}) &= 1 + \mathit{length}(\mathit{tl}) \end{aligned}$$

The next function is more interesting, it takes two lists ℓ_1 and ℓ_2 and creates a new list by concatenating ℓ_1 and ℓ_2.

$$\begin{aligned} \mathit{append} &: \mathit{List} \times \mathit{List} \to \mathit{List} \\ \mathit{append}(\mathit{nil}, \ell_2) &= \ell_2 \\ \mathit{append}(\mathit{hd} :: \mathit{tl}, \ell_2) &= \mathit{hd} :: \mathit{append}(\mathit{tl}, \ell_2) \end{aligned}$$

For example, for $\ell_1 = 4 :: 6 :: 1 :: \mathit{nil}$ and $\ell_2 = 5 :: 1 :: \mathit{nil}$ we get

$$\mathit{append}(\ell_1, \ell_2) = 4 :: 6 :: 1 :: 5 :: 1 :: \mathit{nil} \ .$$

Finally, using *append* we can define a function *reverse* that takes a list ℓ and creates a new list that contains the elements of ℓ in reverse order:

$$\begin{aligned} \mathit{reverse} &: \mathit{List} \to \mathit{List} \\ \mathit{reverse}(\mathit{nil}) &= \mathit{nil} \\ \mathit{reverse}(\mathit{hd} :: \mathit{tl}) &= \mathit{append}(\mathit{reverse}(\mathit{tl}), \mathit{hd} :: \mathit{nil}) \end{aligned}$$

For example, we have $\mathit{reverse}(4 :: 2 :: \mathit{nil}) = 2 :: 4 :: \mathit{nil}$.

Note that the definition of *reverse* is still structurally recursive since in the recursive case *reverse* is only applied to the tail *tl* of the input list.

The mathematical definitions of *length*, *append*, and *reverse* directly translate to corresponding Scala functions:

```
def length(l: List): Int = l match
  case Nil => 0
  case Cons(hd, tl) => 1 + length(tl)

def append(l1: List, l2: List): List = l1 match
  case Nil => l2
  case Cons(hd, tl) => Cons(hd, append(tl, l2))

def reverse(l: List): List = l match
  case Nil => Nil
  case Cons(hd, tl) => append(reverse(tl), Cons(hd, Nil))
```

Unfortunately, these Scala functions are not very efficient. For example, the running time of `reverse` is quadratic in the length of the list `l`. Moreover, the `reverse` function is not tail-recursive and hence requires linear space in the length of `l`. The implementations of `length` and `append` are also not tail-recursive. While we typically do not care about computational efficiency when we define mathematical functions, we do care about it when we write programs. To obtain efficient implementations we would rather implement these functions tail-recursively. For example, we can rewrite `reverse` so that it runs in linear time and constant space:

```
def reverse2(l: List): List =
  def rev(l: List, acc: List): List = l match
    case Nil => acc
    case Cons(h, t) => rev(t, Cons(h, acc))
  rev(l, Nil)
```

Exercise 2.1. *Give the mathematical definition of the tail-recursive* `reverse2` *function. Call this function* $reverse_2$*. Hint: to define* $reverse_2$*, first define an auxiliary function* $rev : List \times List \to List$*.*

Exercise 2.2. *Define tail-recursive Scala versions of the functions* `length` *and* `append`*. Hint: use* `reverse2` *in the definition of* `append`*.*

2.3.3 Proving Properties of Functions on Lists

Lists are defined by structural recursion. Hence, we can use structural induction to prove properties about functions (and programs) that operate on lists. Following the discussion in Section 2.2.3, we derive the following structural induction principle for *List*:

$$\frac{A(nil) \qquad \forall hd \in \mathbb{N}, tl \in List : A(tl) \Rightarrow A(hd :: tl)}{\forall \ell \in List : \ A(\ell)}$$

As an example, the following proposition states that the length of a list obtained by appending two lists ℓ_1 and ℓ_2 is equal to the sum of the lengths of ℓ_1 and ℓ_2.

Proposition 2.4. *For all $\ell_1, \ell_2 \in List$ the following property holds*

$$length(append(\ell_1, \ell_2)) = length(\ell_1) + length(\ell_2) \ .$$

Proof. Let $\ell_1, \ell_2 \in List$. Since *append* is defined by structural recursion on its first argument, the proof proceeds by structural induction on ℓ_1: Base case: assume $\ell_1 = nil$. Then

$$\begin{aligned}
length(append(\ell_1, \ell_2)) &= length(append(nil, \ell_2)) & \\
&= length(\ell_2) & \text{Def. of } append \\
&= 0 + length(\ell_2) & 0 \text{ is neutral element} \\
&= length(nil) + length(\ell_2) & \text{Def. of } length \\
&= length(\ell_1) + length(\ell_2) &
\end{aligned}$$

Induction step: assume $\ell_1 = hd :: tl$. Then

$$\begin{aligned}
length(append(\ell_1, \ell_2)) &= length(append(hd :: tl, \ell_2)) & \\
&= length(hd :: append(tl, \ell_2)) & \text{Def. of } append \\
&= 1 + length(append(tl, \ell_2)) & \text{Def. of } length \\
&= 1 + (length(tl) + length(\ell_2)) & \text{Induction hypothesis} \\
&= (1 + length(tl)) + length(\ell_2) & \text{Associativity of } + \\
&= length(hd :: tl) + length(\ell_2) & \text{Def. of } length \\
&= length(\ell_1) + length(\ell_2) &
\end{aligned}$$

□

Exercise 2.3. *For the function $reverse_2$ that you defined in Exercise 2.1, use structural induction to prove that it computes the same function as reverse. That is, for all $\ell \in List$, $reverse(\ell) = reverse_2(\ell)$.*

Chapter 3

Syntax

When we describe a programming language, we distinguish between the *syntax* and the *semantics* of the language. The syntax describes the structure of a program (i.e., how a program is represented), whereas the semantics describes its meaning (i.e., what the program computes). In order to understand programming languages, it is important to keep these two concepts separated. In particular, we use different mathematical objects to represent syntax and semantics.

When we talk about the syntax of a programming language, we further distinguish between its *concrete syntax* and its *abstract syntax*. The concrete syntax defines which sequences of characters represent programs (i.e., the actual source code stored in text files). The *abstract syntax* describes the structure of a program as an abstract syntax tree. Such a tree abstracts from some of the specifics of the concrete syntax, such as parenthesis in expressions, semicolons after statements, etc. The abstract syntax also makes the precedence and associativity of operators explicit. Abstract syntax trees are used to represent programs internally in a compiler or interpreter.

3.1 Concrete Syntax (optional)

The *parsing problem* is concerned with the conversion of program source code represented as sequences of characters (i.e., concrete syntax) into abstract syntax trees. This is a well-understood problem and you can learn more about it in a compiler construction course. In this course, we will mostly side-step the parsing problem and directly work with abstract syntax trees. Nevertheless, it is useful to have a basic understanding of how the concrete syntax of a programming language can be formalized and what the typical problems are when writing parsers.

3.1.1 Formal Languages

In computer science, we formally describe languages as sets of words. Each word is a finite sequence of symbols drawn from a set that we call the *alphabet* of the language. For example, we can describe an arithmetic expression "3 + 5 * 8" as a word that is given by the sequence of symbols '3', '+', '5', '*', and '8'.

Given an alphabel Σ, we denote by Σ^* the set of all finite words that can be formed using the symbols in Σ[1] The *empty word* is denoted by ϵ.

To describe languages in a compact form, we use *grammars*. A grammar is given by a set of rules, called *productions*. Productions tell us how the words of the language can be constructed from the symbols in the alphabet. For example, the following grammar describes the language of all arithmetic expressions:

$$
\begin{aligned}
&E \to E\,O\,E \\
&E \to (E) \\
&E \to x \qquad \text{where } x \in \mathbb{Z} \\
&O \to \texttt{+} \\
&O \to - \\
&O \to \texttt{*} \\
&O \to \texttt{/}
\end{aligned}
$$

Note that the third rule actually stands for an infinite set of productions (one for each $x \in \mathbb{Z}$):

$$
\begin{aligned}
&\dots \\
&E \to -2 \\
&E \to -1 \\
&E \to 0 \\
&E \to 1 \\
&E \to 2 \\
&\dots
\end{aligned}
$$

We call the uppercase symbols that occur on the left-hand sides of productions, such as E and O, *nonterminal* symbols. The remaining symbols that are drawn from the alphabet of the language such as + and 3 are called *terminal* symbols.

Each grammar has an associated (nonterminal) starting symbol. In our example grammar, this is the symbol E. Starting from this symbol we apply the productions one by one until we obtain a word that consists only of terminal symbols. In each step, we pick one nonterminal in the current working word, choose a production in which this nonterminal occurs on the left-hand side, and replace the chosen nonterminal in the working word by the right-hand side of the chosen production. We call this process *derivation*.

[1] The notation Σ^* should not to be confused with the notation R^* introduced earlier where R is a binary relation, which denotes the reflexive transitive closure of R.

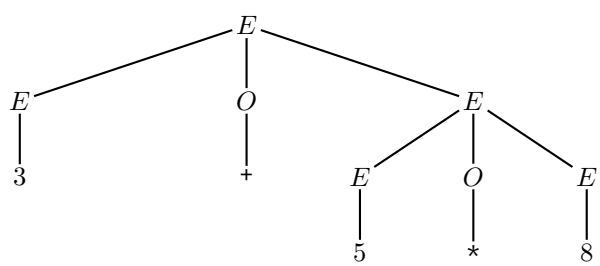
Using the above productions, we can derive words such as

$$
\begin{array}{l}
1 \\
3 + 5 * 8 \\
-14 \,/\, (42 + (0 - 1))
\end{array}
$$

Here is a derivation of the word $3 + 5 * 8$:

$$
\begin{aligned}
E &\Rightarrow E\,O\,E \\
&\Rightarrow 3\,O\,E \\
&\Rightarrow 3 + E \\
&\Rightarrow 3 + E\,O\,E \\
&\Rightarrow 3 + 5\,O\,E \\
&\Rightarrow 3 + 5 * E \\
&\Rightarrow 3 + 5 * 8
\end{aligned}
$$

Alternatively, we can represent this derivation by its *parse tree*:



The problem of constructing a parse tree for a given word in a language is the *parsing problem*. This problem can be solved automatically for the important class of *context-free languages*. In a context-free language, each derivation step rewrites a single nonterminal symbol in the working word regardless of the context in which this nonterminal occurs. The concrete syntax of most programming languages is context-free. So called parser generators can automatically construct parsers for context-free languages from a description of their grammar. We next define this class of languages and their associated grammars formally.

3.1.2 Context-Free Languages and Grammars

A *context-free grammar* is a tuple $G = (\Sigma, N, P, S)$ where

- Σ is a finite set of terminal symbols,
- N is a finite set of nonterminal symbols disjoint from Σ,
- $P \subseteq (N, (\Sigma \cup N)^*)$ is a finite set of productions, and
- $S \in N$ is the starting symbol.

We denote a production $(X, w) \in P$ by $X \to w$.

Let $G = (\Sigma, N, P, S)$ be a context-free grammar. For any two words, $u, v \in (\Sigma \cup N)^*$, we say v directly derives from u, written $u \Rightarrow v$, if there exists a production $X \to w$ in P and $u_1, u_2 \in (\Sigma \cup N)^*$ such that $u = u_1 X u_2$ and $v = u_1 w u_2$. That is, v is the result of applying $X \to w$ to u. We denote by $\Rightarrow^*$ the reflexive and transitive closure of the relation $\Rightarrow$ and we say that v derives from u if $u \Rightarrow^* v$.

The language of G, denoted $\mathcal{L}(G)$, is the set of all terminal words that can be derived from S:

$$\mathcal{L}(G) = \{\, w \in \Sigma^* \mid S \Rightarrow^* w \,\}$$

A language $\mathcal{L} \subseteq \Sigma^*$ is called context-free if it is the language of some context-free grammar G.

Note that the grammar for arithmetic expressions that we gave above is technically not a context-free grammar because the set of productions (as well as the set of terminal symbols) is infinite. For now, we will skim over this technicality. We will see later how we obtain a proper context-free grammar for arithmetic expressions.

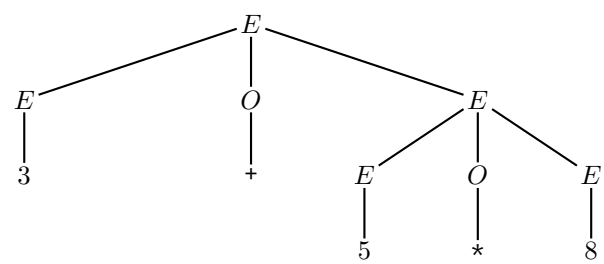
3.1.3 Backus-Naur-Form

Often, context-free grammars are given in so-called *Backus-Naur-Form* (BNF). In this form, we use the symbol ::= instead of $\to$ to separate the two sides of a production. Moreover, in a BNF, productions $X \to w_1, \ldots, X \to w_n$ for the same nonterminal symbol X can be summarized by a single rule $X ::= w_1 \mid \cdots \mid w_n$. For example, here is our grammar of arithmetic expressions in BNF:

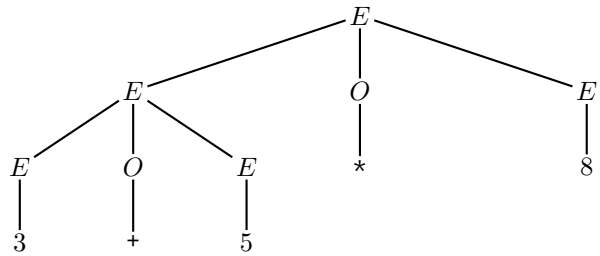
$$\begin{aligned} x &\in \mathbb{Z} \\ E &::= E\,O\,E \mid (E) \mid x \\ O &::= \texttt{+} \mid - \mid \texttt{*} \mid \texttt{/} \end{aligned}$$

3.1.4 Eliminating Ambiguity

Let us reconsider our grammar for arithmetic expressions and the derivation of the expression "3 + 5 * 8" given by the following parse tree:



This is not the only possible derivation of "3 + 5 * 8". Another one is given by the following parse tree:



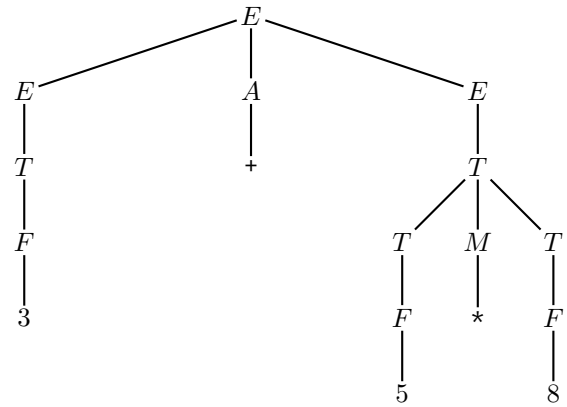
We call a grammar in which a word has more than one derivation *ambiguous*. Ambiguity is a problem because the semantics of programs is given in terms of their abstract syntax trees, which are derived from parse trees. Typically, the semantics of a program depends on the structure of its parse tree. For example, with the canonical semantics of arithmetic expressions, the first parse tree of the expression "3 + 5 * 8" would evaluate to 43 whereas the second parse tree would evaluate to 64. Ideally, we would like to change our grammar so that the second parse tree no longer represents a valid derivation, formalizing the rule of arithmetic notation stating "multiplication before addition". This can be done by augmenting the grammar with additional disambiguation rules.

The problem of detecting whether a given context-free grammar is ambiguous is undecidable. Consequently, there does not exist a general algorithm that turns an ambiguous grammar into an unambiguous one. We therefore have to make do with ad hoc techniques for resolving ambiguities. Fortunately, for grammars that describe programming languages, there exist some general recipes that work well in practice.

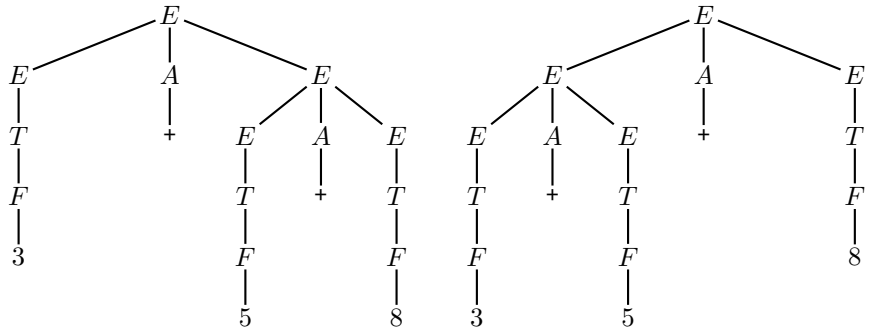
The ambiguity in our arithmetic expression grammar that we have observed for the expression "3 + 5 * 8" stems from the fact that the grammar does not distinguish between the additive operators, '+' and '-', and the multiplicative operators, '*' and '/'. We would like the multiplicative operators to *bind stronger* than the additive operators. We also say that the multiplicative operators have higher *precedence*. We can encode operator precedence by grouping expressions based on the types of operators and changing the productions so that expressions are expanded in the right order:

$$
\begin{aligned}
E &::= E\ A\ E \mid T \\
A &::= \texttt{+} \mid \texttt{-} \\
T &::= T\ M\ T \mid F \\
M &::= \texttt{*} \mid \texttt{/} \\
F &::= x \mid (E)
\end{aligned}
$$

Now the only valid parse tree for the expression "3 + 5 * 8" is the tree:



Our grammar is still ambiguous, though. For example, consider the expression "3 + 5 + 8". Here are two possible parse trees for this expression:



It seems that this ambiguity does not matter from a semantic point of view because addition on the integers is associative. That is, both trees would evaluate to 16. However, in computer programs we are normally working with bounded representations of integers. In this case, the arithmetic operations are often not associative due to potential arithmetic overflow. We would therefore like the operations to be parsed in a specific order. For example, arithmetic operators are usually defined as left-associative rather than right-associative, which means that the expression "3 + 5 + 8" should be parsed similar to "(3 + 5) + 8" rather than "3 +(5 + 8)".

To encode left-associativity of operators in our grammar, we can replace the right side of each binary expression by the base case of that expression type. This will force the repetitive matches of subexpressions onto the left side:

$$
\begin{aligned}
E &::= E\,A\,T \mid T \\
A &::= \texttt{+} \mid \texttt{-} \\
T &::= T\,M\,F \mid F \\
M &::= \texttt{*} \mid \texttt{/} \\
F &::= x \mid (E)
\end{aligned}
$$

3.1.5 Regular Languages

Finally, let us modify our grammar for arithmetic expressions so that it is actually context-free, i.e., so that the terminal symbols and productions are finite sets. We do this in two steps. First, we define a context-free grammar that describes integer numbers in decimal representation:

$$
\begin{aligned}
Z &::= \texttt{-}\,H \mid H \\
H &::= 0 \mid 1T \mid \cdots \mid 9T \\
T &::= \epsilon \mid 0T \mid \cdots \mid 9T
\end{aligned}
$$

The starting symbol of this grammar is Z and the terminal symbols are $\Sigma = \{\texttt{-}, 0, \ldots, 9\}$.

Next, we combine this grammar with the grammar:

$$
\begin{aligned}
E &::= E\,A\,T \mid T \\
A &::= \texttt{+} \mid \texttt{-} \\
T &::= T\,M\,F \mid F \\
M &::= \texttt{*} \mid \texttt{/} \\
F &::= Z \mid (E)
\end{aligned}
$$

to obtain a context-free grammar for arithmetic expressions.

The productions of our grammar for integer numbers have a special form. They all match one of the following shapes:

$$
\begin{aligned}
X &::= \epsilon \\
X &::= a \\
X &::= Y \\
X &::= aY
\end{aligned}
$$

where X, Y are nonterminals and a is a terminal symbol. Grammars in which all productions are of these shapes form a special subclass of context-free grammars, called *regular grammars*. The languages of these grammar are correspondingly called *regular languages*. Another way to describe such languages are *regular expressions*, which you may already be familiar with.

Regular languages can be parsed more efficiently than general context-free languages. Compilers therefore split the parsing of the input program into two phases: a so-called *lexing phase* in which the character sequence representing the input program is converted into a token sequence, and the actual parsing phase in which the token sequence is converted into a parse tree. The tokens in the token sequence are subsequences of characters in the input program that have been grouped together, e.g., to form keywords of the language or numbers (as in the example of our arithmetic expression language). The program that takes care of the lexing phase is called *lexer* or *tokenizer*. The lexer is typically

automatically generated from regular expressions, whereas the actual parser is automatically generated from a context-free grammar defined over the token alphabet.

3.2 Abstract Syntax

In the previous section, we have learned that grammars define formal languages, which are sets of sequences of characters over some alphabet. We refer to this interpretation of a grammar as the concrete syntax of a language. Alternatively, we can also interpret grammars as structural recursive definitions of certain sets of tuples (representing trees). In this view, we speak of the abstract syntax of a language. The abstract syntax abstracts from aspects of the concrete syntax that are only relevant for parsing. This includes, e.g., disambiguation rules for operator precedence and associativity, parenthesis, language keywords, etc. In this section, we study the mathematical objects that describe the abstract syntax of programming languages.

3.2.1 Abstract Syntax Trees

We use Backus-Naur-Form (BNF) notation to describe the abstract syntax of a language. In order to make it easier to detect whether the grammar defines the concrete or abstract syntax of a language, we write the productions of grammars for the abstract syntax as definitions of sets. As an example, consider the following grammar that defines the abstract syntax of an arithmetic expression language:

$$
\begin{array}{rr}
n \in \mathit{Num} & \text{numbers} \\
x \in \mathit{Var} & \text{variables} \\
e \in \mathit{Expr} ::= n \mid x \mid e_1 \, \mathit{bop} \, e_2 & \text{expressions} \\
\mathit{bop} \in \mathit{Bop} ::= \texttt{+} \mid \texttt{*} & \text{binary operators}
\end{array}
$$

Note that in the definition of *Expr*, the (meta) variables e_1 and e_2 also range over expressions *Expr*. In general, we will follow the convention that in recursive grammar definitions we only declare a generic variable for each set that we define, in this case the variable e for the set *Expr*. We then assume that all variables that have the same name but possibly different indices, here the variables e_1 and e_2, also range over the same set.

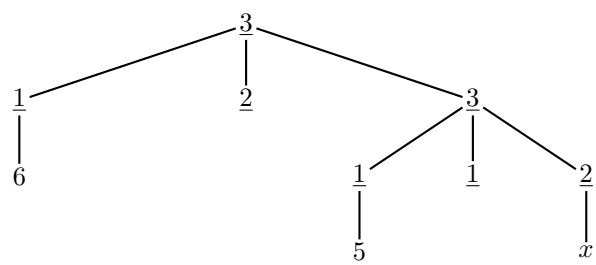
The expression language includes variables x which are drawn from an (infinite) set of variable names *Var*. The sets *Num* and *Var* are parameters of the grammar definition. In the following, you may assume that these two sets are just referring to integer numbers $\mathit{Num} = \mathit{Var} = \mathbb{Z}$. Later in our implementation of arithmetic expressions, which will be part of our interpreter, we will identify *Num* with the type of double-precision floating point numbers, and *Var* with the type of strings.

Our grammar borrows the notation of the concrete syntax to represent the elements of the sets *Expr* and *Bop*. However, we just use the concrete syntax to "sugarcoat" the actual mathematical representation of abstract syntax trees. We now describe this representation formally.

As a first step, we tag the productions for each set in the grammar with a unique number. We refer to these tags as *variant numbers*. In our running example, we obtain the following tagged grammar of arithmetic expressions (for clarity, the variant numbers are underlined):

$$e \in \mathit{Expr} ::= \underline{1} : n \mid \underline{2} : x \mid \underline{3} : e_1\, \mathit{bop}\, e_2 \qquad \text{expressions}$$
$$\mathit{bop} \in \mathit{Bop} ::= \underline{1} : \texttt{+} \mid \underline{2} : \texttt{*} \qquad \text{binary operators}$$

An abstract syntax tree is similar to a parse tree, except that the nodes of the tree are labeled by the variant numbers of the productions that were used in the derivation of the expression. For example, the concrete syntactic expression $6 \texttt{*} (5 \texttt{+} x)$ is notational sugar for the following abstract syntax tree:



We can formally represent such trees by nested tuples. Each node of the tree is represented by a tuple consisting of the variant number that labels that node, followed by the sequence of tuples representing all the subtrees rooted in the children of that node. For example, the abstract syntax tree above stands for the following tuple

$$\langle \underline{3}, \langle \underline{1}, 6\rangle, \langle \underline{2}\rangle, \langle \underline{3}, \langle \underline{1}, 5\rangle, \langle \underline{1}\rangle, \langle \underline{2}, x\rangle\rangle\rangle \ .$$

Consequently, the grammar rules for the sets *Expr* and *Bop* really stand for inference rules that define the sets *Expr* and *Bop* using structural recursion on tuples:

$$\frac{n \in \mathit{Num}}{\langle \underline{1}, n\rangle \in \mathit{Expr}} \qquad \frac{x \in \mathit{Var}}{\langle \underline{2}, x\rangle \in \mathit{Expr}} \qquad \frac{e_1, e_2 \in \mathit{Expr} \qquad \mathit{bop} \in \mathit{Bop}}{\langle \underline{3}, e_1, \mathit{bop}, e_2\rangle \in \mathit{Expr}}$$

$$\langle \underline{1}\rangle \in \mathit{Bop} \qquad \langle \underline{2}\rangle \in \mathit{Bop}$$

By using tuples and inference rules, we obtain a mathematical precise definition of the abstract syntax of a language. However, the tuple representation is notationally heavy and cumbersome to work with. We will therefore continue to represent abstract syntax trees using concrete syntax. Often we will even omit

the definition of the exact concrete syntax, relying instead on our intuition to map concrete to abstract syntax and vice versa using common conventions for operator precedence, etc. This will sometimes lead to ambiguities. For example, in the case of our arithmetic expression language, the object 4 may now stand for the number 4, the concrete expression consisting of the terminal symbol 4, and the abstract syntax tree $\langle \underline{1}, 4 \rangle$ of that expression. This ambiguity might be confusing in the beginning. However, it will always be clear from the context which of these mathematical objects we are referring to. As you get more familiar with these concepts, you will be able to easily distinguish between them.

Since expressions are defined using structural recursion, we can also use structural recursion to define functions on expressions. For example, we can define a function ov that takes an expression e and computes the set of all variables occuring in e:

$$\begin{aligned} ov &: \mathit{Expr} \to 2^{\mathit{Var}} \\ ov(n) &= \emptyset \\ ov(x) &= \{x\} \\ ov(e_1 \, \mathit{bop} \, e_2) &= ov(e_1) \cup ov(e_2) \end{aligned}$$

Recall from Section 2.1 that by 2^{Var} we denote the powerset of the set Var.

It is instructive to compare our definition of the abstract syntax of arithmetic expressions with a corresponding definition given in terms of enums in Scala. This can be done as follows:

```
enum Expr:
  case Num(n: Double)
  case Var(x: String)
  case BinOp(bop: Bop, e1: Expr, e2: Expr)

enum Bop:
  case Plus, Times
```

Here, the variant constructors `Num`, `Var`, `BinOp` and so forth, play the role of the variant numbers in our mathematical representation of the abstract syntax.

Note that we have introduced a minor discrepancy to the mathematical definition of *Expr* and its Scala representation using the type `Expr`: in the `BinOp` case the binary operator `bop` is the first argument (i.e. the left-most child of the node) rather than the second argument. This change in the representation slightly improves the readability of the code in the following, but it is otherwise nonessential.

We can then define the Scala version of the function ov:

```
def ov(e: Expr): Set[String] =
  e match
    case Num(n) => Set()
    case Var(x) => Set(x)
    case BinOp(_, e1, e2) => ov(e1) ++ ov(e2)
```

Note that in the function `ov` we are using the generic type `Set` from the Scala standard library, which can represent finite sets of objects. That is, the type `Set[String]` represents all finite sets of string objects.

3.2.2 Environments and Expression Evaluation

Consider the expression $e = x \star (y + 6)$. If we provide values for the variables x and y, we can compute the value of the entire expression. For example, the expression e evaluates to 12, if we assign $x = 1$ and $y = 2$.

An *environment* is a partial function $env : Var \rightharpoonup Num$, that assigns variables to values. Recall that we represent (partial) functions as sets of pairs. For example, the environment env that assigns x to 1 and y to 2 is denoted by

$$env = \{(x, 1), (y, 2)\}$$

We typically use the following arrow notation for the individual variable assignments in an environment

$$env = \{x \mapsto 1, y \mapsto 2\}$$

Moreover, for an environment env, we write $env[x \mapsto v]$ for the environment that is like env but maps x to the value v:

$$env[x \mapsto v](y) = \begin{cases} env(y) & \text{if } y \neq x \\ v & \text{otherwise} \end{cases}$$

We denote the set of all environment by Env.

Given an environment that assigns values to all the variables in an expression, we can evaluate that expression. We formalize this idea by defining a function $eval$ using structural recursion:

$$\begin{aligned} eval &: Env \times Expr \to Num \\ eval(env, n) &= n \\ eval(env, x) &= env(x) \\ eval(env, e_1 + e_2) &= eval(env, e_1) + eval(env, e_2) \\ eval(env, e_1 \star e_2) &= eval(env, e_1) \cdot eval(env, e_2) \end{aligned}$$

Note that in the third case the symbol + occurs on both sides of the equation. On the left side, it stands for the abstract syntax of the addition operator, i.e., the object $\langle \underline{1} \rangle \in Bop$. On the right side it stands for the mathematical addition operation on integers. We use different fonts to distinguish these different usages of the symbol.

For a given environment env and expression e, $eval(env, e)$ is only well-defined if env is defined on all variables occurring in e. Mathematically, we can express this condition by $ov(e) \subseteq \mathsf{dom}(env)$. Recall that dom is the function that maps a partial function f to its *domain*, i.e., the set of values on which f is defined.

Intuitively, we can think of an environment *env* as the mathematical representation of the program stack, which keeps track of the values of local variables during program execution.

Again, we can directly translate the definition of our function *eval* to a corresponding Scala function. We represent environments in Scala using the type `Env`. We define this type in terms of the `Map` type in the Scala standard library, which we can use to represent finite partial functions:

```
type Env = Map[String, Double]
def dom(env: Env): Set[String] = env.keySet
```

The Scala version of the function *eval* is then defined as follows:

```
def eval(env: Env, e: Expr): Double =
  e match
    case Num(n) => n
    case Var(x) => env(x)
    case BinOp(Plus, e1, e2) =>
      eval(env, e1) + eval(env, e2)
    case BinOp(Times, e1, e2) =>
      eval(env, e2) * eval(env, e2)
```

In order to encode the required precondition of *eval* efficiently, we can rewrite this function using a nested helper function as follows:

```
def eval(env: Env, e: Expr): Double =
  require (ov(e) subsetOf dom(env))
  def eval(e: Expr): Double = e match
    case Num(n) => n
    case Var(x) => env(x)
    case BinOp(Plus, e1, e2) => eval(e1) + eval(e2)
    case BinOp(Times, e1, e2) => eval(e2) * eval(e2)
  eval(e)
```

3.2.3 Substitutions

When we do calculations with expressions, we often have to replace subexpressions by other expressions. We call such replacements *substitutions*. Particularly important are substitution operations that replace variables by expressions.

Consider the following expression:

$$3 * x + x$$

This expression contains two occurrences of the variable x. If we replace both of these occurrences by the expression $x + 4$, we obtain the expression:

$$3 * (x + 4) + (x + 4)$$

Let e and e_s be expressions and $x \in Var$ a variable, then we denote by

$$e[e_s/x]$$

the expression that we obtain by replacing all occurrences of x in e by e_s. In particular, we have:

$$(3 * x + x)[(x + 4)/x] = 3 * (x + 4) + (x + 4)$$

We can define the substitution function formally using structural recursion:

$$\begin{aligned} _[_/_] &: \mathit{Expr} \times \mathit{Var} \times \mathit{Expr} \to \mathit{Expr} \\ n[e_s/x] &= n \\ y[e_s/x] &= \textbf{if } x = y \textbf{ then } e_s \textbf{ else } y \\ (e_1 \, \mathit{bop} \, e_2)[e_s/x] &= (e_1[e_s/x]) \, \mathit{bop} \, (e_2[e_s/x]) \end{aligned}$$

The Scala version of the substitution function looks as follows:

```
def subst(e: Expr, x: String, es: Expr): Expr =
  e match
    case Num(_) => e
    case Var(y) => if x == y then es else e
    case BinOp(bop, e1, e2) =>
      BinOp(bop, subst(e1, x, es), subst(e2, x, es))
```

The following Lemma captures an important property that relates substitution and expression evaluation:

Lemma 3.1 (Substitution Lemma)**.** *Let $e, e_s \in \mathit{Expr}$, $x \in \mathit{Var}$, and $\mathit{env} \in \mathit{Env}$ such that $\mathit{ov}(e) \cup \mathit{ov}(e_s) \subseteq \mathsf{dom}(\mathit{env})$. Then*

$$\mathit{eval}(\mathit{env}, e[e_s/x]) = \mathit{eval}(\mathit{env}[x \mapsto \mathit{eval}(\mathit{env}, e_s)], e)$$

Proof. By structural induction on e (exercise). □

3.3 Binding and Scoping

In programming languages, identifiers are used as names for values. They are introduced through variable and constant declarations, procedures, class declarations, etc. During evaluation, identifiers are bound to values. The lifetime of such a binding is determined by the *scope* of the binding. We will study basic binding and scoping mechanisms using a simple language of arithmetic expressions with constant declarations. We also study an important operation on expressions called *substitution*, which is crucial for many tasks related to programming languages, including program evaluation, optimization, and refactoring.

3.3.1 Expressions with Constant Declarations

We extend our grammar of arithmetic expressions from Section 3.2 with constant declarations of the form:

$$\textbf{const } x = e_d \,;\, e_b$$

During evaluation of the constant declaration, the variable x is bound to the value of the expression e_d so that occurrences of x in e_b refer to that value, much like a `val` declaration in Scala. We call e_d the *defining expression* of the declaration and e_b the *body* of the declaration. We also refer to e_b as the *scope* of the binding of x.

The complete abstract syntax of our extended language is now as follows:

$n \in \mathit{Num}$	numbers
$x \in \mathit{Var}$	variables
$e \in \mathit{Expr} ::= n \mid x \mid e_1\, \mathit{bop}\, e_2 \mid \texttt{const}\ x = e_d\,;\ e_b$	expressions
$\mathit{bop} \in \mathit{Bop} ::= \texttt{+} \mid \texttt{*}$	binary operators

Note that in our concrete syntax, we omit parenthesis in sequenced **const** declarations. For example, the expression

$$\texttt{const}\ x = 5\,;\ \texttt{const}\ y = x + 2\,;\ \texttt{const}\ z = y * 5\,;\ z + z$$

is implicitly parenthesized as follows:

$$\texttt{const}\ x = 5\,;\ (\texttt{const}\ y = x + 2\,;\ (\texttt{const}\ z = y * 5\,;\ z + z))$$

However, for clarity, we will use parenthesis around **const** declarations that occur nested inside the defining expressions of other **const** declarations:

$$\texttt{const}\ y = (\texttt{const}\ x = 5\,;\ x + 2)\,;\ \texttt{const}\ z = y * 5\,;\ z + z$$

We distinguish between *defining occurrences* and *using occurrences* of variables. A defining occurrence introduces a new binding during evaluation of an expression, whereas a using occurrence yields a value according to the current binding of the variable. Consider the following expression:

$$\texttt{const}\ x = 2 * 3\,;\ \texttt{const}\ y = x + 5\,;\ x * (z + y)$$

If we over-line all defining occurrences of variables in this expression, we get the following:

$$\texttt{const}\ \overline{x} = 2 * 3\,;\ \texttt{const}\ \overline{y} = x + 5\,;\ x * (z + y)$$

In addition to defining and using occurrences, we also distinguish between *bound* and *free* occurrences. Intuitively, when you write a program, the bound occurrences of variables refer to variables that are internal to your program (e.g., global and local variables, function parameters, etc.). The free occurrences refer to external dependencies, i.e., the names of things that are not declared in your program but that you use in your program (e.g., the names of functions that are provided by external libraries).

If you consistently rename all bound occurrences of a particular variable using a different (fresh) name, you do not change the behavior of your program. However, if you rename a free variable, you may change the program behavior (e.g., because you change the name of a library function that you are calling, so

after renaming you now call a completely different function). In the rest of this section, we study these concepts in detail.

Defining occurrences of variables are always bound occurrences, whereas using occurrences can be either bound or free. A using occurrence of a variable is bound if it occurs inside the scope of a defining occurrence of the same variable. A using occurrence of a variable that has no associated defining occurrence is free. For example, consider again the expression from above:

$$\texttt{const}\ \overline{x} = 2 * 3;\ \texttt{const}\ \overline{y} = x + 5;\ x * (z + y)$$

The two using occurrences of x in this expression are bound because they are in the scope of the outer **const** declaration, which binds x. Similarly, the using occurrence of y is bound because it is in the scope of the inner **const** declaration, which binds y. The using occurrence of z is free because it is not in the scope of any **const** declaration that binds z.

We can make the binding structure of an expression explicit using arrows:

$$\texttt{const}\ \overline{x} = 2 * 3;\ \texttt{const}\ \overline{y} = x + 5;\ x * (z + y)$$

Every arrow points from a bound using occurrence of a variable to the associated defining occurrence. In the example, the using occurrence of z is the only free occurrence. Hence, this occurrence has no arrow to any binding occurrence.

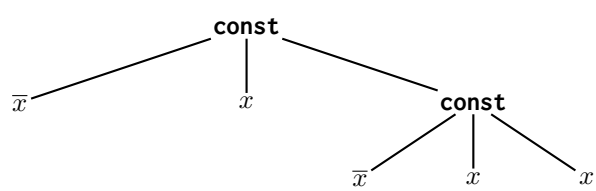
In general, a variable may be bound in more than one place. Variables may also occur both bound and free in the same expression. As an example, consider the following expression:

$$\texttt{const}\ x = x;\ \texttt{const}\ x = x;\ x$$

Here, the using occurrence of x in the defining expression of the outer **const** declaration is free since this occurrence is not in scope of any **const** declaration. The using occurrence of x in the defining expression of the inner **const** declaration is bound by the outer **const** declaration and the using occurrence in the body of the inner **const** declaration is bound by the inner **const** declaration:

$$\texttt{const}\ \overline{x} = x;\ \texttt{const}\ \overline{x} = x;\ x$$

If you have difficulties determining the binding structure of an expression, it is helpful to use the AST representation of the expression. For example, here is the AST for the expression **const** $x = x$; **const** $x = x$; x:



For simplicity, we omit the variant numbers in the AST representation. Note that for **const** nodes, the left child represents the variable defined by that **const** declaration, the middle child represents the defining expression, and the right child represents the body. We have already over-lined all the defining occurrences of variables in the AST.

For any using occurrence of a variable x in the AST, you can determine whether it is bound or free, respectively, find its associated defining occurrence as follows: Search for the node in the tree that represents the specific using occurrence of x you want to analyze. Start from that node and walk the tree upwards through the node's ancestors towards the root of the tree. If you visit a **const** node and you came from the right subtree representing the body of that **const** declaration, check whether the left child of that **const** node is labeled by x. If yes, the using occurrence of x that you are analyzing is a bound occurrence. If the current **const** node is the first node you have seen that defines x, you have found the associated defining occurrence of the using occurrence. If the name of the variable defined in the current **const** node is different from x, continue the upwards traversal to the root. If you reach the root without coming through the body of any **const** node that defines x, the using occurrence of x that you are analyzing is a free occurrence.

With the different notions of variable occurrences in place, we can define functions ov, fv, and bv that, given an expression e, compute the set of all variables occuring in e, the set of all free variables occurring in e, respectively, the set of all bound variables occuring in e.

$$
\begin{aligned}
ov &: \mathit{Expr} \to 2^{\mathit{Var}} \\
ov(n) &= \emptyset \\
ov(x) &= \{x\} \\
ov(e_1 \mathit{bop}\, e_2) &= ov(e_1) \cup ov(e_2) \\
ov(\texttt{const}\ x = e_d\,;\ e_b) &= \{x\} \cup ov(e_d) \cup ov(e_b)
\end{aligned}
$$

$$
\begin{aligned}
fv &: \mathit{Expr} \to 2^{\mathit{Var}} \\
fv(n) &= \emptyset \\
fv(x) &= \{x\} \\
fv(e_1 \mathit{bop}\, e_2) &= fv(e_1) \cup fv(e_2) \\
fv(\texttt{const}\ x = e_d\,;\ e_b) &= (fv(e_b) \setminus \{x\}) \cup fv(e_d)
\end{aligned}
$$

$$
\begin{aligned}
bv &: \mathit{Expr} \to 2^{\mathit{Var}} \\
bv(n) &= \emptyset \\
bv(x) &= \emptyset \\
bv(e_1 \mathit{bop}\, e_2) &= bv(e_1) \cup bv(e_2) \\
bv(\texttt{const}\ x = e_d\,;\ e_b) &= \{x\} \cup bv(e_d) \cup bv(e_b)
\end{aligned}
$$

An expression e is called *closed* if is has no free variables, i.e., e satisfies $fv(e) = \emptyset$. Think of a closed expression as a program that has no external dependencies to symbols defined elsewhere (e.g. in an external library).

3.3.2 Evaluation with Static Binding

The notion of variable binding is strongly related to evaluation because it is during evaluation when a defining occurrence of a variable is bound to a specific value. We can make this formally precise by extending the *eval* function from Section 3.2.2 to our expression language with constant declarations.

As before, the evaluation function will be defined with respect to a value environment $env : Var \rightharpoonup Num$ and we denote by Env the set of all such environments.

Before we give the definition of the *eval* function, let us go through an example. Consider the expression

$$\texttt{const}\ x = x + 2;\ x * y$$

and the environment

$$env = \{x \mapsto 1, y \mapsto 2\}$$

Evaluation of a constant declaration proceeds as follows. First, we evaluate the defining expression $x + 2$ in the current environment env. In particular, the free occurrence of x in this expression is evaluated to $env(x) = 1$. The result of evaluating the expression $x + 1$ is thus 3. Next, we bind the result value to the declared variable x, obtaining a new environment env':

$$env' = env[x \mapsto 3] = \{x \mapsto 3, y \mapsto 2\}$$

Then, we evaluate the body $x * y$ of the constant declaration using the updated environment env', yielding 6 as the value of the entire expression. Note that if the constant declaration is nested within a larger expression, then the updated environment env' is discarded once evaluation of the body is completed (the body of the declaration is the scope of the binding). We then proceed with the original environment env to evaluate some other part of the larger expression. For example, consider the following expression

$$\texttt{const}\ z = (\texttt{const}\ x = x + 2;\ x * y);\ z + x$$

which contains the expression from our earlier example as a subexpression. When we evaluate the outer **const** declaration in the environment env, we obtain 6 for the value of the defining expression of z. We then discard the intermediate environment that we generated during the evaluation of the defining expression of z, and evaluate the body $z + x$ of the outer **const** declaration using the environment:

$$env'' = env[z \mapsto 6] = \{x \mapsto 1, y \mapsto 2, z \mapsto 6\}$$

The result value that we obtain for the entire expression is thus 7.

We can formalize the new evaluation function *eval* using structural recursion as follows:

$$
\begin{aligned}
eval &: Env \times Expr \to Num \\
eval(env, n) &= n \\
eval(env, x) &= env(x) \\
eval(env, e_1 \texttt{+} e_2) &= eval(env, e_1) + eval(env, e_2) \\
eval(env, e_1 \texttt{*} e_2) &= eval(env, e_1) \cdot eval(env, e_2) \\
eval(env, \texttt{const}\ x \texttt{=} e_d \texttt{;}\ e_b) &= \textbf{let}\ \ env' = env[x \mapsto eval(env, e_d)]\ \textbf{in} \\
&\qquad eval(env', e_b)
\end{aligned}
$$

To ensure that *eval* is well-defined we must require that *env* is defined on all free variables of the evaluated expression e, i.e., $fv(e) \subseteq \mathsf{dom}(env)$.

To summarize, the scope of a binding between a variable and its value determines the lifetime of that binding during evaluation of an expression. If the scope of a binding is completely determined by the syntactic structure of the expression, as in the case of our language with constant declarations, we speak of *static binding* and otherwise we speak of *dynamic binding*. We will see examples of dynamic binding later when we introduce procedural abstractions. However, most modern programming languages use only static binding.

3.3.3 Substitutions and Bindings

In Section 3.2.3, we have seen that substitution is a simple affair for languages that do not have any binding constructs. For languages with binding constructs the situation is more complicated because we may run into the problem of *variable capturing*. To understand this problem, consider the expression

$$e = \texttt{const}\ y \texttt{=} 2 \texttt{*} 3\texttt{;}\ y \texttt{+} x$$

and suppose we want to substitute the free occurrence of x in e by the expression y, i.e., compute $e[y/x]$. If we compute the substitution naively, we obtain the expression

$$e' = \texttt{const}\ y \texttt{=} 2 \texttt{*} 3\texttt{;}\ y \texttt{+} y$$

However, this substitution is not correct because in e' the originally free occurrence of y in the expression is *captured* by the constant declaration that binds y. We want to define the substitution function in such a way that variable capturing is avoided. The reason for this is that we always want the substitution property to hold. For our new language, the substitution property can be formulated as follows: for all $env \in Env$, $x \in Var$, $e, e_s \in Expr$, if $fv(e) \cup fv(e_s) \subseteq \mathsf{dom}(env)$, then

$$eval(env, e[e_s/x]) = eval(env[x \mapsto eval(env, e_s)], e)$$

For the example above, the substitution property is violated because for the environment

$$env = \{x \mapsto 1, y \mapsto 2\}$$

we have

$$eval(env, e') = 12$$

whereas

$$\begin{array}{rl} & eval(env[x \mapsto eval(env, y)], e) \\ = & eval(env[x \mapsto 2], e) \\ = & eval(\{x \mapsto 2, y \mapsto 2\}, e) \\ = & 8 \end{array}$$

Before we can define a capture-avoiding substitution function, let us first define a preliminary substitution function *subst* that satisfies the substitution property under certain conditions:

$$\begin{aligned} subst &: Expr \times Var \times Expr \to Expr \\ subst(n, x, e_s) &= n \\ subst(y, x, e_s) &= \textbf{if } x = y \textbf{ then } e_s \textbf{ else } y \\ subst(e_1 \, bop \, e_2, x, e_s) &= subst(e_1, x, e_s) \, bop \, subst(e_2, x, e_s) \\ subst((\texttt{const } y \texttt{=} e_d \texttt{;} \, e_b), x, e_s) &= \textbf{let } e'_b = \textbf{if } x = y \textbf{ then } e_b \textbf{ else } subst(e_b, x, e_s) \textbf{ in} \\ &\qquad \texttt{const } y \texttt{=} subst(e_d, x, e_s) \texttt{;} \, e'_b \end{aligned}$$

A substitution $subst(e, x, e_s)$ that uses the preliminary substitution function satisfies the substitution property provided that none of the free variables of the substituent expression e_s are bound anywhere in e, formally $fv(e_s) \cap bv(e) = \emptyset$. This additional side condition ensures that in $subst(e, x, e_s)$, none of the free variables of e_s will be captured. In particular, this condition is always satisfied if e_s is closed, i.e., $fv(e_s) = \emptyset$.

For example, consider the expression

$$e_r = \texttt{const } z \texttt{=} 2 \texttt{*} 3 \texttt{;} \, z \texttt{+} x$$

then we have

$$subst(e_r, x, y) = \texttt{const } z \texttt{=} 2 \texttt{*} 3 \texttt{;} \, z \texttt{+} y$$

which is a valid substitution.

We now have a substitution function that yields valid substitutions under certain conditions. Still, it is useful to have a substitution function that always satisfies the substitution property, so that we don't have to worry about clumsy side conditions.

The key observation to obtain a general substitution function is that for the evaluation of an expression the actual names of bound variables don't matter. Given an expression, we are free to rename bound variables, as long as we rename them consistently and without variable capturing. Consistent renaming of bound variables does not affect the result of evaluation. For example, the expression e_r defined above is a consistent renaming of our earlier expression

$$e = \texttt{const } y \texttt{=} 2 \texttt{*} 3 \texttt{;} \, y \texttt{+} x$$

In particular, we have for all environments env with $x \in \mathsf{dom}(env)$:

$$eval(env, e) = eval(env, e_r)$$

Thus, in terms of evaluation, the two expressions e and e_r are equivalent. This gives us the following recipe for a general substitution function: given e, x, and e_s, compute $e[e_s/x]$ as follows

- first, consistently rename e to obtain e_r such that $bv(e_r) \cap fv(e_s) = \emptyset$
- and then compute $subst(e_r, x, e_s)$.

In programming language terminology we often speak of *α-renaming* instead of consistent renaming and we call two terms e and e_r *α-equivalent* if e can be obtained from e_r by consistent renaming of bound variables. We can quite easily formalize the intuitive idea of *α-equivalence* by defining an appropriate equivalence relation $\overset{\alpha}{\sim}$ on expressions:

$$\frac{e'_b = subst(e_b, x, y) \qquad y \notin ov(e_b) \qquad e_d \overset{\alpha}{\sim} e'_d}{\texttt{const}\ x = e_d\,;\ e_b \overset{\alpha}{\sim} \texttt{const}\ y = e'_d\,;\ e'_b} \qquad \frac{e_1 \overset{\alpha}{\sim} e'_1 \qquad e_2 \overset{\alpha}{\sim} e'_2}{e_1\ bop\ e_2 \overset{\alpha}{\sim} e'_1\ bop\ e'_2}$$

$$\frac{e_d \overset{\alpha}{\sim} e'_d \qquad e_b \overset{\alpha}{\sim} e'_b}{\texttt{const}\ x = e_d\,;\ e_b \overset{\alpha}{\sim} \texttt{const}\ x = e'_d\,;\ e'_b} \qquad e \overset{\alpha}{\sim} e \qquad \frac{e' \overset{\alpha}{\sim} e}{e \overset{\alpha}{\sim} e'} \qquad \frac{e \overset{\alpha}{\sim} e'' \qquad e'' \overset{\alpha}{\sim} e'}{e \overset{\alpha}{\sim} e'}$$

With the definition of α-equivalence in place, we can now formally define the condition that a general substitution function must satisfy so that we always ensure the substitution property.

Definition 3.2. *A* substitution function *for Expr is a function*

$$s : Expr \times Var \times Expr \to Expr$$

such that for all $e, e_s, e_r \in Expr$ and $x \in Var$, if $e \overset{\alpha}{\sim} e_r$ and $bv(e_r) \cap fv(e_s) = \emptyset$ then

$$s(e, x, e_s) \overset{\alpha}{\sim} subst(e_r, x, e_s)$$

One can show that a general substitution function always exists, provided that *Var* is infinite (i.e., we never run out of variables for renaming). However, it is somewhat cumbersome to formally define a specific substitution function that satisfies the above definition. The problem is that a concrete substitution function requires us to make a deterministic choice how we pick fresh variables for α-renaming. When one implements substitution functions in an interpreter or compiler, the easiest way to solve this problem is to maintain a global counter that can be used to generate fresh variable names when they are needed.

Whenever we will use substitutions $e_r[e_s/x]$ throughout the remainder of the course, we will always satisfy the condition that the expression e_s is closed. Hence, we do not have to worry about variable capturing and we can just use the function *subst* to compute the substitution. Nevertheless, it is important to understand the concepts of variable capturing and consistent renaming.

Summary. Think of a substitution $e[e_s/x]$ as a way of eliminating an external dependency to x in the expression e by replacing it with another expression e_s. For instance, if e is your program and x is the name of a library function that you are calling in e, you can eliminate the dependency of your program on x by replacing all free occurrences of x in e by the definition of x in the library.

When you compute a substitution $e[e_s/x]$, you want to preserve the result of the evaluation of e with respect to an environment where x is bound to the value obtained by evaluating e_s. This property is called the substitution property. There are two things to keep in mind when computing $e[e_s/x]$:

(1) only free occurrences of x in e are replaced by e_s

(2) if e_s has free variables, these variables should remain free in each context in e where a free using occurrence of x is replaced by e_s. To ensure this, you may have to rename some of the bound variables in e using fresh names so that they do not clash with the free variables of e_s.

Note that when you compute a substitution $e[e_s/x]$, you should *never* rename a free occurrence of a variable in e. For example, we have:

$$(y + x)[y/x] = (y + y)$$

In this example, you should not rename y by some other variable (e.g., to obtain $z+y$ as the result). Here, we have $e = (y+x)$ and y occurs free in e. So y should not be renamed. If you rename a free variable occurrence, you may change the evaluation result and violate the substitution property (e.g., because you change the name of an external library function that you are calling, so after renaming you now call a completely different function). On the other hand, we have

$$(\textbf{const}\ y = 3; y + x)[y/x] = \textbf{const}\ z = 3; z + y$$

Here, the bound occurrence of y must be renamed to avoid capturing of the free occurrence of y in the expression that is substituted in for x.

Chapter 4

Semantics

The semantics of a programming language assigns meaning to programs. Typically, one distinguishes three types of semantics: *denotational semantics*, *axiomatic semantics*, and *operational semantics*. Denotational semantics assigns to each program expression the mathematical object that it computes. We have already seen an example of denotational semantics, namely the *eval* function that maps an arithmetic expression to the mathematical number that the expression evaluates to. Axiomatic semantics, on the other hand, describes the meaning of programs in terms of its effect on assertions about the program's states. It makes statements of the form: if P holds in a program state, then Q holds in the new state after a command c has been executed. The main use of axiomatic semantics is to formally prove that a program satisfies its specification expressed in a formal logic. Finally, operational semantics focuses on the computational aspects of a programming language. It is the type of semantics that is best suited for studying programming language interpreters. We will therefore focus our attention on operational semantics.

The operational semantics of a language describes how a program is evaluated, one step at a time. Typically, it is defined using structural recursion over the syntax of the language. One therefore also speaks of *structural operational semantics*, or *SOS* for short. We distinguish between *big-step* and *small-step* SOS. We will discuss both of these variants, starting with big-step SOS.

4.1 Big-Step Structural Operational Semantics

The big-step SOS of a language defines what values a given expression in a language may evaluate to. It is perhaps the most natural type of semantics for studying the operational aspects of a language because it provides a blueprint for an interpreter of the language that almost immediately translates into an actual implementation. For this reason, some people also refer to the big-step SOS as the *natural semantics*.

4.1.1 Defining the Big-Step SOS

We first extend our language from Sec. 3.3 with new programming constructs to make it feel more like a realistic subset of JavaScript. Specifically, we introduce:

- a new type of values, Booleans, along with Boolean expressions for conjunctions e_1 `&&` e_2 and disjunctions e_1 `||` e_2;
- comparison operators for equality e_1 `===` e_2 and disequality e_1 `!==` e_2; and
- conditional expressions e_1 `?` e_2 `:` e_3, which evaluate to e_2 if e_1 is true and e_3, otherwise.

The abstract syntax of our new language is as follows:

$$
\begin{array}{rcll}
x \in \mathit{Var} & & & \text{variables} \\
n \in \mathit{Num} & & & \text{numbers} \\
b \in \mathit{Bool} & ::= & \texttt{true} \mid \texttt{false} & \text{Booleans} \\
v \in \mathit{Val} & ::= & n \mid b & \text{values} \\
e \in \mathit{Expr} & ::= & v \mid x \mid e_1\, \mathit{bop}\, e_2 \mid \textbf{const}\ x \texttt{=} e_d; e_b & \text{expressions} \\
 & & \mid e_1 \texttt{ ? } e_2 \texttt{ : } e_3 & \\
\mathit{bop} \in \mathit{Bop} & ::= & \texttt{+} \mid \texttt{*} \mid \texttt{\&\&} \mid \texttt{||} \mid \texttt{===} \mid \texttt{!==} & \text{binary operators}
\end{array}
$$

Note that we introduce a new syntactic class *Val* that consists of numbers and Booleans. We call the elements of *Val* *values*. A value is an expression that does not require further evaluation.

We now define the big-step structural operational semantics of our language. As for the *eval* function from the previous chapter, the big-step semantics will be defined with respect to a value environment *env* that assigns values to the free variables of the expression under evaluation. We modify the type of environments to be partial mappings of variables to values to accommodate the fact that values include both numbers and Booleans:

$$\mathit{env} \in \mathit{Env} = \mathit{Var} \rightharpoonup \mathit{Val}$$

This is a subtle but important change: from now on, all computations in our language will produce values, which are themselves expressions of our language. That is, we will view computation as rewriting of syntactic expressions.

The definition of the big-step SOS will look similar to the *eval* function that we have seen in Chapter 3. In particular, it will still give meaning to expressions by mapping them to values and it will still be defined using structural recursion on the abstract syntax of expressions. One important difference to the *eval* function is that the big-step SOS is a relation rather than a function. This means that the semantics may allow for non-determinism, i.e., one expression e may evaluate to more than one value. Although, we will not make use of this feature, a non-deterministic semantics is often useful to give more freedom to a language implementer. For example, the semantics of many programming

languages do not determine the order in which expressions are evaluated. This gives a compiler or interpreter for the language more room for applying program optimizations. If expression can have side-effects (such as the post-increment operator x`++` that is supported by many languages), then changing the order in which an expression is evaluated may change the result of the evaluation.

We formalize the big-step SOS as a relation

$$_ \vdash _ \Downarrow _ \subseteq \mathit{Env} \times \mathit{Expr} \times \mathit{Val}$$

A *judgment form* $\mathit{env} \vdash e \Downarrow v$ states that under value environment env, the expression e evaluates to value v. We define this evaluation relation by providing one or more inference rules for each syntactic construct in our language. The complete set of rules is given in Figure 4.1. We discuss them one at a time.

We start with values v, which by their very nature do not need to be further evaluated. This is captured by the following simple inference rule:

$$\begin{array}{l}\textsc{EvalVal}\\ \mathit{env} \vdash v \Downarrow v\end{array}$$

We give each inference rule a name, which we put above the rule. Next are variables, which are assigned to values by the environment, yielding the following inference rule:

$$\textsc{EvalVar}\quad \frac{x \in \mathsf{dom}(\mathit{env})}{\mathit{env} \vdash x \Downarrow \mathit{env}(x)}$$

We move on to the arithmetic operators `+` and `*` for which the semantics is getting more interesting. Since our language has two types of values, numbers and Booleans, we can write expressions such as `1 + false`. The question is: how should we evaluate such expressions? There is no clear answer to this question and it often depends on choices made in the overall language design. Most languages associate types to expressions that govern how such mixed expressions should be treated. Some languages, including Scala, provide typing rules that strictly forbid expressions such as `1 + false` to be even considered for evaluation. These typing rules can be enforced statically at compile time or dynamically at run time. Other languages such as JavaScript have no strict typing rules. Instead, they define *type coercions* that determine how values of one type are converted to values of another type. We will study typed languages later. For now, we follow JavaScript's treatment of expressions with mixed types. To do so, we introduce two coercion functions that convert values of type *Bool* to

values of type *Num* and vice versa:

$$
\begin{aligned}
\mathit{toNum} &: \mathit{Val} \to \mathit{Num} \\
\mathit{toNum}(n) &= n \\
\mathit{toNum}(\texttt{true}) &= 1 \\
\mathit{toNum}(\texttt{false}) &= 0 \\
\mathit{toBool} &: \mathit{Val} \to \mathit{Bool} \\
\mathit{toBool}(b) &= b \\
\mathit{toBool}(n) &= \textbf{if } n = 0 \textbf{ then } \texttt{false} \textbf{ else } \texttt{true}
\end{aligned}
$$

These two functions are consistent with the type coercions between numbers and Booleans performed by JavaScript.

We can now define the semantics of the operator `+` by the following rule:

$$
\textsc{EvalPlus}\quad
\frac{\mathit{env} \vdash e_1 \Downarrow v_1 \qquad \mathit{env} \vdash e_2 \Downarrow v_2 \qquad v = \mathit{toNum}(v_1) + \mathit{toNum}(v_2)}{\mathit{env} \vdash e_1 \texttt{+} e_2 \Downarrow v}
$$

Note how the coercion function *toNum* is used to convert the intermediate values v_1 and v_2 to numbers so that the result of the addition operation is well-defined. The semantics of the operator `*` is defined accordingly by a rule called EvalTimes.

One important detail that you need to understand about the rule EvalPlus is that this rule does not specify the order in which the two subexpressions e_1 and e_2 are evaluated. Although the premise $\mathit{env} \vdash e_1 \Downarrow v_1$ comes before the premise $\mathit{env} \vdash e_2 \Downarrow v_2$ in the rule, the order in which these two premises are checked is unspecified. The behavior of the rule is therefore non-deterministic and it is up to the implementer of the rule to decide which order should be used. Note that evaluation order is often important in practice. For example, if the evaluation of e_1 and e_2 has side-effects, then the evaluation order matters. If the operational semantics should define a specific evaluation order, e.g. left-to-right, then one needs to switch to a small-step SOS, which we will discuss in Section 4.2.

Next, we define the semantics of constant declarations, which closely follows the semantics that we used in the *eval* function:

$$
\textsc{EvalConstDecl}\quad
\frac{\mathit{env} \vdash e_d \Downarrow v_d \qquad \mathit{env}' = \mathit{env}[x \mapsto v_d] \qquad \mathit{env}' \vdash e_b \Downarrow v_b}{\mathit{env} \vdash \textbf{const } x \texttt{=} e_d; e_b \Downarrow v_b}
$$

We next move on to the comparison operators, which we define in terms of mathematical equality of values:

$$
\textsc{EvalEqual}\quad
\frac{\mathit{env} \vdash e_1 \Downarrow v_1 \qquad \mathit{env} \vdash e_2 \Downarrow v_2 \qquad b = (v_1 = v_2)}{\mathit{env} \vdash e_1 \texttt{===} e_2 \Downarrow b}
$$

Again, the order in which the subexpressions are evaluated is not specified by the rule. The semantics of `!==` is given accordingly by a rule EvalDisequal.

The semantics of conditional expressions e_1 `?` e_2 `:` e_3 is now straightforward. To simplify the presentation, we introduce two rules, one for the case where e_1 evaluates to `true` (after coercion) and one for the case where it evaluates to `false`:

$$\textsc{EvalIfThen}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad toBool(v_1) = \texttt{true} \qquad env \vdash e_2 \Downarrow v_2}{env \vdash e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \Downarrow v_2}$$

$$\textsc{EvalIfElse}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad toBool(v_1) = \texttt{false} \qquad env \vdash e_3 \Downarrow v_3}{env \vdash e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \Downarrow v_3}$$

Finally, we define the semantics of the Boolean operators `&&` and `||`. There are some subtleties in how Boolean operators are treated in programming languages. We start simple and give a preliminary semantics in which we treat the Boolean operators as if they were logical operators. For this purpose, we define the two logical operators for conjunction, $\wedge$, and disjunction, $\vee$, on Booleans as follows:

$$\begin{aligned}
\wedge &: Bool \times Bool \to Bool \\
\texttt{true} \wedge b &= b \\
\texttt{false} \wedge b &= \texttt{false} \\
\vee &: Bool \times Bool \to Bool \\
\texttt{true} \vee b &= \texttt{true} \\
\texttt{false} \vee b &= b
\end{aligned}$$

The semantics of the operator `&&` is then given by the following inference rule:

$$\textsc{EvalAnd}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad b_1 = toBool(v_1) \qquad env \vdash e_2 \Downarrow v_2 \qquad b_2 = toBool(v_2)}{env \vdash e_1 \texttt{ \&\& } e_2 \Downarrow b_1 \wedge b_2}$$

The semantics of `||` is given in terms of $\vee$ by a corresponding rule EvalOr.

4.1.2 Short-Circuit Evaluation

Our preliminary semantics of the Boolean operators that we formalized in the rules EvalAnd and EvalOr does not capture the actual implementation of these operators in JavaScript and most other programming languages. Boolean operators are typically implemented more efficiently. Observe that the operator $\wedge$ has an interesting property: the term $\texttt{false} \wedge b_2$ is `false` no matter whether b_2 is `true` or `false`. This allows us to optimize the evaluation of e_1 `&&` e_2 as follows. If e_1 evaluates to `false`, then there is no point in continuing evaluation of e_2. We can immediately return the result of evaluating e_1 as the result of the entire

$$\textsc{EvalVal}\quad env \vdash v \Downarrow v$$

$$\textsc{EvalVar}\quad \frac{x \in \mathsf{dom}(env)}{env \vdash x \Downarrow env(x)}$$

$$\textsc{EvalPlus}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad env \vdash e_2 \Downarrow v_2 \qquad v = toNum(v_1) + toNum(v_2)}{env \vdash e_1 \texttt{+} e_2 \Downarrow v}$$

$$\textsc{EvalTimes}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad env \vdash e_2 \Downarrow v_2 \qquad v = toNum(v_1) \cdot toNum(v_2)}{env \vdash e_1 \texttt{*} e_2 \Downarrow v}$$

$$\textsc{EvalConstDecl}\quad \frac{env \vdash e_d \Downarrow v_d \qquad env' = env[x \mapsto v_d] \qquad env' \vdash e_b \Downarrow v_b}{env \vdash \textbf{const}\ x \texttt{=} e_d; e_b \Downarrow v_b}$$

$$\textsc{EvalEqual}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad env \vdash e_2 \Downarrow v_2 \qquad b = (v_1 = v_2)}{env \vdash e_1 \texttt{===} e_2 \Downarrow b}$$

$$\textsc{EvalDisequal}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad env \vdash e_2 \Downarrow v_2 \qquad b = (v_1 \neq v_2)}{env \vdash e_1 \texttt{!==} e_2 \Downarrow b}$$

$$\textsc{EvalIfThen}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad toBool(v_1) = \texttt{true} \qquad env \vdash e_2 \Downarrow v_2}{env \vdash e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \Downarrow v_2}$$

$$\textsc{EvalIfElse}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad toBool(v_1) = \texttt{false} \qquad env \vdash e_3 \Downarrow v_3}{env \vdash e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \Downarrow v_3}$$

$$\textsc{EvalAndFalse}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{false} = toBool(v_1)}{env \vdash e_1 \texttt{\&\&} e_2 \Downarrow v_1}$$

$$\textsc{EvalAndTrue}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{true} = toBool(v_1) \qquad env \vdash e_2 \Downarrow v_2}{env \vdash e_1 \texttt{\&\&} e_2 \Downarrow v_2}$$

$$\textsc{EvalOrTrue}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{true} = toBool(v_1)}{env \vdash e_1 \texttt{||} e_2 \Downarrow v_1}$$

$$\textsc{EvalOrFalse}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{false} = toBool(v_1) \qquad env \vdash e_2 \Downarrow v_2}{env \vdash e_1 \texttt{||} e_2 \Downarrow v_2}$$

Figure 4.1: Inference rules that define the big-step SOS of our expression language

expression. Only if e_1 evaluates to `true`, do we continue evaluation of e_2. We refer to this optimization as *short-circuit* evaluation. Short-circuit evaluation is a useful programming feature. Not only does it safe computation time, it can also help us write more compact code. For example, if e_2 is an expression that can only be safely evaluated if some condition e_1 is satisfied, then we can simply write this as $e_1 \,\texttt{\&\&}\, e_2$. The short-circuit semantics ensures that e_2 will never be evaluated, unless e_1 is `true`.

To formalize the short-circuit semantics of `&&`, we introduce two rules: one for the case where e_1 evaluates to `true` and one were it evaluates to `false`. We start with the second case:

$$\textsc{EvalAndFalse}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{false} = toBool(v_1)}{env \vdash e_1 \,\texttt{\&\&}\, e_2 \Downarrow v_1}$$

One interesting detail of this rule is that the result of the evaluation is not `false` but v_1, which gives `false` when it is coerced to the value type *Bool*. This means that the `&&` operator can actually yield values that are not Booleans. For example, the following expression now evaluates to `true` because $0 \,\texttt{\&\&}\, \texttt{true}$ evaluates to 0:

$$0 \;\texttt{===}\; (0 \,\texttt{\&\&}\, \texttt{true})$$

This might be counter-intuitive but the EvalAndFalse rule faithfully reflects the actual semantics of the `&&` operator in JavaScript.

The case where e_1 evaluates to `true` is covered by the following rule:

$$\textsc{EvalAndTrue}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{true} = toBool(v_1) \qquad env \vdash e_2 \Downarrow v_2}{env \vdash e_1 \,\texttt{\&\&}\, e_2 \Downarrow v_2}$$

Similar to the rule EvalAndFalse, the result of the evaluation of $e_1 \,\texttt{\&\&}\, e_2$ in this case is the uncoerced result value of the subexpression e_2, which may not be a Boolean.

The short-circuit semantics of the operator `||` can be defined correspondingly. The complete set of rules that defines the big-step semantics of our language is given in Figure 4.1

4.1.3 Interpreting Big-Step Derivations

We can use the inference rules of the big-step semantics to build derivation trees for the judgment forms $env \vdash e \Downarrow v$ top to bottom, i.e., starting with the base cases given by the rules EvalVal and EvalVar. The derivations constructed this way represent proofs of valid evaluations. For example, the following derivation proves that $\{x \mapsto 2\} \vdash x + 3 \Downarrow 5$ is a valid evaluation:

$$\textsc{EvalPlus}\ \frac{\textsc{EvalVar}\ \dfrac{x \in \mathsf{dom}(\{x \mapsto 2\})}{\{x \mapsto 2\} \vdash x \Downarrow 2} \qquad \textsc{EvalVal}\ \{x \mapsto 2\} \vdash 3 \Downarrow 3 \qquad 5 = 3 + 2}{\{x \mapsto 2\} \vdash x + 3 \Downarrow 5}$$

We can also build the derivations bottom-up, starting with some concrete judgment form $env \vdash e \Downarrow v$. This is possible because the inference rules are syntax directed. By looking at the syntactic structure of the expression e we always know which rule to apply next. In other words, by reading the inference rules bottom-up, we obtain the blueprint for the implementation of a language interpreter, effectively reconstructing the *eval* function from the inference rules. We explain this through an example.

Suppose we are given the environment

$$env = \{x \mapsto 2\}$$

and we want to evaluate the expression

$$e = x \ ? \ 5 : \texttt{false}$$

Then we can compute the result of the evaluation by filling in the red question marks in the following partial derivation where the ? indicate missing pieces of the derivation that we still need to fill in:

$$\textsc{Eval}? \ \frac{?}{\{x \mapsto 2\} \vdash x \ ? \ 5 : \texttt{false} \Downarrow ?}$$

First, we need to find out which rule we could have possibly applied in the final step of the derivation. Looking at the rules in Figure 4.1 and the shape of expression e, we observe that the only candidates that we have to consider are the rules EvalIfThen and EvalIfElse. Although, we do not yet know which of the two rules we need to use, they have a common premise that continues evaluation with the subexpression e_1. The expression e_1 in the rule matches the subexpression x in e. Thus, we can fill in some of the missing parts of the derivation as follows:

$$\textsc{EvalIf}? \ \frac{\textsc{Eval}? \ \dfrac{?}{\{x \mapsto 2\} \vdash x \Downarrow ?} \qquad ? = \mathit{toBool}(?) \qquad \textsc{Eval}? \ \dfrac{?}{\{x \mapsto 2\} \vdash ? \Downarrow ?}}{\{x \mapsto 2\} \vdash x \ ? \ 5 : \texttt{false} \Downarrow ?}$$

Next, we focus our attention on the evaluation of x. We see that the only rule that can complete this subderivation is the rule EvalVar. This allows us to fill in more of the missing parts:

$$\textsc{EvalIf}? \ \frac{\dfrac{\textsc{EvalVar} \quad x \in \mathsf{dom}(\{x \mapsto 2\})}{\{x \mapsto 2\} \vdash x \Downarrow 2} \qquad ? = \mathit{toBool}(?) \qquad \textsc{Eval}? \ \dfrac{?}{\{x \mapsto 2\} \vdash ? \Downarrow ?}}{\{x \mapsto 2\} \vdash x \ ? \ 5 : \texttt{false} \Downarrow ?}$$

Now, we have obtained 2 as the result of evaluating x in *env*. Applying the function *toBool* to 2 yields `true`. Thus we must use the rule EvalIfThen,

which allows us to fill more of the remaining holes in the derivation:

$$\textsc{EvalIfThen}\ \dfrac{\dfrac{\textsc{EvalVar}\ x \in \mathsf{dom}(\{x \mapsto 2\})}{\{x \mapsto 2\} \vdash x \Downarrow 2} \qquad \texttt{true} = \mathit{toBool}(2) \qquad \dfrac{\textsc{Eval}?\ ?}{\{x \mapsto 2\} \vdash 5 \Downarrow ?}}{\{x \mapsto 2\} \vdash x\ \texttt{?}\ 5\ \texttt{:}\ \texttt{false} \Downarrow ?}$$

To complete the derivation, we observe that the only matching rule for the missing rule application is EvalVal. This allows us to fill in the remaining missing parts:

$$\textsc{EvalIfThen}\ \dfrac{\dfrac{\textsc{EvalVar}\ x \in \mathsf{dom}(\{x \mapsto 2\})}{\{x \mapsto 2\} \vdash x \Downarrow 2} \qquad \texttt{true} = \mathit{toBool}(2) \qquad \textsc{EvalVal}\ \{x \mapsto 2\} \vdash 5 \Downarrow 5}{\{x \mapsto 2\} \vdash x\ \texttt{?}\ 5\ \texttt{:}\ \texttt{false} \Downarrow 5}$$

Thus, the result of evaluating e in environment *env* is 5.

4.2 Small-Step Operational Semantics

One drawback of the big-step structural operational semantics is that it does faithfully model all the details of a programming language's semantics. In particular, the inductive definition of the big-step SOS given in the previous section only captures computations that can be described by finite derivation trees constructed from the inference rules. That is, it only captures computations that eventually terminate. The behavior of infinite computations is left undefined. This is not an issue for the simple language that we have considered so far because the evaluation of any expression in this language is guaranteed to terminate. However, this will no longer be true once we extend the language with functions.

Another aspect that is not captured by our big-step semantics is the order in which expressions are evaluated. In a programming language where the evaluation of expressions may have side effects, the evaluation order matters. If a language fixes a specific evaluation order, then this should be reflected in the operational semantics. For example, in JavaScript all binary operators are evaluated left to right.

One way to capture these finer details of a language's semantics is to switch to a *small-step* SOS. This type of SOS is more fine-grained and can model execution order and non-terminating computations. It is often the preferred style of semantics when one is interested in proving properties about a programming language as a whole.

4.2.1 Step-Wise Expression Evaluation

We will be using the same language as for the big-step SOS in the previous section. We will formalize the small-step SOS by a step relation that iteratively

simplifies a given expression until a value has been obtained. For example, the expression

$$(2+8)*3$$

will be simplified as follows:

$$(\underline{2+8})*3 \to \underline{10*3} \to 30$$

In each step, a subexpression that can be simplified in a single step is chosen in the current expression e. Then that subexpression is simplified and the result substituted back into the complete expression e. In the above example, the subexpressions that are chosen for simplification in the next step are underlined.

We will again define the semantics using syntax-directed inference rules. These rules can be categorized into two types:

- *"Do" rules*: these rules perform an actual simplification step. The second step in our example corresponds to the application of a do rule: we simplified the expression $10*3$ by multiplying the two values, yielding the value 30.
- *"Search" rules*: these rules select the next subexpressions that should be simplified and then apply the corresponding do rule. The first step in our example corresponds to the application of a search rule: we selected the subexpression $2+8$ in the expression $(2+8)*3$ and performed the do rule for that subexpression, yielding $10*3$.

The step relation will still be depending on a value environment, as we still have to be able to assign values to free variables occuring in an expression. That is, formally the small-step SOS is a relation

$$_ \vdash _ \to _ \subseteq \mathit{Env} \times \mathit{Expr} \times \mathit{Expr}$$

Observe the difference to the big-step SOS, which related expressions and the final values of the computation. Instead, the small-step SOS relates expressions with expressions that capture the intermediate results of the computation. A judgment form $\mathit{env} \vdash e \to e'$ states that under value environment env, the expression e' is obtained from e by a single simplification step. The do rules are summarized in Figure 4.2 and the search rules in Figure 4.3. We discuss the two sets of rules in tandem.

First, note that there is no rule for values. By definition a value cannot be simplified any further. Once simplification of an expression yields a value, the evaluation of that expression terminates. In fact, we will define the small-step SOS in such a way that if for some environment env and expression e, there exists no expression e' such that $\mathit{env} \vdash e \to e'$, then e must be a value. That is, whenever the small-step evaluation terminates, we obtain a value.

Our first do rule concerns variables. The rule is almost identical to the EvalVar rule:

$$\textsc{DoVar}\quad \frac{x \in \mathsf{dom}(\mathit{env})}{\mathit{env} \vdash x \to \mathit{env}(x)}$$

$$\textsc{DoVar}\quad \frac{x \in \mathsf{dom}(env)}{env \vdash x \to env(x)}$$

$$\textsc{DoPlus}\quad \frac{v = toNum(v_1) + toNum(v_2)}{env \vdash v_1 \texttt{+} v_2 \to v}$$

$$\textsc{DoTimes}\quad \frac{v = toNum(v_1) \cdot toNum(v_2)}{env \vdash v_1 \texttt{*} v_2 \to v}$$

$$\textsc{DoEqual}\quad \frac{b = (v_1 = v_2)}{env \vdash v_1 \texttt{===} v_2 \to b}$$

$$\textsc{DoDisequal}\quad \frac{b = (v_1 \neq v_2)}{env \vdash v_1 \texttt{!==} v_2 \to b}$$

$$\textsc{DoConstDecl}\quad env \vdash \textbf{const}\ x \texttt{=} v_d; v_b \to v_b$$

$$\textsc{DoIfThen}\quad \frac{toBool(v_1) = \texttt{true}}{env \vdash v_1 \texttt{ ? } e_2 \texttt{ : } e_3 \to e_2}$$

$$\textsc{DoIfElse}\quad \frac{toBool(v_1) = \texttt{false}}{env \vdash v_1 \texttt{ ? } e_2 \texttt{ : } e_3 \to e_3}$$

$$\textsc{DoAndFalse}\quad \frac{\texttt{false} = toBool(v_1)}{env \vdash v_1 \texttt{ \&\& } e_2 \to v_1}$$

$$\textsc{DoAndTrue}\quad \frac{\texttt{true} = toBool(v_1)}{env \vdash v_1 \texttt{ \&\& } e_2 \to e_2}$$

$$\textsc{DoOrTrue}\quad \frac{\texttt{true} = toBool(v_1)}{env \vdash v_1 \texttt{ || } e_2 \to v_1}$$

$$\textsc{DoOrFalse}\quad \frac{\texttt{false} = toBool(v_1)}{env \vdash v_1 \texttt{ || } e_2 \to e_2}$$

Figure 4.2: Do rules for the small-step SOS of our expression language

$$\textsc{SearchBop1}\quad \frac{env \vdash e_1 \to e_1'}{env \vdash e_1\, bop\, e_2 \to e_1'\, bop\, e_2}$$

$$\textsc{SearchBop2}\quad \frac{bop \notin \{\texttt{\&\&}, \texttt{||}\} \qquad env \vdash e_2 \to e_2'}{env \vdash v_1\, bop\, e_2 \to v_1\, bop\, e_2'}$$

$$\textsc{SearchConstDecl1}\quad \frac{env \vdash e_d \to e_d'}{env \vdash \textbf{const}\ x \texttt{=} e_d; e_b \to \textbf{const}\ x \texttt{=} e_d'; e_b}$$

$$\textsc{SearchConstDecl2}\quad \frac{env' = env[x \mapsto v_d] \qquad env' \vdash e_b \to e_b'}{env \vdash \textbf{const}\ x \texttt{=} v_d; e_b \to \textbf{const}\ x \texttt{=} v_d; e_b'}$$

$$\textsc{SearchIf}\quad \frac{env \vdash e_1 \to e_1'}{env \vdash e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \to e_1' \texttt{ ? } e_2 \texttt{ : } e_3}$$

Figure 4.3: Search rules for the small-step SOS of our expression language

The arithmetic expressions $e_1 + e_2$ and $e_1 * e_2$ are handled by a combination of search and do rules. The search rules enforce that first e_1 and then e_2 are completely evaluated to values. Then a do rule performs the actual computation. We first give the do rule for +:

$$\textsc{DoPlus}\quad \frac{v = \mathit{toNum}(v_1) + \mathit{toNum}(v_2)}{\mathit{env} \vdash v_1 + v_2 \to v}$$

The important point is that the rule only applies to addition of values $v_1 + v_2$ but not expressions $e_1 + e_2$ where e_1 and e_2 still need to be evaluated, i.e., where either $e_1 \notin \mathit{Val}$ or $e_2 \notin \mathit{Val}$ holds. The premise that v_1 and v_2 are values is stated implicitly through the choice of the names of these meta variables. We could make this premise explicit by writing the DoPlus rule like this:

$$\textsc{DoPlus}\quad \frac{e_1, e_2 \in \mathit{Val} \quad v = \mathit{toNum}(e_1) + \mathit{toNum}(e_2)}{\mathit{env} \vdash e_1 + e_2 \to v}$$

The two DoPlus rules are equivalent. For notational convenience, we will continue to state the domains over which meta variables in rules range by choosing appropriate names for these meta variables.

The cases where one of the subexpressions of $e_1 + e_2$ has not yet been reduced to a value are handled by appropriate search rules that we discuss next. We want to define the search rules in such a way that they enforce a left-to-right evaluation order for an expression $e_1 + e_2$. We do this by defining two separate rules. The first rule is

$$\textsc{SearchPlus1}\quad \frac{\mathit{env} \vdash e_1 \to e_1'}{\mathit{env} \vdash e_1 + e_2 \to e_1' + e_2}$$

This rule takes an expression $e_1 + e_2$, recursively applies a single simplification step in the subexpression e_1, which yields a simpler subexpression e_1', and then returns the overall simpler expression $e_1' + e_2$. Note that the premise of the rule, $\mathit{env} \vdash e_1 \to e_1'$, implies that e_1 is not a value (because values cannot be further simplified).

Once e_1 has been fully evaluated, a second search rule takes over to evaluate e_2. This second rule is as follows:

$$\textsc{SearchPlus2}\quad \frac{\mathit{env} \vdash e_2 \to e_2'}{\mathit{env} \vdash v_1 + e_2 \to v_1 + e_2'}$$

Note that the rule only applies if the first subexpression is already a value. The premise of the rule again implies that e_2 is not yet a value.

The complete evaluation of an expression $e_1 + e_2$ then works as follows. First, the rule SearchPlus1 iteratively reduces e_1 to a value v_1, obtaining the expression $v_1 + e_2$. Then the rule SearchPlus2 iteratively reduces e_2 to a value v_2,

obtaining the expression $v_1 + v_2$. Finally, the rule DoPlus simplifies $v_1 + v_2$ to the sum of v_1 and v_2. Note that for any expression $e_1 + e_2$ only one of the three rules applies, i.e., the evaluation order is completely deterministic. This is in contrast to the big-step SOS, where the evaluation order was not fixed.

The semantics of the operator `*` is captured by an according do rule called DoTimes and search rules that are almost identical to the rules SearchPlus1 and SearchPlus2. In fact, all binary operators except the short-circuiting ones are evaluated left-to-right. Hence, we can generalize the search rules SearchPlus1 and SearchPlus2 so that they also handle all these other operators:

$$\textsc{SearchBop1}\quad \frac{env \vdash e_1 \to e_1'}{env \vdash e_1\, bop\, e_2 \to e_1'\, bop\, e_2} \qquad \textsc{SearchBop2}\quad \frac{bop \notin \{\texttt{\&\&}, \texttt{||}\} \qquad env \vdash e_2 \to e_2'}{env \vdash v_1\, bop\, e_2 \to v_1\, bop\, e_2'}$$

Note that the rule SearchBop2 is restricted to non-short-circuiting operators. For the operators `&&` and `||` we have two additional do rules that reduce the expression to the second operand in the non-short-circuiting case:

$$\textsc{DoAndTrue}\quad \frac{\texttt{true} = toBool(v_1)}{env \vdash v_1 \,\texttt{\&\&}\, e_2 \to e_2} \qquad \textsc{DoOrFalse}\quad \frac{\texttt{false} = toBool(v_1)}{env \vdash v_1 \,\texttt{||}\, e_2 \to e_2}$$

Due to these rules, we do not need search rules that reduce the second operand for `&&` and `||`.

We do not discuss the do rules of the remaining binary operators as they are straightforward adaptations of the corresponding rules from the big-step SOS.

Next, we discuss the rules for constant declarations **const** $x = e_d; e_b$. Again, we will need to split the evaluation into three rules, one do rule and two search rules. We start with the search rules. The first search rule handles the evaluation of the defining expression e_d:

$$\textsc{SearchConstDecl1}\quad \frac{env \vdash e_d \to e_d'}{env \vdash \textbf{const}\ x = e_d; e_b \to \textbf{const}\ x = e_d'; e_b}$$

The second rule evaluates the body expression e_b, but it only applies once evaluation of e_d has been completed:

$$\textsc{SearchConstDecl2}\quad \frac{env' = env[x \mapsto v_d] \qquad env' \vdash e_b \to e_b'}{env \vdash \textbf{const}\ x = v_d; e_b \to \textbf{const}\ x = v_d; e_b'}$$

Note that this rule does not yet eliminate the actual declaration of x, even though its defining expression has already been reduced to a value. We need to keep the declaration alive while e_b is still being reduced in order to be able to compute the updated environment env'. The final elimination of the constant declaration is done by the following do rule:

$$\textsc{DoConstDecl}\quad env \vdash \textbf{const}\ x = v_d; v_b \to v_b$$

The rules for conditional expressions e_1 `?` e_2 `:` e_3 are again straightforward.

Chapter 5

Procedural Abstraction

Procedures are a central feature of almost all programming languages. A procedure allows us to abstract from the specifics of a particular computation by parameterizing over the concrete values used in the evaluation of an expression. This way, we can describe similar computations without having to write the code for the common steps of these computations repeatedly in the program. Moreover, by allowing procedures to call themselves, we obtain recursion. As we have seen, recursion is the key to describing *unbounded* computations where the number of computation steps to be executed depends on the input values.

You will often hear the terms *procedure* and *function* being used synonymously in programming language jargon. Some people consider *functions* to be the more general concept and think of a procedure as a special case of a function that has no return value. In the setting of functional programming languages, where every procedure has a return value, this distinction does not make sense. Here, some people use the term *procedure* for the general concept and call a procedure *function* only if it has no side effects. To avoid any confusions with our terminology, we will exclusively use the term *function* from now on.

We will study functions in their most general form. Namely, we consider functions to be just another type of expressions in our language. That is, a function may occur anywhere in our program where an expression may occur. Specifically, we allow functions to occur both as arguments and return values of other functions. We will see that this generality enables some powerful programming techniques.

5.1 Functions and Function Calls

We extend our language with function expressions of the form:

$$x \texttt{ => } e$$

We refer to such expressions as *anonymous functions* because the function expression itself is not bound to a name. We call x the *(formal) parameter* of the

function expression and e the *body* of the function expression. Note that the function expression binds the parameter variable x. The scope of this binding is the body e.

In order to be able to use functions, we additionally introduce *function call* expressions, e_1`(`e_2`)`, where e_1 must evaluate to a function expression and e_2 evaluates to the argument that is passed to that function. We will use the term *function application* synonymously to function call.

Intuitively, a call expression (x `=>` e_1)(e_2) evaluates to expression e_1 with all free occurrences of x replaced by e_2. For example, consider the following (mathematical) function *plusTwo*:

$$plusTwo : Num \to Num$$
$$plusTwo(x) = x + 2$$

suppose we want to express this function in our new language in order to compute the value $plusTwo(3) = 5$. We can do this in our extended language as follows:

```
const plusTwo = x => x + 2;
plusTwo(3)
```

Note that we consider the "arrow" operator for constructing anonymous functions to have the lowest precedence of all operators. In particular, `x => x + 2` should be read as `x => (x + 2)` and not `(x => x) + 2`.

Remark. With anonymous functions and function calls alone, we cannot express recursive functions directly. Surprisingly, this does not restrict the expressivity of our language. That is, even without explicit recursive functions, we can still encode recursion indirectly. In fact, one of the fundamental results of computability theory states that any mathematical function that is computable (by a Turing machine) can be expressed in the subset of our new language that consists only of variables, function expressions, and function calls. This restricted subset of our language is referred to as the *lambda calculus*. If you are interested in learning more about this result, you can read Section 5.6. Due to the connection to the lambda calculus, function expressions are also often referred to as *lambda abstractions*.

For the convenience of writing recursive functions succinctly, we will also add named function expressions to our language. These take the form

function x_1`(`x_2`)` e

where x_1 refers to the name of the function, x_2 is the formal parameter, and e is the body. The scope of both x_1 and x_2 is restricted to e.

Here is how we write a program in our extended language that computes the factorial of `5` using a recursive definition of the factorial function:

```
const factorial =
```

```
  function factorial(n)
    (n > 1 ? n * factorial(n - 1) : 1);
factorial(5)
```

We assume that the additional operators `>` and `-` are interpreted as expected.

In our treatment of recursive functions **function** x_1`(`x_2`)` e we deviate slightly from JavaScript syntax. First, for the sake of simplicity, we restrict the scope of x_1 to the function body e. This explains why in the example above, the name `factorial` is bound twice: once by **function** `factorial(n)` to define the function, and once by **const** `factorial` to make the function name available for subsequent calls in the program, i.e. in `factorial(5)`. Second, we allow the body of a recursive function to be a single expression whose value determines the return value of the function, just like for anonymous functions. In actual JavaScript syntax, the body of a recursive function is a statement and the function's result value is determined by a **return** statement in the body.

Here is how the factorial example would look like in proper JavaScript syntax:

```
function factorial(n) {
  return n > 1 ? n * factorial(n - 1) : 1
}
factorial(5)
```

Note that in JavaScript, we do not need to bind the recursive function again to its name using a `const` declaration in order to make it callable by its name outside of its body. Instead, the scope of the name of a recursive function includes both the body of the function and the remainder of the statement block where the function is declared.

We will sometimes write examples in actual JavaScript syntax so that you can easily play with the code using a JavaScript interpreter.

The abstract syntax of our new language is given by the following grammar. The new constructs are highlighted.

$x \in \mathit{Var}$		variables
$n \in \mathit{Num}$		numbers
$b \in \mathit{Bool}$	$::=$ `true` $\mid$ `false`	Booleans
$v \in \mathit{Val}$	$::= n \mid b \mid$ **function** p`(`x`)` $e \mid$ typeerror	values
$e \in \mathit{Expr}$	$::= v \mid x \mid e_1\ \mathit{bop}\ e_2 \mid$ **const** x `=` e_d; e_b	expressions
	$\mid e_1$ `?` e_2 `:` $e_3 \mid e_1$`(`e_2`)`	
$\mathit{bop} \in \mathit{Bop}$	$::=$ `+` $\mid$ `*` $\mid$ `&&` $\mid$ `||` $\mid$`===`$\mid$`!==`	binary operators
p	$::= x \mid \epsilon$	function names

For uniformity of representation, we write **function** p`(`x`)` e for both anonymous and named functions, and separately introduce a case allowing p to be either ϵ, in the case of an anonymous function, and some variable x, in the case of a named function. In examples, we will use the more compact syntax x `=>` e for anonymous functions.

Note that function expressions are values just like numbers and Booleans. We will see that this has some surprising consequences. In our concrete syntax, we assume that function application is left-associative, i.e., the expression $f(x)(y)$ is equivalent to $(f(x))(y)$.

In addition to function expressions and function calls, we also introduce the value typeerror. This value is not part of the concrete syntax of our language. We will use typeerror as a placeholder value for evaluating certain expressions when we define the operational semantics of our new language constructs.

5.2 Currying and Partial Function Application

In our new language, a function expression **function** $p(x)\ e$ only allows a single parameter x. At first, this seems overly restrictive because we cannot write functions that take more than one parameter. For example, we cannot directly express the simple function *plus* that takes two numbers and adds them:

$$\begin{aligned} \mathit{plus} &: \mathit{Num} \times \mathit{Num} \to \mathit{Num} \\ \mathit{plus}(x, y) &= x + y \end{aligned}$$

However, allowing functions to only have one parameter does not represent a fundamental restriction to the expressivity of our language, because we can redefine *plus* as a cascaded function that first takes x and then takes y, instead of taking both values simultaneously:

$$\begin{aligned} \mathit{plus} &: \mathit{Num} \to (\mathit{Num} \to \mathit{Num}) \\ \mathit{plus}(x)(y) &= x + y \end{aligned}$$

Observe that *plus* is now a function that takes a number x and returns another function $\mathit{plus}(x) : \mathit{Num} \to \mathit{Num}$. This returned function takes a number y and returns the number $x + y$. The transformation of functions that take more than one parameter into cascaded functions that each take one parameter is referred to as *currying*[1]. We also call the second version of *plus* a *curried function*. The grammar of our language allows the body e of a function expression **function**$(x)\ e$ to be itself a function expression. Thus, we can express the curried version of *plus* in our language as follows:

```
const plus = x => y => x + y;
plus(2)(3)
```

This program will evaluate to 5 as expected.

One useful feature of curried functions is that we can apply them partially. For example, we can express the function *plusTwo* that adds the value 2 to its argument using the function *plus* as follows:

[1] The term *currying* is derived from the name of the logician Haskell Curry, who popularized this technique.

```
const plus = x => y => x + y;
const plusTwo = plus(2);
plusTwo(3) + plusTwo(2)
```

This program will evaluate to `9`.

5.3 Operational Semantics of Function Calls

We now formalize the operational semantics of our extended language. We will focus on big-step SOS. Once the rules for big-step semantics are understood, it is straightforward to use them to define small-step SOS.

We discuss two different semantics of function calls: one with *dynamic binding* and one with *static binding*. We start with the former as it is slightly simpler to implement.

5.3.1 Environment-based Semantics with Dynamic Binding

First, note that we defined function expressions as values of our language. That is, by our definition of values, a function expression **function** p(x) e should not be further evaluated. In fact, from our old rule EvalVal for evaluating values, we immediately obtain

$$env \vdash \texttt{function}\ p(x)\ e \Downarrow \texttt{function}\ p(x)\ e$$

For a function expression to have any computational effect, we must call it. Thus, we have to define inference rules for function calls. We first handle the case of a call expression $e_1(e_2)$ where e_1 evaluates to an anonymous function:

EvalCall

$$\frac{v_1 = \texttt{function}(x)\ e \qquad \begin{array}{c} env \vdash e_1 \Downarrow v_1 \\ env \vdash e_2 \Downarrow v_2 \end{array} \qquad env' = env[x \mapsto v_2] \qquad env' \vdash e \Downarrow v}{env \vdash e_1(e_2) \Downarrow v}$$

The rule follows our intuition of how a function call $e_1(e_2)$ should work. We first evaluate the callee e_1 and only if the resulting value v_1 is a function expression **function**(x) e can we continue. Then we evaluate the argument expression e_2 of the call, yielding value v_2. Next, we bind the parameter x to the argument v_2 to obtain a new environment env'. Finally, we evaluate the body, e, of the function under the new environment to obtain the final result value v of the call.

Note that the rule does not actually specify the order in which the callee e_1 and its argument e_2 are evaluated. The actual evaluation order of call expressions in JavaScript is left-to-right. However, we need to switch to a small-step SOS to capture this detail.

The rule EvalCall does capture one detail in the execution order of call expressions: the argument e_2 is evaluated before we evaluate the body expression

e. This semantics of function calls is referred to as *call-by-value*. This is in contrast to a semantics where e_2 is evaluated each time x is used inside of e. We refer to the latter form as *call-by-name* semantics. In Chapter 7, we will discuss the different variants of function call semantics in more detail.

The generalization of the rule EvalCall to calls that involve recursive functions is now almost obvious:

$$\textsc{EvalCallRec}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad v_1 = \texttt{function}\ x_1\texttt{(}x_2\texttt{)}\ e \qquad env \vdash e_2 \Downarrow v_2 \qquad env' = env[x_1 \mapsto v_1][x_2 \mapsto v_2] \qquad env' \vdash e \Downarrow v}{env \vdash e_1\texttt{(}e_2\texttt{)} \Downarrow v}$$

In addition to binding the parameter x_2 to the argument value v_2 of the call, we also bind the name of the function x_1 to the function expression v_1 itself. Any recursive calls to x_1 inside of e will then be properly resolved during evaluation.

5.3.2 Dynamic Type Errors

Before we added functions and function calls to our language, all expressions could be evaluated to some value, thanks to automatic type conversion. With functions, we encounter one of the very few run-time errors in JavaScript: trying to call something that is not a function. In JavaScript, calling a non-function raises a run-time error. In the formal semantics, we model this using the placeholder value typeerror:

$$\textsc{EvalTypeErrorCall}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad v_1 \neq \texttt{function}\ p\texttt{(}x\texttt{)}\ e}{env \vdash e_1\texttt{(}e_2\texttt{)} \Downarrow \mathsf{typeerror}}$$

Such a run-time error is known as *dynamic type error*. Languages are called dynamically typed when they allow all syntactically valid programs to run and only check for type errors during program execution.

Once we have encountered a dynamic type error somewhere during evaluation, there is no way to recover from it. We model this by adding propagation rules that propagate type errors encountered in subexpressions to the top-level. For example, the following rule propagates a type error while evaluating the defining expression in a constant declaration:

$$\textsc{EvalPropConst}\quad \frac{env \vdash e_d \Downarrow \mathsf{typeerror}}{env \vdash \texttt{const}\ x\texttt{=}e_d\texttt{;}\ e_b \Downarrow \mathsf{typeerror}}$$

The complete set of type error (propagation) rules is summarized in Figure 5.2.

The other rules of the big-step semantics are given in Figure 5.1. Note that in the rules EvalEqual and EvalDisequal, we disallow equality and disequality checks (i.e., === and !==) on function expressions. If either argument to these two operators is a function expression, then we consider this a dynamic type error. This design decision is reflected by the type error rules

EvalTypeErrorEqual$_1$ and EvalTypeErrorEqual$_2$ of Figure 5.2. The choice of disallowing equality between functions is a slight departure from the semantics of JavaScript. In JavaScript, equality between functions is reduced to equality of pointers to memory locations where the implementations of the involved functions are stored. We cannot model this notion of equality faithfully in our current semantics because we don't have an explicit memory (yet).

5.3.3 Dynamic vs. Static Binding

Recall that when we defined the binding structure and scoping rules of **const** declarations **const** $x = e_d$; e_b, we required that every free using occurrence of x in e_b should be bound to the value of e_d. We referred to this type of binding as *static binding*. That is, by looking at the syntactic structure of a program we can determine which value each variable refers to. Unfortunately, with the big-step semantics for function calls that we have given above, we no longer correctly implement static binding.

In order to understand this issue, consider the following program:

```
1 const x = 2;
2 const plusTwo = y => x + y;
3 const f = x => plusTwo(x);
4 f(3)
```

With static binding, the using occurrence of `x` on line 2 should always refer to the defining occurrence of `x` on line `1`. That is, this occurrence of `x` should always evaluate to `2`. We would therefore expect that the program above evaluates to `5`. However, if we evaluate the program according to the rules of our big-step semantics, we obtain the value `6`.

The problem is the rule EvalCall. When we evaluate the call `f(3)`, then the parameter `x` of `f` is bound to the value `3`, overwriting the current binding of `x` to `2` in the environment. Thus, when we subsequently look-up the value of `x` in the updated environment during the call to `plusTwo` inside of `f`, we retrieve `3` instead of `2`.

We refer to this semantics of variable bindings as *dynamic binding*. The problem with dynamic binding is that the actual binding of variables to values depends on how a program is evaluated. It cannot be derived statically from the syntactic structure of the program. Consequently, programs that are evaluated under dynamic binding semantics often have unintuitive behavior that is difficult to debug. All modern programming languages therefore use static binding instead of dynamic binding by default. The use of dynamic binding in some early programming languages is now considered a historical mistake.

The problem with dynamic binding is that before the call to `plusTwo` on line 3 is evaluated, the variable `x` is rebound to the value 3 that is passed into `f` on line 4. Thus, we evaluate the body of `plusTwo` in the environment $[\mathsf{x} \mapsto 3, \mathsf{y} \mapsto 3]$, yielding 6 as the overall result of the evaluation.

We now modify our operational semantics so that we obtain the desired static binding semantics.

$$\textsc{EvalVal}\quad env \vdash v \Downarrow v$$

$$\textsc{EvalVar}\quad \frac{x \in \mathsf{dom}(env)}{env \vdash x \Downarrow env(x)}$$

$$\textsc{EvalPlus}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad env \vdash e_2 \Downarrow v_2 \qquad v = toNum(v_1) + toNum(v_2)}{env \vdash e_1 \texttt{+} e_2 \Downarrow v}$$

$$\textsc{EvalTimes}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad env \vdash e_2 \Downarrow v_2 \qquad v = toNum(v_1) \cdot toNum(v_2)}{env \vdash e_1 \texttt{*} e_2 \Downarrow v}$$

$$\textsc{EvalConstDecl}\quad \frac{env \vdash e_d \Downarrow v_d \qquad env' = env[x \mapsto v_d] \qquad env' \vdash e_b \Downarrow v_b}{env \vdash \textbf{const}\ x \texttt{=} e_d; e_b \Downarrow v_b}$$

$$\textsc{EvalEqual}\quad \frac{\begin{array}{c} env \vdash e_1 \Downarrow v_1 \qquad env \vdash e_2 \Downarrow v_2 \\ b = (v_1 = v_2) \qquad v_1 \neq \texttt{function}\ p_1\texttt{(}x_1\texttt{)}\ e_3 \qquad v_2 \neq \texttt{function}\ p_2\texttt{(}x_2\texttt{)}\ e_4 \end{array}}{env \vdash e_1 \texttt{===} e_2 \Downarrow b}$$

$$\textsc{EvalDisequal}\quad \frac{\begin{array}{c} env \vdash e_1 \Downarrow v_1 \qquad env \vdash e_2 \Downarrow v_2 \\ b = (v_1 \neq v_2) \qquad v_1 \neq \texttt{function}\ p_1\texttt{(}x_1\texttt{)}\ e_3 \qquad v_2 \neq \texttt{function}\ p_2\texttt{(}x_2\texttt{)}\ e_4 \end{array}}{env \vdash e_1 \texttt{!==} e_2 \Downarrow b}$$

$$\textsc{EvalIfThen}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad toBool(v_1) = \texttt{true} \qquad env \vdash e_2 \Downarrow v_2}{env \vdash e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \Downarrow v_2}$$

$$\textsc{EvalIfElse}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad toBool(v_1) = \texttt{false} \qquad env \vdash e_3 \Downarrow v_3}{env \vdash e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \Downarrow v_3}$$

$$\textsc{EvalAndFalse}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{false} = toBool(v_1)}{env \vdash e_1 \texttt{ \&\& } e_2 \Downarrow v_1}$$

$$\textsc{EvalAndTrue}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{true} = toBool(v_1) \qquad env \vdash e_2 \Downarrow v_2}{env \vdash e_1 \texttt{ \&\& } e_2 \Downarrow v_2}$$

$$\textsc{EvalOrTrue}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{true} = toBool(v_1)}{env \vdash e_1 \texttt{ || } e_2 \Downarrow v_1}$$

$$\textsc{EvalOrFalse}\quad \frac{env \vdash e_1 \Downarrow v_1 \qquad \texttt{false} = toBool(v_1) \qquad env \vdash e_2 \Downarrow v_2}{env \vdash e_1 \texttt{ || } e_2 \Downarrow v_2}$$

$$\textsc{EvalCall}\quad \frac{\begin{array}{c} env \vdash e_1 \Downarrow v_1 \\ v_1 = \texttt{function}(x)\ e \qquad env \vdash e_2 \Downarrow v_2 \qquad env' = env[x \mapsto v_2] \qquad env' \vdash e \Downarrow v \end{array}}{env \vdash e_1\texttt{(}e_2\texttt{)} \Downarrow v}$$

$$\textsc{EvalCallRec}\quad \frac{\begin{array}{c} env \vdash e_1 \Downarrow v_1 \qquad v_1 = \texttt{function}\ x_1\texttt{(}x_2\texttt{)}\ e \\ env \vdash e_2 \Downarrow v_2 \qquad env' = env[x_1 \mapsto v_1][x_2 \mapsto v_2] \qquad env' \vdash e \Downarrow v \end{array}}{env \vdash e_1\texttt{(}e_2\texttt{)} \Downarrow v}$$

$$\textsc{EvalTypeErrorEqual}_1 \quad \frac{env \vdash e_1 \Downarrow \texttt{function}\ p(x)\ e \qquad bop \in \{\texttt{===}, \texttt{!==}\}}{env \vdash e_1\ bop\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalTypeErrorEqual}_2 \quad \frac{env \vdash e_2 \Downarrow \texttt{function}\ p(x)\ e \qquad bop \in \{\texttt{===}, \texttt{!==}\}}{env \vdash e_1\ bop\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalTypeErrorCall} \quad \frac{env \vdash e_1 \Downarrow v_1 \qquad v_1 \neq \texttt{function}\ p(x)\ e}{env \vdash e_1\texttt{(}e_2\texttt{)} \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropIf} \quad \frac{env \vdash e_1 \Downarrow \mathsf{typeerror}}{env \vdash e_1\ \texttt{?}\ e_2\ \texttt{:}\ e_3 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropBop}_1 \quad \frac{env \vdash e_1 \Downarrow \mathsf{typeerror}}{env \vdash e_1\ bop\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropBop}_2 \quad \frac{env \vdash e_2 \Downarrow \mathsf{typeerror} \quad bop \notin \{\texttt{\&\&}, \texttt{||}\}}{env \vdash e_1\ bop\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropAnd} \quad \frac{env \vdash e_1 \Downarrow v_1 \qquad v_1 \neq \mathsf{typeerror} \qquad \texttt{true} = toBool(v_1) \qquad env \vdash e_2 \Downarrow \mathsf{typeerror}}{env \vdash e_1\ \texttt{\&\&}\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropOr} \quad \frac{env \vdash e_1 \Downarrow v_1 \qquad v_1 \neq \mathsf{typeerror} \qquad \texttt{false} = toBool(v_1) \qquad env \vdash e_2 \Downarrow \mathsf{typeerror}}{env \vdash e_1\ \texttt{||}\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropCall}_1 \quad \frac{env \vdash e_1 \Downarrow \mathsf{typeerror}}{env \vdash e_1\texttt{(}e_2\texttt{)} \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropCall}_2 \quad \frac{env \vdash e_2 \Downarrow \mathsf{typeerror}}{env \vdash e_1\texttt{(}e_2\texttt{)} \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropConstDecl} \quad \frac{env \vdash e_d \Downarrow \mathsf{typeerror}}{env \vdash \texttt{const}\ x\texttt{=}e_d\texttt{;}\ e_b \Downarrow \mathsf{typeerror}}$$

Figure 5.2: Big-step operational semantics: dynamic type error rules

5.3.4 Substitution-based Semantics with Static Binding

We consider a simple substitution-based semantics to reestablish static binding. This semantics is not used in practice, because it is computationally inefficient. However, it is easy to understand and implement. In the following, we first use a small-step semantics to introduce the intuition behind substitution-based semantics but only provide the inference rules for a big-step semantics. However, everything we discuss equally works for a small-step and a big-step semantics.

For our new semantics, we assume that we start our evaluation with an expression e that is closed. That is, e does not contain any free variables: $fv(e) = \emptyset$. The semantics then maintains this property throughout the entire evaluation of e, i.e., during evaluation we will only have to deal with closed expressions. The new semantics will therefore no longer need an environment to store the values bound to the free variables that occur in the evaluated expression.

In order to avoid the introduction of free variables during the evaluation of subexpressions, we have to eliminate free variables as soon as they appear in subexpressions as evaluation passes through a variable binding construct, such as **const** declarations or function expressions. These are the points during evaluation where substitutions come into play. For example, when we now evaluate a **const** declaration **const** $x = e_d; e_b$, we first evaluate the defining expression e_d of x as before to obtain a result value v_d. Though, we no longer store the binding of v_d to x in an environment for later retrieval during the evaluation of the body expression e_b. Instead, we use a substitution $e_b[v_d/x]$ to replace all free occurrences of x in e_b in a single step. Then we continue evaluation with the expression resulting from the substitution. This expression is again closed. Moreover, the substitution realizes the intended static binding semantics of variables.

We demonstrate this using the example program from the previous section:

```
const x = 2;
const plusTwo = y => x + y;
const f = x => plusTwo(x);
f(3)
```

The evaluation of this program with the substitution-based small-step semantics now looks as follows:

```
const x = 2;
const plusTwo = y => x + y;
const f = x => plusTwo(x);
f(3)
->
const plusTwo = y => 2 + y;
const f = x => plusTwo(x);
f(3)
->
const f = x => (y => 2 + y)(x);
```

```
f(3)
-> (x => (y => 2 + y)(x))(3)
-> (y => 2 + y)(3)
-> 2 + 3
-> 5
```

Note that the final value that we compute is `5` instead of `6`, which was the value that we obtained with the dynamic-binding semantics.

Let us now formalize the substitution-based semantics in terms of inference rules. We do this using a big-step semantics. The most drastic change compared to the dynamic binding semantics that we discussed in Section 5.3.1 is that we eliminate value environments from the evaluation relation. That is, we define a new relation $e \Downarrow v$ that relates expressions and values directly. For this to work, we must require that the expressions e that we consider for evaluation are all closed, i.e., they contain no free variables $fv(e) = \emptyset$.

The rules that define the new big-step semantics are summarized in Figures 5.3 and 5.4. The crucial changes are in the rules EvalConstDecl, EvalCall, and EvalCallRec, i.e., the rules that previously modified the environment by introducing new variable bindings. Moreover, the rule EvalVar for evaluating free variables is no longer needed, since we do not have free variables in the evaluated expressions by assumption. The remaining rules, which used to simply pass the environment along for evaluating subexpressions are almost unchanged. We only removed the environments. We discuss the rules EvalConstDecl, EvalCall, and EvalCallRec in detail, starting with the rule EvalConstDecl:

$$\textsc{EvalConstDecl}\quad \frac{e_d \Downarrow v_d \quad e'_b = e_b[v_d/x] \quad e'_b \Downarrow v}{\texttt{const}\ x = e_d\texttt{;}\ e_b \Downarrow v}$$

As before, the rule evaluates the defining expression e_d, obtaining some value v_d. Now, recall that the old rule bound x to v_d in an environment to evaluate the body e_b of the **const** declaration. Instead, the new rule simply calculates a new expression e'_b that is obtained from e_b by substituting all free occurrences of x in e_b by the value v_d. We can think of this substitution as performing, in a single step, all recursive applications of the rule EvalVar when evaluating e_b in the old environment-based semantics. Recall from Section 3.3.3 that we defined the substitution $e_b[v_d/x]$ in such a way that it respects the static binding structure of the expression. That is, only the free occurrences of x in e_b will be substituted. This way, we obtain the desired static binding semantics.

The rules EvalCall and EvalCallRec for evaluating function calls are

adapted in a similar fashion:

$$\textsc{EvalCall}\quad \frac{e_1 \Downarrow v_1 \qquad v_1 = \texttt{function}(x)\, e \qquad e_2 \Downarrow v_2 \qquad e' = e[v_2/x] \qquad e' \Downarrow v}{e_1\texttt{(}e_2\texttt{)} \Downarrow v}$$

$$\textsc{EvalCallRec}\quad \frac{\begin{array}{c} e_1 \Downarrow v_1 \\ v_1 = \texttt{function}\, x_1\texttt{(}x_2\texttt{)}\, e \qquad e_2 \Downarrow v_2 \qquad e' = e[v_2/x_2][v_1/x_1] \qquad e' \Downarrow v \end{array}}{e_1\texttt{(}e_2\texttt{)} \Downarrow v}$$

The remaining evaluation rules are straight-forward adaptations of the corresponding rules in the dynamic binding semantics.

Exercise 5.1. *Reconsider the problematic example program:*

```
const x = 2;
const plusTwo = y => x + y;
const f = x => plusTwo(x);
f(3)
```

Verify that this program indeed evaluates to `5` *under the new substitution-based big-step semantics.*

5.4 Higher-Order Functions

In Section 5.2, we have seen that the body of a function expression can itself be a function expression. This observation led us to the notion of curried functions. What about call expressions $e_1\texttt{(}e_2\texttt{)}$? We require that e_1 must be a function expression for a call expression to be meaningful. However, we impose no syntactic constraints on the argument expression e_2 of the call. In particular, we allow e_2 to be again a function expression. We refer to functions that can be applied to other functions as *higher-order functions*. If you want to become a better programmer, learning how to use higher-order functions effectively is an important step.

5.4.1 Abstracting from Computations

We start with a simple example to motivate the usefulness of higher-order functions. Suppose that we want to write a function `sumInts` that takes the bounds `a` and `b` of a (half-open) interval of integer numbers `[a,b)` and computes the sum of the values in that interval. For example, `sumInts(1)(4)` should yield `6`. The following recursive implementation does what we want:

```
function sumInts(a) {
  return b => a < b ? a + sumInts(a+1)(b) else 0;
}
```

EvalVal
$$v \Downarrow v$$

EvalPlus
$$\frac{e_1 \Downarrow v_1 \qquad e_2 \Downarrow v_2 \qquad v = toNum(v_1) + toNum(v_2)}{e_1 \texttt{ + } e_2 \Downarrow v}$$

EvalTimes
$$\frac{e_1 \Downarrow v_1 \qquad e_2 \Downarrow v_2 \qquad v = toNum(v_1) \cdot toNum(v_2)}{e_1 \texttt{ * } e_2 \Downarrow v}$$

EvalConstDecl
$$\frac{e_d \Downarrow v_d \qquad e_b' = e_b[v_d/x] \qquad e_b' \Downarrow v_b}{\textbf{const}\ x \texttt{ = } e_d \texttt{;}\ e_b \Downarrow v_b}$$

EvalEqual
$$\frac{e_1 \Downarrow v_1 \qquad e_2 \Downarrow v_2 \qquad b = (v_1 = v_2) \qquad v_1 \neq \texttt{function}\ p_1 \texttt{(}x_1\texttt{)}\ e_3 \qquad v_2 \neq \texttt{function}\ p_2 \texttt{(}x_2\texttt{)}\ e_4}{e_1 \texttt{ === } e_2 \Downarrow b}$$

EvalDisequal
$$\frac{e_1 \Downarrow v_1 \qquad e_2 \Downarrow v_2 \qquad b = (v_1 \neq v_2) \qquad v_1 \neq \texttt{function}\ p_1 \texttt{(}x_1\texttt{)}\ e_3 \qquad v_2 \neq \texttt{function}\ p_2 \texttt{(}x_2\texttt{)}\ e_4}{e_1 \texttt{ !== } e_2 \Downarrow b}$$

EvalIfThen
$$\frac{e_1 \Downarrow v_1 \qquad toBool(v_1) = \texttt{true} \qquad e_2 \Downarrow v_2}{e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \Downarrow v_2}$$

EvalIfElse
$$\frac{e_1 \Downarrow v_1 \qquad toBool(v_1) = \texttt{false} \qquad e_3 \Downarrow v_3}{e_1 \texttt{ ? } e_2 \texttt{ : } e_3 \Downarrow v_3}$$

EvalAndFalse
$$\frac{e_1 \Downarrow v_1 \qquad \texttt{false} = toBool(v_1)}{e_1 \texttt{ \&\& } e_2 \Downarrow v_1}$$

EvalAndTrue
$$\frac{e_1 \Downarrow v_1 \qquad \texttt{true} = toBool(v_1) \qquad e_2 \Downarrow v_2}{e_1 \texttt{ \&\& } e_2 \Downarrow v_2}$$

EvalOrTrue
$$\frac{e_1 \Downarrow v_1 \qquad \texttt{true} = toBool(v_1)}{e_1 \texttt{ || } e_2 \Downarrow v_1}$$

EvalOrFalse
$$\frac{e_1 \Downarrow v_1 \qquad \texttt{false} = toBool(v_1) \qquad e_2 \Downarrow v_2}{e_1 \texttt{ || } e_2 \Downarrow v_2}$$

EvalCall
$$\frac{e_1 \Downarrow v_1 \qquad v_1 = \texttt{function}(x)\ e \qquad e_2 \Downarrow v_2 \qquad e' = e[v_2/x] \qquad e' \Downarrow v}{e_1 \texttt{(}e_2\texttt{)} \Downarrow v}$$

EvalCallRec
$$\frac{e_1 \Downarrow v_1 \qquad v_1 = \texttt{function}\ x_1 \texttt{(}x_2\texttt{)}\ e \qquad e_2 \Downarrow v_2 \qquad e' = e[v_1/x_1][v_2/x_2] \qquad e' \Downarrow v}{e_1 \texttt{(}e_2\texttt{)} \Downarrow v}$$

Figure 5.3: Inference rules that define the substitution-based big-step SOS for static binding semantics of our language with functions

.

$$\textsc{EvalTypeErrorEqual}_1 \quad \frac{e_1 \Downarrow \texttt{function}\ p(x)\ e \qquad bop \in \{\texttt{===}, \texttt{!==}\}}{e_1\ bop\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalTypeErrorEqual}_2 \quad \frac{e_2 \Downarrow \texttt{function}\ p(x)\ e \qquad bop \in \{\texttt{===}, \texttt{!==}\}}{e_1\ bop\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalTypeErrorCall} \quad \frac{e_1 \Downarrow v_1 \qquad v_1 \neq \texttt{function}\ p(x)\ e}{e_1(e_2) \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropIf} \quad \frac{e_1 \Downarrow \mathsf{typeerror}}{e_1\ \texttt{?}\ e_2\ \texttt{:}\ e_3 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropBop}_1 \quad \frac{e_1 \Downarrow \mathsf{typeerror}}{e_1\ bop\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropBop}_2 \quad \frac{e_2 \Downarrow \mathsf{typeerror} \quad bop \notin \{\texttt{\&\&}, \texttt{||}\}}{e_1\ bop\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropAnd} \quad \frac{e_1 \Downarrow v_1 \qquad v_1 \neq \mathsf{typeerror} \qquad \texttt{true} = toBool(v_1) \qquad e_2 \Downarrow \mathsf{typeerror}}{e_1\ \texttt{\&\&}\ e_2 \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropOr} \quad \frac{v_1 \neq \mathsf{typeerror} \qquad \texttt{false} = toBool(v_1) \qquad \dfrac{e_1 \Downarrow v_1 \quad e_2 \Downarrow \mathsf{typeerror}}{e_1\ \texttt{||}\ e_2 \Downarrow \mathsf{typeerror}}}{}$$

$$\textsc{EvalPropCall}_1 \quad \frac{e_1 \Downarrow \mathsf{typeerror}}{e_1(e_2) \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropCall}_2 \quad \frac{e_2 \Downarrow \mathsf{typeerror}}{e_1(e_2) \Downarrow \mathsf{typeerror}}$$

$$\textsc{EvalPropConstDecl} \quad \frac{e_d \Downarrow \mathsf{typeerror}}{\texttt{const}\ x = e_d\ \texttt{;}\ e_b \Downarrow \mathsf{typeerror}}$$

Figure 5.4: Substitution-based big-step operational semantics: dynamic type error rules

Now, consider the following function `sumSqrs` that computes the sum of the squares of the numbers in an interval `[a,b)`:

```
function sumSqrs(a) {
  return b => a < b ? a * a + sumSqrs(a+1)(b) else 0;
}
```

The functions `sumInts` and `sumSqrs` are almost identical. They only differ in the summand that is added in each recursive call. In the case of `sumInts` it is `a`, and in the case of `sumSqrs`, it is `a * a`. We can write a higher-order function `sum` that abstracts from these differences. The function `sum` takes another function `f` as additional parameter. The function `f` captures the computation that is performed in the summand:

```
function sum(f) {
  return a => b =>
      a < b ? f(a) + sum(f)(a + 1)(b) else 0
}
```

Now we can define `sumInts` and `sumSqrs` more succinctly in terms of `sum` as follows:

```
const sumInts = sum(a => a);
const sumSqrs = sum(a => a * a);
```

5.4.2 Realizing `for` Loops with Higher-Order Functions

The combination of curried functions, partial application, and higher-order functions yields an extremely powerful programming technique that often allows us to express programs much more succinctly than in traditional imperative programming languages such as C. In the following, we discuss an example that uses higher-order functions and currying to extend our language with a loop construct.

Consider the following JavaScript program, which uses a **`for`** loop over an accumulator variable `acc` to compute the factorial of `5`:

```
function factorial(n) {
  var acc = 1;
  for (i = 1; i <= n; i++) {
    acc = acc * i;
  }
  return acc;
}
factorial(5)
```

While our language provides neither **`for`** loops nor mutable **`var`** variables, we can faithfully encode the above implementation of factorial using a curried tail-recursive function:

```
const factorial = n =>
    function loop(i) (
      acc => i <= n ? loop(i + 1)(acc * i) : acc
    )
    (1)(1)
  ;
factorial(5)
```

Since **for** loops are such a useful programming construct, we might want to make this construct available more generally in our language. We can do this by taking the function `loop` out of the body of `factorial` and abstracting over the actual computation performed by that function:

```
const for =
  n => body => init =>
    function loop (i) (acc =>
      i <= n ? loop(i + 1)(body(i)(acc)) : acc
    )(1)(init);
const factorial =
  const body = i => acc => acc * i;
  n => for(n)(body)(1);
factorial(5)
```

The function **for** is a curried higher-order function. The expression

```
  for(n)(body)(init)
```

computes the value that is stored in the variable `acc` when the following JavaScript program is executed:

```
var acc = init;
for (i = 1; i <= n; i++) {
  acc = body(i)(acc);
}
```

5.5 Higher-Order Functions and Collections in Scala

Higher-order functions provide a powerful mechanism for abstracting over common computation patterns in programs. This mechanism is particularly useful for designing libraries with rich interfaces that support callbacks to client code. We will study these mechanisms using the example of Scala's collection libraries.

5.5.1 Higher-Order Functions in Scala

Before we dive into the intricacies of Scala collections, let us first see how higher-order functions are defined in Scala. To this end, we revisit the `sum` function from Section 5.4.1. Here is how we can define this function in Scala:

```
def sum(f: Int => Int, a: Int, b: Int) =
  if a < b then f(a) + sum(f, a + 1, b) else 0
```

The *function type* `Int =>Int` of the parameter `f` indicates that `f` is a function that takes a value of type `Int` and maps it again to an `Int`.

We can now define the function `sumSqrs` by first defining a function `square` that squares an integer number, and then applying `sum` to `square`:

```
def square(i: Int) = i * i
def sumSqrs(a: Int, b: Int) = sum(square, a, b)
```

In order to make the use of higher-order functions more convenient, Scala supports writing anonymous functions (aka function literals), similar to JavaScript. In Scala, anonymous functions take the general form:

```
(x1: T1, ..., xn: Tn) => body
```

where the `xi` are the parameters of the function, the `Ti` are the associated types, and `body` is the body of the function. We can thus define the functions `sumInts` and `sumSqrs` using anonymous functions as follows:

```
def sumInts(a: Int, b: Int) = sum((i: Int) => i, a, b)
def sumSqrs(a: Int, b: Int) = sum((i: Int) => i * i, a, b)
```

5.5.2 Curried Functions in Scala

Reconsider our definition of `sumInts` and `sumSqrs` in terms of `sum`:

```
def sumInts(a: Int, b: Int) = sum((i: Int) => i, a, b)
def sumSqrs(a: Int, b: Int) = sum((i: Int) => i * i, a, b)
```

One annoyance with these definitions is that we have to redeclare the parameters `a` and `b` which are simply passed to `sum`. We can avoid this by redefining `sum` as a curried function that first takes the function `f` applied to the values in the interval and then returns a function that takes the bounds of the interval `a` and `b`.

There are various ways to define curried functions in Scala. One way is to define the nested function explicitly by name using a nested **`def`** declaration and then returning that function:

```
def sum(f: Int => Int): (Int, Int) => Int =
  def sumHelp(a: Int, b: Int): Int =
    if a < b then f(i) + sum(f)(a+1, b) else 0
  sumHelp
```

Using the curried version of `sum` the definition of `sumInts` and `sumSqrs` can be simplified like this:

```
def sumInts = sum((i: Int) => i)
def sumSqrs = sum((i: Int) => i * i)
```

Note that when we apply curried higher-order functions to anonymous functions, then the compiler can often infer the parameter types. This simplifies the definitions even further:

```
def sumInts = sum(i => i)
def sumSqrs = sum(i => i * i)
```

In our curried version of `sum`, the function `sumHelp` is not recursive and is directly returned after being declared. We can thus simplify the definition of `sum` further by turning `sumHelp` into an anonymous function:

```
def sum(f: Int => Int): (Int, Int) => Int =
  (a: Int, b: Int) =>
    if a < b then f(i) + sum(f)(a+1, b) else 0
```

Since curried functions are so common in functional programs, the Scala language provides a special syntax for them. Instead of nesting the function declarations, we can write a curried function by providing the parameters of the nested function in a separate parameter list:

```
def sum(f: Int => Int)(a: Int, b: Int): Int =
  if a < b then f(a) + sum(f)(a, b) else 0
```

If we partially apply a curried function written in this form, we have to make this explicit by appending the underscore symbol _ to the partial application. The definitions of `sumInts` and `sumSqrs` thus look as follows in this case:

```
def sumInts = sum(i => i)_
def sumSqrs = sum(i => i * i)_
```

5.5.3 Higher-Order Functions on Lists

An important use case of higher-order functions is to realize callbacks to client code from within library functions. We discuss this scenario using the specific example of the class `List` in the Scala standard library.

Recall the list data structure that we studied in Section 2.3. We can define lists of integers recursively using an algebraic data type as follows:

```
enum List:
  case Nil
  case Cons(hd: Int, tl: List)
```

That is, a list is either empty, denoted by `Nil`, or a cons cell consisting of an integer `hd` and the tail of the list `tl`.

The generic class `List[A]` in the Scala standard library generalizes this data structure to lists over an arbitrary element type `A`. The empty list is also denoted by `Nil` and a consn cell is denoted `hd :: tl`. We can thus construct lists as follows:

```
scala> val l1 = 1 :: (4 :: (2 :: Nil))
l1: List[Int] = List(1, 4, 2)

scala> val l2 = "apple" :: ("banana" :: Nil)
l2: List[String] = List(apple, banana)
```

Note that the infix cons operator `::` is right-associative, so the parenthesis in the above example can be omitted:

```
scala> val l1 = 1 :: 4 :: 2 :: Nil
l1: List[Int] = List(1, 4, 2)
```

As expected, we can use pattern matching to deconstruct lists into their components:

```
scala> val h :: t = l1
h: Int = 1
t: List[Int] = List(4, 2)

scala> l match
  case Nil => println("l␣is␣empty")
  case hd :: tl => println(s"l's␣head␣is␣$hd.")
l's␣head␣is␣1.
```

Here is how we can define the `append` function from Section 2.3 using Scala's `List` class:

```
def append[A](l1: List[A], l2: List[A]): List[A] =
  l1 match
    case Nil => l2
    case hd :: tl => hd :: append(tl, l2)
```

Note that `append` is a *generic* function that is parameterized by the type `A` of the elements stored in the lists. Here is an example of how to use `append`:

```
scala> append(List(3,4,1), List(2, 6))
res0: List[Int] = List(3,4,1,2,6)
```

Here is how we can define the tail-recursive version of the `reverse` function:

```
def reverse[A](l: List[A]): List[A] =
  def rev(l: List[A], acc: List[A]): List[A] =
    l match
      case Nil => acc
      case hd :: tl => rev(tl, hd :: acc)
  rev(l, Nil)

scala> reverse(List(3,4,1,2))
res0: List[Int] = List(2,1,4,3)
```

From the above examples we can see that functions operating on lists follow a common pattern: they traverse the list, decomposing it into its elements,

and then apply some operation to each of the elements. We can extract this common pattern and implement them in more general higher-order functions that abstract from the specific operations being performed on the elements.

A particularly common operation on lists is to traverse a list and applying some function to each element, obtaining a new list. For example, suppose we have a list of `Double` values which we want to scale by a given factor to obtain a list of scaled values. The following function implements this operation:

```
def scale(factor: Double, l: List[Double]): List[Double] =
  l match
    case Nil => Nil
    case hd :: tl => factor * hd :: scale(factor, tl)
```

A similar operation is implemented by the following function, which takes a list of integers and increments each element to obtain a new list:

```
def incr(l: List[Int]): List[Int] =
  l match
    case Nil => Nil
    case hd :: tl => hd + 1 :: incr(tl)
```

The type of operation that is performed by `scale` and `incr` is called a *map*. We can implement the map operation as a higher-order function that abstracts from the concrete operation that is applied to each element in the list:

```
def map[A, B](l: List[A])(op: A => B): List[B] =
  l match
    case Nil => Nil
    case hd :: tl => op(hd) :: map(tl)(op)
```

The function `map` is parametric in both the element type `A` of the input list, as well as the element type `B` of the output list. That is, a map operation transforms a `List[A]` into a `List[B]` by applying an operation `op: A =>B` to each element in the input list. Note that the order of the elements in the input list is preserved.

We can now redefine `scale` and `incr` as instances of `map`:

```
def scale(factor: Double, l: List[Double]) =
  map(l)(x => factor * x)
def incr(l: List[Int]) = map(l)(x => x + 1)
```

Note that Scala provides a syntactic short form for defining anonymous functions by replacing variables in expressions with underscores. This notation is often useful to obtain succinct code when using higher-order functions. For example, the Scala compiler will automatically expand the following code to the above definitions of `scale` and `incr`:

```
def scale(factor: Double, l: List[Double]) =
  map(l)(factor * _)
def incr(l: List[Int]) = map(l)(_ + 1)
```

5.5.4 Folding Lists

We have seen that we can often identify common patterns in functions on data structures and implement them in generic higher-order functions. We can then conveniently reuse these generic functions, reducing the amount of code we have to write. In this section, we will look at the most general patterns for performing operations on collections, namely *fold operations*.

As a motivating example, consider the following function, which computes the sum of the values stored in a list of integers

```
def sum(l: List[Int]): Int = l match
  case Nil => 0
  case hd :: tl => hd + sum(tl)
```

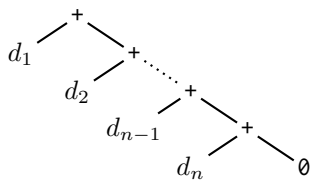
Consider a list `l` of n integer values:

d_1 `::` d_2 `::` `...` `::` d_n `::` `Nil`

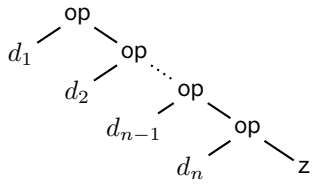
Then unrolling the recursion of `sum` on `l` yields the following computation

d_1 `+ (`d_2 `+ ... (`d_n `+ 0)...)`

That is, in the ith recursive call, we add the current head d_i to the sum of the values in the current tail. Here, we consider the sum of an empty list `Nil` to be 0. If we represent this computation as a tree, this tree looks as follows:



We can now generalize from the specific computation performed by the represented expression. That is, in the general case we have a list of values of type `A` instead of `Int`. Then, instead of the specific initial value `0` for the empty list, we are given an initial value `z` of some type `B`. Finally, instead of adding the current head to the sum of the current tail of the list, we apply a generic operation `op` in each step. The operation `op` takes the current value d_i, which is of type `A`, and the result of the computation on the tail, which is of type `B`, and returns again a value of type `B`. The resulting expanded recursive computation is then represented by the following tree:



We refer to this type of computation as a *fold* of the list because the list is traversed and recursively folded into a single value. Note that the tree is leaning

towards the right. We therefore refer to this type of fold operation as a *fold-right*. That is, the recursive computation is performed in right-to-left order of the values stored in the list.

The following higher-order function implements the fold-right operation:

```
def foldRight[A,B](l: List[A])(z: B)(op: (A, B) => B): B =
  l match
    case Nil => z
    case hd :: tl => op(hd, foldRight(tl)(z)(op))
```

We can now redefine `sum` in terms of `foldRight`:

```
def sum(l: List[Int]): Int = foldRight(l)(0)(_ + _)
```

Many of the other functions that we have seen before perform fold-right operations on lists. In particular, we can define `append` using `foldRight` as follows:

```
def append[A](l1: List[A], l2: List[A]): List[A] =
  foldRight(l1)(l2)(_ :: _)
```

Also the higher-order function `map` is just a special case of a fold-right:

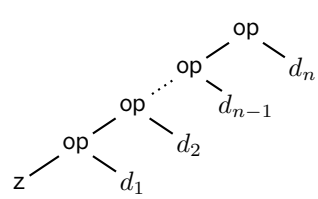
```
def map[A, B](l: List[A])(op: A => B): List[B] =
  foldRight(l)((Nil: List[B]))((h, l) => op(h) :: l)
```

Note that due to limitations of Scala's type inference algorithm, we have to manually annotate the type `List[B]` of the empty list constructor `Nil` that we use to build the resulting list of the map operation.

All the above operations on lists have in common that they combine the elements in the input list and the result of the recursive computation in right-to-left order. We can also consider fold operations that perform the computation in left-to-right order:

`op(op(...(op(op(z,` d_1`),` d_2`), ...),` d_{n-1}`),` d_n`)`

The corresponding computation tree then looks as follows:



Note that the tree is now leaning towards the left and the elements are combined in left-to-right order. We therefore refer to this type of computation as a *fold-left*. The following function implements the generic fold-left operation on lists:

```
def foldLeft[A,B](l: List[A])(z: B)(op: (B, A) => B): B =
  l match
    case Nil => z
    case hd :: tl => foldLeft(tl)(op(z, hd))(op)
```

Since addition is associative and commutative, we can alternatively define `sum` using `foldLeft` instead of `foldRight`:

```
def sum(l: List[Int]): Int = foldLeft(l)(0)(_ + _)
```

In fact, this definition of `sum` is more efficient than our previous implementations because `foldLeft` is tail-recursive, whereas our implementation of `foldRight` is not. In general, only one of the two types of fold operations can be used to implement a specific computation on lists. For example, we can express `reverse` in terms of a fold-left as follows:

```
def reverse[A](l: List[A]): List[A] =
  foldLeft(l)((Nil: List[A]))((l1, x) => x :: l1)
```

If we replaced `foldLeft` by `foldRight` in this definition, we would not obtain the correct result. The computed output list would be structurally identical to the input list.

5.5.5 Scala's Collection Classes

Since higher-order functions on collections are so incredibly useful for writing concise code, the data structures in the Scala standard API already implement a large number of these functions. The functions are realized as methods of the corresponding collection classes. For example Scala's `List` class already provides the methods `foldLeft`, `foldRight`, `map`, etc.

As with any programming language, you should study the Scala standard API carefully so that you get an overview of the provided functionality and so that you do not "reinvent the wheel" when you write your own programs.

To get a glimpse of the expressive power of the functions implemented in the collection classes, consider the following code snippet. The code defines a list of integers and a list of strings and then folds the two lists into a single string. This string consists of a comma separated sequence of strings, where each string is a pair of elements from the two lists concatenated together using the colon symbol:

```
scala> val l1 = List(1, 2, 3)
l1: List[Int] = List(1, 2, 3)

scala> val l2 = List("a", "b","c")
l2: List[String] = List(a, b, c)

scala> ((l1,l2) zipped) map
(_ + ":" + _) reduce
(_ + ",␣" + _)
res0: String = 1:a, 2:b, 3:c
```

It is instructive to re-implement this code snippet in a language such as Java to appreciate how much more concise and comprehensive the functional implementation is.

5.6 Computability (optional)

Computability theory studies the question "What is computable?". The notion of *computability* relies on formal models of computation. Many formal models of computations have been proposed:

- Functions computable by Turing machines (Turing)
- General recursive functions defined by means of an equation calculus (Gödel-Herbrand-Kleene)
- μ-recursive functions and partial recursive functions (Gödel-Kleene)
- Functions defined from canonical deduction systems (Post)
- Functions given by certain algorithms over finite alphabets (Markov)
- Universal Register Machine-computable functions (Shepherdson-Sturgis)
- ...
- Any function you can write in your favorite programming language.

A fundamental result of computability theory is that all of these (and many other) models of computation are equivalent. That is, they give rise to the same class of computable functions. Any such model of computation is said to be *Turing complete*.[2]

So is our programming language Turing-complete? Alan Turing showed this in 1937. More specifically, he showed this for what is now called the *untyped lambda calculus*. This calculus corresponds to the following sub-language of our programming language considered in this section:

$$x \in \mathit{Var} \qquad \text{variables}$$
$$e \in \mathit{Expr} \ ::= x \mid e_1\texttt{(}e_2\texttt{)} \mid x \texttt{ => } e \qquad \text{expressions}$$

That is, expressions in the untyped lambda calculus are restricted to variables, function calls, and anonymous functions. The semantics of the calculus is usually given in terms of a small-step semantics (with static scoping) similar to the rules given in Figure 5.4.

In 1936, Alonzo Church introduced the untyped lambda calculus to prove the nonexistence of a solution to David Hilbert's *Entscheidungsproblem*[3]. This problem asks whether there exists an algorithm that, given a logical formula as input, decides whether the formula is true or false for all possible assignments of values to the formula's free variables. The lambda calculus served as a formalization of what it actually means for something to be an algorithm.

[2] An as of today unproved hypothesis attributed to Alonzo Church and Alan Turing is that there exists no meaningful computational model that is more expressive than a Turing-complete computational model.

[3] German for *decision problem*

Alan Turing proved the same negative result to the Entscheidungsproblem independently shortly after Church using Turing machines as an alternative formal computational model. That is, both the lambda calculus and Turing machines were invented to solve an open problem in logic, several years before the first programmable computer even existed.

How can it be that the simple language of the untyped lambda calculus is enough to express any computable function?

- There are no inbuilt types other than functions (e.g., no Booleans, integers, etc.).
- There are no control constructs like conditional expressions.
- There are no recursive definitions or looping constructs.

It turns out that the expressive power of the calculus stems from the fact that we can treat functions as data. It is all just a matter of finding the right encoding of concepts like numbers, Booleans, and recursion. The encodings presented below are inspired by Church's original work and are therefore also referred to as *Church encodings* in his honor.

5.6.1 Church Encodings

When we talk about data types such as integers and Booleans in a programming language, it is important to distinguish the abstract mathematical concept behind these data types (e.g. integer numbers), from the representation of these concepts in the language (e.g. numerals that stand for integer numbers). For example, consider

- 15,
- fifteen,
- XV, and
- `0F`.

These are different numerals that all represent the same number.

To show that concepts such as integers, Booleans, and the standard operations on these data types can be expressed in the lambda calculus, we simply have to agree on a particular representation of these concepts in terms of anonymous functions. The trick is to pick a representation of these data types that lends itself well to implementing their standard operations using the representation itself for performing the involved computation via function applications.

Before we start, we note that although the lambda calculus does not have **`const`** declarations built in, these can be mimicked by defining them as "syntactic sugar" using function applications like this:

$$\texttt{const}\ x\ \texttt{=}\ e_d\texttt{;}\ e_b \stackrel{\text{def}}{=} \texttt{(}x\ \texttt{=>}\ e_b\texttt{)(}e_d\texttt{)}$$

In the following, we will therefore continue to use **const** declarations for improved readability of code.

Let us start with our encoding of Booleans. How can we represent **true** and **false** in the lambda calculus? One reasonable definition is as follows:

- **true** is a function that takes two values and returns the first, i.e.

```
const true = x => y => x
```

- **false** is a function that takes two values and returns the second, i.e.

```
const false = x => y => y
```

The intuition behind this definition is that we can think of **true** and **false** as implementing the two branches of a conditional expression. That is, if we now define

```
const ite = b => x => y => b(x)(y)
```

then `ite` is a ternary function that implements a conditional expression, provided its first argument `b` is always either **true** or **false** as defined above.

Exercise 5.2. *Convince yourself that*

```
ite(b)(v1)(v2)
```

evaluates to `v1` *if* `b` *is* **true** *and to* `v2` *if* `b` *is* **false**.

Using these definitions, we can now define the standard logical operations on Booleans:

```
const and = a => b => a(b)(false)

const or = a => b => a(true)(b)

const not = b => x => y => b(y)(x)
// alternative: const not = b => b(false)(true)
```

Using a similar idea, one can encode natural numbers using anonymous functions. The number n is represented by a function which maps a successor function s and a zero element z to n applications of s to z: $s(s(\ldots s(z)\ldots))$:

```
const zero = s => z => z

const one = s => z => s(z)

const two = s => z => s(s(z))

const three = s => z => s(s(s(z)))

...
```

Note that `zero` and **`false`** are actually the same functions modulo α-renaming. As is common, we use the same mathematical object to mean different things in different contexts.

Here are the encodings of some operations on natural numbers:

```
// test for zero
const isZero = n => n(x => false)(true)

// addition by one (successor)
const succ = n => s => z => s(n(s)(z))

// addition
const plus = n => m => n(succ)(m)

// multiplication
const mult = n => m => n(plus(m))(zero)

// exponentiation
const exp = m => n => n(m)
```

For example, the definition of `plus` essentially says: apply `succ` `m`-times to `n`. In particular, if we take the expression `plus(three)(two)`, then this reduces to the computation: `succ(succ(succ(two)))`. Since `succ` encodes addition by one, this expression thus computes the encoding of 5. Here is a complete evaluation of the expression `plus(one)(two)`, which shows that the value obtained from this expression is `three`:

```
plus(one)(two)

= (n => m => n(succ)(m))(one)(two) // Definition of plus

→ (n => one(succ)(n))(two) // DoCall

→ one(succ)(two) // DoCall

= (s => z => s(z))(succ)(two) // Definition of one

→ (z => succ(z))(two) // DoCall

→ succ(two) // DoCall

= (n => s => z => s(n(s)(z)))(two) // Definition of succ

→ s => z => s(two(s(z))) // DoCall

= s => z => s((s1 => z1 => s1(s1(z1)))(s)(z)) // Definition of two
```

```
→ s => z => s((z1 => s(s(z1)))(z1)) // "DoCall"

→ s => z => s(s(s(z))) // "DoCall"

= three // Definition of three
```

The other operators work similarly. For instance, definition of `mult` says: apply `plus(n)` `m`-times to `zero`. In particular, the expression `mult(three)(two)` expands to the expression

```
plus(two)(plus(two)(plus(two)(zero)))
```

which reduces to the encoding of the number 6.

Encoding subtraction is a bit more involved. We proceed analogous to the case for addition and first define a function `pred` that encodes subtraction by one. The idea for `pred` is to define a function that given the number n, uses n to enumerate all pairs of numbers $\langle i, i-1\rangle$ for i between 1 and n. Then projection on the second component of the last pair $\langle n, n-1\rangle$ gives us $n-1$.

Let us start by defining functions for constructing and decomposing pairs:

```
// constructing a pair <x,y>
const pair = x => y => b => b(x)(y)

// projecting a pair p=<x,y> to x
const fst = p => p(true)

// projecting a pair p=<x,y> to y
const snd = p => p(false)
```

Note that we have

```
fst(pair(x)(y)) → ... → x
```

and similarly

```
snd(pair(x)(y)) → ... → y
```

Then subtraction by one can be defined as

```
const pred =
  n => snd(n(p => pair(succ(fst(p)))(fst(p)))
            (pair(zero)(zero)))
```

Using `pred` we can now define subtraction

```
const minus = n => m => m(pred)(n)
```

5.6.2 Expressing Recursion

How can we express recursion in the lambda calculus?

As an example, we will implement the factorial function $n!$ in the lambda calculus. Let us start from a standard JavaScript definition of factorial:

```
const fac = function fac(n) (n == 0 ? 1 : n * fac (n - 1))
```

First, we do a literal translation of this definition into the lambda calculus, making use of the operations and constants we have defined so far:

```
const fac = n =>
  ite(isZero(n))(one)(mult(n)(fac(minus(n)(one))))
```

Note that this is not quite right since we still have the call to `fac` in the body of the function definition, which used to be the recursive call to the function itself. Since we do not have recursion built into the lambda calculus, the definition does not work because `fac` now occurs free in the body of the function.

So let us do a simple trick and define a new function `funFac` that takes the function `fac` that we need for the recursive call as an additional input:

```
const facFun = fac => n =>
  ite(isZero(n))(one)(mult(n)(fac(minus(n)(one))))
```

Note that this definition is no longer recursive: the function `facFun` simply delegates the work that needs to be done in the recursive call to a function `fac` that is provided as an additional parameter.

How does this help? We can view `facFun` as a function that constructs the factorial function iteratively. That is, `facFun` takes `fac` as input and if `fac` is a function that approximates the factorial function (i.e. produces the same result as the factorial function on some inputs), then `facFun` returns a new function that is a better approximation of the factorial function than `fac`. To see this, consider the following sequence of definitions:

```
const fac0 = n => zero
const fac1 = facFun(fac0)
const fac2 = facFun(fac1)
...
```

Observe that this sequence of functions approximates the factorial function with increasing precision: $\texttt{fac}_0$ always returns the representation of the number 0, so it never produces the correct result. However, $\texttt{fac}_1$ returns 1 if the input n is 0. So it is already a better approximation than $\texttt{fac}_0$. The function $\texttt{fac}_2$ returns the correct result for $n = 0$ and $n = 1$, etc. In general, for all natural numbers i and $n < i$, the expression $\texttt{fac}_i(n)$ reduces to the Church numeral representing $n!$. If we let i go to infinity, the sequence of approximations $\texttt{fac}_i$ converges to the factorial function itself. Formally, the factorial function `fac` is a function that satisfies the equation:

```
fac = facFun(fac)
```

In other words, we aim to construct `fac` as the *fixpoint* of `facFun`. What we thus need is a lambda expression that serves as a *fixpoint operator*, i.e., a function that calculates for us the fixpoint of another function (here `facFun`). Concretely, we seek a lambda expression `fix` such that the small-step semantics yields a sequence of reduction steps like this:

```
fix(f) → ... → f(fix(f)) → ... → f(f(fix(f))) → ...
```

If we find a term `fix` with this property, we can use it to construct for us automatically on demand the approximation $\texttt{fac}_{n+1}$ that is needed to calculate $n!$ for a given natural number n.

There are various ways to define such a fixpoint operator in the lambda calculus. The most well-known and simplest one is the so-called *Y combinator* due to the logician Haskell Curry. To derive the Y combinator, we start with the following *self-replicating* expression:

```
(x => x(x))(x => x(x))
```

Observe that this expression is "stable" under small-step reductions:

```
  (x => x(x))(x => x(x))
→ (x => x(x))(x => x(x))
→ (x => x(x))(x => x(x))
→ ...
```

That is, the above expression describes an infinite self-replicating computation. Now all we have to do is "squeeze" the function `f` whose fixpoint we want to compute into the self-replicating expression. This yields the Y combinator:

```
const fix = f => (x => f(x(x)))(x => f(x(x)))
```

Observe what happens when we apply `fix` to some function `f`:

```
fix(f)
→ (x => f(x(x)))(x => f(x(x)))          // = fix(f)
→ f((x => f(x(x))(x => f(x(x)))         // = f(fix(f))
→ f(f((x => f(x(x)))(x => f(x(x)))))    // = f(f(fix(f))
→ ...
```

The sequence of reduction steps computes the approximation sequence for the fixpoint of `f` as required!

A minor complication that we still need to solve is that the above definition of the Y combinator does not quite work, yet. The problem is that the evaluation of `fix(f)` diverges: the self-application `x(x)` is repeatedly evaluated before the outermost call to `f` is ever evaluated. This is because the `DoCall` rule requires that the argument expression to a function call is fully evaluated, before the call itself is evaluated.

A simple solution to overcome this problem is to slightly rewrite the Y combinator so that the self-application `x(x)` is "protected" by another anonymous function `(y => f(x(x))(y))`:

```
const fix = f => (x => f(y => x(x)(y)))(x => f(y => x(x)(y)))
```

Effectively, this rewrite accounts for the fact that we we only apply `fix` to functions like `facFun` whose fixpoints are themselves functions.

With the new definition of `fix` we now get

```
fix(f)
→ (x => f(y => x(x)(y)))(x => f(y => x(x)(y)))
→ f(y => (x => f(y => x(x)(y)))(x => f(y => x(x)(y))))
=  f(y => fix(f)(y))
→ ... // continues with the evaluation of the body of f!
```

We now define

```
const fac = fix(facFun)
```

If we now evaluate `fac(two)`, then the value for the parameter `y` of the additional anonymous function in `fix` will be supplied by the argument `minus(two)(one)` of the call to `fac` inside of `facFun`. That is, the fixpoint of `fac` is now computed on-demand and only up to the approximation $\texttt{fac}_{n+1}$ needed for computing `fac(`n`)` for a given n.

Note that the expression `fix(facFun)` is indeed a valid expression in the lambda calculus. In particular, it does not make use of explicit recursion nor does it rely on any inbuilt data types or operations for numbers and Booleans.